Projekt_DeutscheBahn

September 24, 2026

1 Deutsche Bahn: Explorative Analyse historischer Zugverspätungen

1.1 Projektübersicht

Dieses Projekt untersucht einen historischen Datensatz zu Zugverspätungen der Deutschen Bahn. Im Mittelpunkt stehen die Datenqualität, die explorative Analyse von Verspätungsmustern sowie die Bewertung möglicher ergänzender Referenzdaten aus offiziellen DB-APIs.

Die Analyse erfolgt ergebnisoffen. Zunächst wird geprüft, welche Qualität, Vollständigkeit und Aussagekraft der historische Datensatz tatsächlich besitzt.

1.2 Zielsetzung

Ziel des Projekts ist es, historische Verspätungsdaten strukturiert zu untersuchen und deren Eignung für weiterführende Analysen zu bewerten.

1.2.1 Zentrale Forschungsfragen

1. Welche Struktur und Qualität besitzt der historische Datensatz?
2. Welche zeitlichen und regionalen Muster zeigen sich bei den Verspätungen?
3. Welche Datenformate und Frameworks eignen sich für die Verarbeitung?
4. Lassen sich an ausgewählten Knotenpunkten potenzielle Anschlussrisiken erkennen?
5. Welchen zusätzlichen analytischen Nutzen bieten offizielle DB-Referenzdaten?

2 Beschreibung

2.1 Datenquellen

2.1.1 Primärdaten

Der historische Primärdatensatz stammt aus dem Kaggle Open Data Repository und liegt als komprimierte CSV-Datei mit mehr als zwei Millionen Zeilen vor.

Die Datenquelle ist für die Analyse praktisch nutzbar, ihre vollständige Repräsentativität, Konsistenz und zeitliche Einordnung müssen jedoch zunächst überprüft werden.

2.1.2 Optionale Sekundärdaten

Optional werden offizielle Referenzdaten aus dem DB API Marketplace betrachtet. Dazu gehören insbesondere Stations- und Fahrplandaten.

Diese Daten können genutzt werden, um ausgewählte Informationen des historischen Datensatzes stichprobenartig zu spiegeln oder zu ergänzen. Eine vollständige Live-Integration ist für den Projekterfolg nicht erforderlich.

2.1.3 Einschränkungen

- Die Qualität und Vollständigkeit der historischen Daten sind zunächst unbekannt.
- API-Zugriffe erfordern Registrierung und Authentifizierungsdaten.
- Die API-Erweiterung wird nur im verfügbaren Zeitrahmen umgesetzt.
- Ergebnisse beziehen sich auf die tatsächlich vorhandenen Daten und dürfen nicht automatisch auf den gesamten Bahnverkehr verallgemeinert werden.

2.2 Projektphasen

2.2.1 Phase 1: Datenimport und technische Voranalyse

Zu Beginn werden die verfügbaren Daten eingelesen und hinsichtlich ihrer Struktur, Größe und Verarbeitbarkeit untersucht. Dabei werden verschiedene Dateiformate, Kompressionsverfahren und Einlesemöglichkeiten verglichen.

2.2.2 Phase 2: Explorative Datenanalyse

Anschließend werden Datenqualität, Datentypen, fehlende Werte, Verspätungsverteilungen und zeitliche Muster untersucht.

2.2.3 Phase 3: Referenzdaten und Qualitätsabgleich

Optional werden zusätzliche offizielle Referenzdaten beschafft und stichprobenartig mit dem historischen Datensatz verglichen.

2.2.4 Phase 4: Optionale Anschluss- und Netzwerkanalyse

Abhängig von Datenqualität und Projektfortschritt werden mögliche Anschlussrisiken, Knotenpunkte und ausgewählte Netzwerkmuster untersucht.

2.3 Technischer Arbeitsablauf

Das Projekt verwendet eine zentrale Konfigurationsdatei zur Verwaltung aller wichtigen Projektpfade. Dadurch werden Daten, Ergebnisse, Abbildungen sowie erzeugte HTML- und PDF-Dokumente einheitlich abgelegt.

Die Notebook-Dateien greifen auf diese zentral definierten Pfade zu, anstatt Dateinamen und Speicherorte mehrfach direkt im Code festzulegen.

Zusätzlich wird ein separates Hilfsnotebook zur automatisierten Erstellung von HTML- und PDF-Dokumenten verwendet. Dadurch können Analyse-Notebooks als eigenständige Präsentations- und Dokumentationsdateien exportiert werden, ohne die Quelldateien zu verändern.

Die technische Voranalyse wird nur einmal ausgeführt. Das Hauptnotebook verwendet anschließend die bereits erzeugten HTML-, PDF- und Parquet-Dateien. Dadurch muss die teilweise zeitaufwendige Voranalyse nicht erneut ausgeführt werden.

3 Phase 1: Datenimport und technische Voranalyse

[1]:
```
from pathlib import Path
import subprocess
import sys
import matplotlib.pyplot as plt
import plotly.express as px
import os, time
import pandas as pd
import numpy as np

import plotly.graph_objects as go
import plotly.io as pio

from wordcloud import WordCloud

import geopandas as gpd
from shapely.geometry import Point

from dash import (
    Dash,
    html,
    dash_table,
    dcc,
    callback,
    Output,
    Input,
    State
)

from IPython.display import (
    Markdown,
    IFrame,
    display
)

from project_config import (
    get_project_relative_path,
    PROJECT_ROOT,
    PDF_DIR,
    PANDAS_ANALYSIS_HTML,
    PANDAS_ANALYSIS_PDF,
    DATEIFORMATE_RESULTS,
    FRAMEWORK_ANALYSIS_PDF,
    FRAMEWORK_ANALYSIS_HTML,
    FRAMEWORK_RESULTS,
    DASH_HOST,
```

```
    DASH_PORT_DATAFORMAT,
    DASH_URL_DATAFORMAT,
    DASH_PORT_FRAMEWORK,
    DASH_URL_FRAMEWORK,
    ORIGINAL_DS_ZIP_PATH,
    CLEANED_DS_PARQUET_PATH,
    PYSPARK_RESULTS,
    DASH_PORT_PHASE2,
    DASH_URL_PHASE2,
    PYSPARK_ANALYSIS_HTML,
    PYSPARK_ANALYSIS_PDF,
    STADA_API_RESULTS,
    STADA_API_HTML,
    STADA_API_PDF,
    DASH_PORT_PHASE3,
    DASH_URL_PHASE3,
    DASH_PORT_PHASE4,
    DASH_URL_PHASE4,
    TIMETABLE_API_HTML,
    TIMETABLE_API_PDF,
    TIMETABLE_API_RESULTS
)
```

[2]:
```
pio.renderers.default = "jupyterlab+notebook_connected"
```

[3]:
```
assert PANDAS_ANALYSIS_HTML.exists(), (
    f"HTML-Datei nicht gefunden: {PANDAS_ANALYSIS_HTML}"
)

assert PANDAS_ANALYSIS_PDF.exists(), (
    f"PDF-Datei nicht gefunden: {PANDAS_ANALYSIS_PDF}"
)
```

[4]:
```
relative_data_formats_html = get_project_relative_path(PANDAS_ANALYSIS_HTML)
relative_data_formats_pdf = get_project_relative_path(PANDAS_ANALYSIS_PDF)
```

3.1 Data Formats

3.1.1 Ausführliche Ergebnisse und Handlungsempfehlung

Die detaillierte technische Auswertung der untersuchten Datenformate wird im Notebook `01_Dateiformate_Pandas.ipynb` dynamisch erzeugt und in den zugehörigen HTML- und PDF-Dokumenten vollständig dokumentiert.

Die Analyse enthält unter anderem:

- die kleinste gemessene Datei,
- die schnellste Schreibvariante,
- die schnellste Lesevariante,

- Vergleiche zwischen CSV- und Parquet-Formaten,
- den Einfluss von Engines, Codecs und Kompressionsstufen,
- sowie eine technische Entscheidungshilfe für die weitere Verwendung.

Eine erneute Zusammenfassung der Ergebnisse erfolgt an dieser Stelle nicht, um redundante Inhalte zu vermeiden.

[5]:
```
display(Markdown(f"""
### Verknüpfte Analyse-Dokumente

- HTML: `{relative_data_formats_html}`
- PDF: `{relative_data_formats_pdf}`
"""))
```

3.1.2 Verknüpfte Analyse-Dokumente

- HTML: `outputs/html/01_Dateiformate_Pandas.html`
- PDF: `outputs/pdf/01_Dateiformate_Pandas.pdf`

[6]:
```
path_data_formats_html = Path(PANDAS_ANALYSIS_HTML).resolve()
path_data_formats_pdf = Path(PANDAS_ANALYSIS_PDF).resolve()


if not path_data_formats_html.exists():
    print(f"Hinweis: HTML-Datei nicht gefunden: {path_data_formats_html}")

if not path_data_formats_pdf.exists():
    print(f"Hinweis: PDF-Datei nicht gefunden: {path_data_formats_pdf}")
```

[7]:
```
display(Markdown(f"""
### Vollständige technische Data Format Voranalyse

Die vollständige Analyse des Notebooks `01_Dateiformate_Pandas` kann
als PDF-Dokument geöffnet werden:

[PDF der technischen Voranalyse öffnen]({relative_data_formats_pdf})
"""))
```

3.1.3 Vollständige technische Data Format Voranalyse

Die vollständige Analyse des Notebooks `01_Dateiformate_Pandas` kann als PDF-Dokument geöffnet werden:

PDF der technischen Voranalyse öffnen

3.1.4 Technische Voranalyse als HTML

Die folgende Darstellung zeigt die bereits erzeugte HTML-Version der technischen Voranalyse. Sie wird direkt aus der gespeicherten Datei geladen.

Das ursprüngliche Analyse-Notebook muss dafür nicht erneut ausgeführt werden. Dadurch bleiben die längeren Einlese- und Benchmark-Zeiten des ersten Analyseschritts unberücksichtigt.

[8]:
```
display(
    IFrame(
        src=relative_data_formats_html,
        width="100%",
        height=900,
    )
)
```

```
<IPython.lib.display.IFrame at 0x15e3fd369c0>
```

3.1.5 Auswahl der Visualisierungsbibliothek

Im Rahmen der Voranalyse wurden verschiedene Python-Bibliotheken für die grafische Darstellung getestet: Seaborn, Plotly Express und Altair.

Seaborn eignet sich besonders für die schnelle Erstellung statischer Diagramme. Die erzeugten Abbildungen sind gut für Berichte und Präsentationen geeignet, bieten jedoch keine integrierten interaktiven Auswahlmöglichkeiten.

Altair ermöglicht ebenfalls deklarative und interaktive Visualisierungen. Bei der Einbindung in exportierte Dokumente kann die Darstellung jedoch aufwendiger sein. In der PDF-Ausgabe wurden beispielsweise teilweise nur Ausdrücke wie `alt.Chart(...)` angezeigt, wenn die Grafik nicht zuvor als statisches Bild exportiert wurde.

Plotly Express wurde für die interaktive Darstellung der Ergebnisse ausgewählt. Die Diagramme lassen sich direkt in JupyterLab darstellen und eignen sich außerdem als Grundlage für Dash-Anwendungen. Dadurch können Auswahlmöglichkeiten, Filter und interaktive Diagramme in einem gemeinsamen Dashboard verbunden werden.

Für dieses Projekt wird daher folgende Aufgabenteilung verwendet:

- **Seaborn:** schnelle statische Diagramme für explorative Analysen
- **Plotly Express:** interaktive Diagramme und Dashboards
- **Altair:** grundsätzlich möglich, aber für die geplante Dokument- und Dashboard-Integration weniger praktisch

Die interaktive Darstellung wird dabei als Ergänzung verstanden. Die eigentlichen Messergebnisse bleiben unabhängig vom Dashboard in strukturierten Dateien gespeichert.

3.1.6 Interaktives Dashboard zur Voranalyse

Das folgende Dashboard basiert auf den bereits gespeicherten Ergebnissen der Voranalyse `01_Dateiformate_Pandas`.

Die ursprünglichen Benchmark-Messungen müssen nicht erneut ausgeführt werden. Stattdessen wird die vorhandene Parquet-Datei geladen.

Die interaktive Anwendung dient der vertieften Untersuchung bereits berechneter Ergebnisse. Sie ersetzt nicht die technische Dokumentation und führt standardmäßig keine zeitaufwendigen Benchmark-Messungen erneut aus.

Die Varianten können einzeln ein- oder ausgeschaltet werden. Zusätzlich kann zwischen Dateigröße, Schreibzeit und Lesezeit gewechselt werden.

[9]:
```
df_dataformat_results = pd.read_parquet(DATEIFORMATE_RESULTS)
```

[10]:
```
df_dashboard_dataformat = df_dataformat_results.copy()

dataformat_metric_options = [
    {"label": "Dateigröße (MB)", "value": "Größe_MB"},
    {"label": "Schreibzeit (s)", "value": "Schreibzeit_s"},
    {"label": "Lesezeit (s)", "value": "Lesezeit_s"},
]

dataformat_metric_select = dcc.RadioItems(
    id="metric-selector",
    options=dataformat_metric_options,
    value="Lesezeit_s",
    inline=False,
)

dataformat_variant_options = [
    {"label": variant, "value": variant}
    for variant in sorted(df_dashboard_dataformat["Variante"].unique())
]

dataformat_dropdown = dcc.Dropdown(
    id="variant-selector",
    options=dataformat_variant_options,
    value=df_dashboard_dataformat["Variante"].unique().tolist(),
    multi=True,
    searchable=True,
    clearable=False,
    closeOnSelect=False,
    maxHeight=300,
    placeholder="Varianten auswählen ...",
)


table_columns = [
    {"name": "Format", "id": "Format"},
    {"name": "Engine", "id": "Engine"},
    {"name": "Codec", "id": "Codec"},
    {"name": "Level", "id": "Level"},
    {"name": "Größe (MB)", "id": "Größe_MB", "type": "numeric"},
    {"name": "Schreibzeit (s)", "id": "Schreibzeit_s", "type": "numeric"},
    {"name": "Lesezeit (s)", "id": "Lesezeit_s", "type": "numeric"},
    {"name": "Variante", "id": "Variante"},
```

```
]
```

[11]:
```
controls = html.Div(
    [
        html.Div(
            [
                html.Label("Kennzahl auswählen"),
                dataformat_metric_select,
            ],
            style={
                "width": "35%",
                "padding": "10px",
            },
        ),

        html.Div(
            [
                html.Label("Anzuzeigende Varianten"),
                dataformat_dropdown,
            ],
            style={
                "width": "65%",
                "padding": "10px",
            },
        ),
    ],
    style={
        "display": "flex",
        "flexDirection": "row",
        "gap": "20px",
        "alignItems": "flex-start",
        "marginBottom": "20px",
    },
)
```

[12]:
```
dataformat_table = dash_table.DataTable(
    id="dataformat-table",
    columns=table_columns,
    data=df_dashboard_dataformat.to_dict("records"),
    page_size=15,
    sort_action="native",
    filter_action="native",

    style_table={
        "width": "95%",
        "height": "100%",
        "overflowX": "auto",
```

```
    },

    style_cell={
        "fontFamily": "Arial",
        "fontSize": "clamp(11px, 1vw, 14px)",
        "padding": "0.4rem",
        "textAlign": "left",
        "whiteSpace": "normal",
        "height": "auto",
        "minWidth": "8%",
        "maxWidth": "30%",
    },

    style_cell_conditional=[
        {
            "if": {"column_id": "Format"},
            "width": "8%",
        },
        {
            "if": {"column_id": "Engine"},
            "width": "10%",
        },
        {
            "if": {"column_id": "Codec"},
            "width": "10%",
        },
        {
            "if": {"column_id": "Level"},
            "width": "8%",
        },
        {
            "if": {"column_id": "Größe_MB"},
            "width": "12%",
        },
        {
            "if": {"column_id": "Schreibzeit_s"},
            "width": "125px",
        },
        {
            "if": {"column_id": "Lesezeit_s"},
            "width": "30%x",
        },
        {
            "if": {"column_id": "Variante"},
            "width": "30%",
        },
    ],
```

```
    style_header={
        "fontWeight": "bold",
        "backgroundColor": "#e9ecef",
        "whiteSpace": "normal",
        "height": "auto",
    },
)
```

[13]:
```
app_dataformat = Dash(__name__)

app_dataformat.layout = html.Div(
    [
        html.H2("Voranalyse der Datenformate"),
        html.P(
            "Die Messergebnisse können nach Kennzahl und Variante "
            "interaktiv eingeschränkt werden."
        ),
        controls,
        html.Hr(),
        dcc.Graph(
            id="dataformat-graph",
            style={
               "width": "100%",
               "height": "clamp(420px, 60vh, 750px)",
           },
        ),
        html.H3("Tabelle"),
        dataformat_table
    ],
    style={
        "width": "100%",
        "padding": "20px",
        "boxSizing": "border-box",
    },
)
```

[14]:
```
@app_dataformat.callback(
    Output("dataformat-graph", "figure"),
    Output("dataformat-table", "data"),
    Input("metric-selector", "value"),
    Input("variant-selector", "value"),
)
def update_dashboard(selected_metric, selected_variants):

    filtered_df = df_dashboard_dataformat[
        df_dashboard_dataformat["Variante"].isin(selected_variants)
```

```
    ].copy()

    if filtered_df.empty:
        empty_figure = px.scatter(
            title="Keine Variante ausgewählt"
        )

        return empty_figure, []

    figure = px.box(
        filtered_df,
        x="Engine",
        y=selected_metric,
        color="Engine",
        hover_name="Variante",
        points="all",
        title=f"{selected_metric} nach Engine",
        labels={
            selected_metric: selected_metric,
            "Engine": "Engine",
        },
    )

    figure.update_layout(
        uirevision="data-format-dashboard",
        legend_title_text="Engine",
        hovermode="closest",
        margin={
            "l": 60,
            "r": 30,
            "t": 80,
            "b": 60,
        },
    )

    figure.update_xaxes(fixedrange=True)

    return figure, filtered_df.to_dict("records")
```

Dashboard starten

[15]:
```
app_dataformat.run(
    debug=False,
    port=DASH_PORT_DATAFORMAT,
    jupyter_mode="inline",
    jupyter_width="90%",
    jupyter_height=1600,
)
```

```
<IPython.lib.display.IFrame at 0x15e800af740>
```

[16]:
```
display(Markdown(f"""
#### Link für Webbrowser
Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht␣
 ↪werden:

[Dashboard im Browser öffnen]({DASH_URL_DATAFORMAT})

Das Dashboard wird lokal gestartet und ist nur erreichbar,
solange die zugehörige Dash-Anwendung ausgeführt wird.
"""))
```

Link für Webbrowser Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht werden:

Dashboard im Browser öffnen

Das Dashboard wird lokal gestartet und ist nur erreichbar, solange die zugehörige Dash-Anwendung ausgeführt wird.

[17]:
```
relativ_framework_evaluation_pdf =␣
 ↪get_project_relative_path(FRAMEWORK_ANALYSIS_PDF)
relativ_framework_evaluation_html =␣
 ↪get_project_relative_path(FRAMEWORK_ANALYSIS_HTML)
```

3.2 Frameworks

3.2.1 Ausführliche Ergebnisse und Handlungsempfehlung

Die detaillierte technische Auswertung von Frameworsvergleich wird im Notebook `02_framework_evaluation.ipynb` dynamisch erzeugt und in den zugehörigen HTML- und PDF-Dokumenten vollständig dokumentiert.

Als Frameworks für die Analyse wurden:

- Pandas,
- Dask,
- PySpark

genommen.

Eine erneute Zusammenfassung der Ergebnisse erfolgt an dieser Stelle nicht, um redundante Inhalte zu vermeiden.

[18]:
```
display(Markdown(f"""
### Verknüpfte Analyse-Dokumente

- HTML: `{relativ_framework_evaluation_html}`
- PDF: `{relativ_framework_evaluation_pdf}`
"""))
```

3.2.2 Verknüpfte Analyse-Dokumente

- HTML: `outputs/html/02_Framework_Vergleich.html`
- PDF: `outputs/pdf/02_Framework_Vergleich.pdf`

[19]:
```
path_framework_html = Path(FRAMEWORK_ANALYSIS_HTML).resolve()
path_framework_pdf = Path(FRAMEWORK_ANALYSIS_PDF).resolve()


if not path_framework_html.exists():
    print(f"Hinweis: HTML-Datei nicht gefunden: {path_data_formats_html}")

if not path_framework_pdf.exists():
    print(f"Hinweis: PDF-Datei nicht gefunden: {path_data_formats_pdf}")
```

[20]:
```
display(Markdown(f"""
### Vollständige Frameworks Voranalyse

Die vollständige Analyse des Notebooks `02_framework_evaluation` kann
als PDF-Dokument geöffnet werden:

[PDF der technischen Voranalyse öffnen]({relativ_framework_evaluation_pdf})
"""))
```

3.2.3 Vollständige Frameworks Voranalyse

Die vollständige Analyse des Notebooks `02_framework_evaluation` kann als PDF-Dokument geöffnet werden:

PDF der technischen Voranalyse öffnen

3.2.4 Voranalyse von Frameworks als HTML

Die folgende Darstellung zeigt die bereits erzeugte HTML-Version der technischen Voranalyse über Vergleich von Frameworks. Sie wird direkt aus der gespeicherten Datei geladen.

Das ursprüngliche Analyse-Notebook muss dafür nicht erneut ausgeführt werden. Dadurch bleiben die längeren Einlese- und Benchmark-Zeiten des zweiten Analyseschritts unberücksichtigt.

[21]:
```
display(
    IFrame(
        src=relativ_framework_evaluation_html,
        width="100%",
        height=900,
    )
)
```

```
<IPython.lib.display.IFrame at 0x15e3fe13020>
```

3.2.5 Interaktives Dashboard zu Frameworks

Das folgende Dashboard basiert auf den bereits gespeicherten Ergebnissen der Voranalyse `02_framework_evaluation`.

Die ursprünglichen Benchmark-Messungen müssen nicht erneut ausgeführt werden. Stattdessen wird die vorhandene Parquet-Datei geladen.

Die interaktive Anwendung dient der vertieften Untersuchung bereits berechneter Ergebnisse. Sie ersetzt nicht die technische Dokumentation und führt standardmäßig keine zeitaufwendigen Benchmark-Messungen erneut aus.

Die Verarbeitungsmodus sowie Dateienformate können einzeln ein- oder ausgeschaltet werden. Zusätzlich kann zwischen Ladezeit , Berechnungszeit, RAM Belegung sowie RAM-Peaks beim Laden und beim Berechnen gewechselt werden.

[22]:
```
df_framework_results = pd.read_parquet(FRAMEWORK_RESULTS)
```

[23]:
```
print(DASH_URL_FRAMEWORK)
```

```
http://127.0.0.1:8061
```

[24]:
```
# Daten-Vorbereitung & Verarbeitungsmodus erstellen
df_dashboard_frameworks = df_framework_results.copy()
df_dashboard_frameworks["Verarbeitungsmodus"] =␣
 ↪df_dashboard_frameworks["Framework"] + " (" +␣
 ↪df_dashboard_frameworks["Methode"] + ")"

# Klassifizierung der Validität basierend auf deinen Projekterkenntnissen
def classify_validity(row):
    # Unnatürliche/Übersprungene Kombinationen
    if row["Framework"] == "Pandas" and row["Methode"] == "Chunking" and␣
 ↪row["Format"] == "Parquet":
        return "Nicht unterstützt (Übersprungen)"
    # Technisch ineffiziente/Fehlgeschlagene Kombinationen (Stresstest-Limits)
    if row["Framework"] == "Dask" and row["Format"] in ["ZIP", "BZ2"]:
        return "Stresstest-Limit (Nicht nativ splittbar unter Windows)"
    if row["Framework"] == "PySpark" and row["Format"] == "ZIP":
        return "Ineffizient (ZIP nicht parallelisierbar)"

    return "Valide Praxis-Kombination"
df_dashboard_frameworks["Validität"] = df_dashboard_frameworks.
 ↪apply(classify_validity, axis=1)
```

[25]:
```
# Empfehlungen
RECOMMENDATIONS = {
    "ZIP": "**ZIP (Ausgangsformat):** Hervorragend für die direkte Verarbeitung␣
 ↪mit *Pandas* geeignet. Für *Dask* und *PySpark* als direkte Eingabe nicht␣
 ↪zuverlässig nutzbar. **Empfehlung:** Mit Pandas einlesen und in Parquet␣
 ↪überführen.",
```

```
    "BZ2": "**BZ2 (Archivformat):** Flexibler als ZIP, aber gegenüber Parquet␣
 ↪nicht die günstigste Arbeitsgrundlage. Eignet sich als reines Austausch-␣
 ↪oder Archivformat.",
    "Parquet": "**Parquet (Zentrales Arbeitsformat):** Erweist sich als die␣
 ↪beste Grundlage. Spaltenorientiert, effiziente Kompression, geringer␣
 ↪Speicherbedarf. Perfekt für den Wechsel zwischen Pandas und PySpark.",
    "ALL": "**Zentrale Architektur-Empfehlung:** Eine Kombination aus␣
 ↪**Pandas** (Einlesen & Bereinigen der ZIP), **Parquet** (als hocheffizientes␣
 ↪Übergabeformat) und **PySpark** (für komplexe, skalierbare Analysen)."
}
```

[26]:
```
framework__metrics_dropdown = dcc.Dropdown(
                    id="metric-selector",
                    options=[
                        {"label": "Ladezeit (s)", "value": "Ladezeit (s)"},
                        {"label": "Berechnungszeit (s)", "value":␣
 ↪"Berechnungszeit (s)"},
                        {"label": "RAM Python Peak (MB)", "value":␣
 ↪"RAM_Python_Peak_MB"},
                        {"label": "RAM Python Calc Peak (MB)", "value":␣
 ↪"RAM_Python_Calc_Peak_MB"},
                        {"label": "RAM Data Netto (MB)", "value":␣
 ↪"RAM_Data_Netto_MB"}
                    ],
                    value="Ladezeit (s)",
                    clearable=False
)
```

[27]:
```
framework_compability_table_page_size = 8
framework_compability_table = dash_table.DataTable(
    id="compatibility-table",
    columns=[
        {"name": "Verarbeitungsmodus", "id": "Verarbeitungsmodus"},
        {"name": "Format", "id": "Format"},
        {"name": "Status", "id": "Status"},
    ],
    data=df_dashboard_frameworks.to_dict("records"),
    row_selectable="single",
    selected_rows=[],
    page_size=framework_compability_table_page_size,
    cell_selectable=True,
    style_cell={"fontFamily": "Arial", "fontSize": "13px", "padding": "8px"},
    style_header={"fontWeight": "bold", "backgroundColor": "#e9ecef"},
    style_data_conditional=[
        {"if": {"filter_query": "{Status} eq 'fehlgeschlagen'"},␣
 ↪"backgroundColor": "#f8d7da", "color": "#721c24"},
```

```
        {"if": {"filter_query": "{Status} eq 'übersprungen'"},␣
 ↪"backgroundColor": "#fff3cd", "color": "#856404"},
        {"if": {"filter_query": "{Status} eq 'erfolgreich'"}, "backgroundColor":
 ↪ "#d4edda", "color": "#155724"},
        # Zeile Auswahl
        {
            "if": {"state": "selected"},
            "backgroundColor": "rgba(0, 0, 0, 0.08)",
            "border": "1px solid #6c757d"
        },
        # Zelle aktiv
        {
            "if": {"state": "active"},
            "backgroundColor": "rgba(0, 0, 0, 0.12)",
            "border": "1px solid #495057"
        }
    ]
)
```

[28]:
```
framework_modus_options = [
    {"label": mode, "value": mode}
    for mode in sorted(df_dashboard_frameworks["Verarbeitungsmodus"].unique())
]

modus_dropdown = dcc.Dropdown(id="mode-dropdown",
    options = framework_modus_options,
    value = df_dashboard_frameworks["Verarbeitungsmodus"].unique().tolist(),
    multi=True,
    placeholder="Verarbeitungsmodus auswählen ...",
)

framework_format_options = [
     {"label": dataformat, "value": dataformat}
    for dataformat in sorted(df_dashboard_frameworks["Format"].unique())
]

framework_format_dropdown = dcc.Dropdown(
    id="format-dropdown",
    options = framework_format_options,
    value = df_dashboard_frameworks["Format"].unique().tolist(),
    multi=True,
    placeholder="Format auswählen ...",
)
```

[29]:
```
# App-Frameworks
app_frameworks = Dash(__name__)
```

```
app_frameworks.layout = html.Div(
    [
        html.H2("Voranalyse 02: Interaktiver Framework- &␣
↪Architektur-Vergleich"),
        html.P("Dieses Dashboard verdeutlicht die Performance-Unterschiede und␣
↪System-Limits von Pandas, Dask und PySpark."),


        # Dynamische Architektur-Infobox
        dcc.Markdown(
            id="dynamic-info-box",
            children=RECOMMENDATIONS["ALL"],
            style={"padding": "15px", "backgroundColor": "#e8f4fd",␣
↪"borderRadius": "5px", "borderLeft": "5px solid #2196F3", "marginBottom":␣
↪"20px"}
        ),

        # TABS
        dcc.Tabs([
            # Graph mit Auswahl
            dcc.Tab(label="Performance-Verteilungen (Boxplots)", children=[
                html.Div([
                    html.Div([
                        html.Label(" Filter: Verarbeitungsmodus",␣
↪style={"fontWeight": "bold", "marginTop": "10px"}),
                        modus_dropdown
                    ], style={"width": "48%"}),
                    html.Div([
                        html.Label(" Filter: Dateiformat", style={"fontWeight":
↪ "bold", "marginTop": "10px"}),
                        framework_format_dropdown
                    ], style={"width": "48%"})
                ], style={"display": "flex", "justifyContent": "space-between",␣
↪"padding": "10px"}),
                       html.Div([
                        html.Div([
                            html.Label(" Kennzahl (Metrik)",␣
↪style={"fontWeight": "bold"}),
                           framework__metrics_dropdown,
                           ], style={"width": "70%", "padding": "10px"})
                           ], style={"display": "flex", "flexDirection": "row",␣
↪"justifyContent": "space-between", "marginBottom": "20px"}),
                # box
                dcc.Graph(id="performance-boxplot", style={"height": "550px"}),

            ]),
```

```
            dcc.Tab(label="Kompatibilitätstabelle",children=[
                html.Div([
                    html.Label("Robustheit & Framework-Kompatibilität",␣
 ↪style={"fontWeight": "bold", "marginTop": "10px"}),
                    framework_compability_table,
                    html.Div(id="error-detail-output", style={"marginTop":␣
 ↪"20px", "padding": "15px", "backgroundColor": "#f8f9fa", "borderRadius":␣
 ↪"5px", "border": "1px solid #dee2e6"})
                ])
            ]),
        ]),


    ],
    style={"width": "100%", "padding": "20px", "boxSizing": "border-box"}
)

# --- CALLBACKS ---

# Callback: Dynamische Infobox steuern
@app_frameworks.callback(
    Output("dynamic-info-box", "children"),
    Input("format-dropdown", "value")
)
def update_infobox(selected_formats):
    if not selected_formats or len(selected_formats) > 1:
        return RECOMMENDATIONS["ALL"]
    fmt = selected_formats[0]
    return RECOMMENDATIONS.get(fmt, RECOMMENDATIONS["ALL"])

# Callback: Boxplot
@app_frameworks.callback(
    Output("performance-boxplot", "figure"),
    Input("metric-selector", "value"),
    Input("mode-dropdown", "value"),
    Input("format-dropdown", "value"),
)
def update_boxplot(metric, selected_modes, selected_formats):
    # Basis-Filterung
    dff = df_dashboard_frameworks[
        (df_dashboard_frameworks["Verarbeitungsmodus"].isin(selected_modes)) &
        (df_dashboard_frameworks["Format"].isin(selected_formats))
    ]

    # Nur Zeilen mit echten Messwerten vergleichen
    dff_plot = dff[dff["Status"] == "erfolgreich"].copy()
```

```
    if dff_plot.empty:
        return px.scatter(title="Keine erfolgreichen Messdaten für diese␣
↪Auswahl vorhanden.")

    fig = px.box(
        dff_plot,
        x="Framework",
        y=metric,
        color="Framework",
        boxmode="group",
        points="all",
        hover_data=["Verarbeitungsmodus", "Methode"],
        title=f"Verteilung der Messergebnisse: {metric}"
    )

    # Diagramm-Styling
    fig.update_layout(
        boxgap=0.2,
        boxgroupgap=0.1,
        margin={"t": 50, "b": 40, "l": 60, "r": 20},
        xaxis_title="Framework",
        yaxis_title=metric
    )
    return fig
```

[30]:
```
# Tabelle
# CALLBACK:  Zellencklick
@app_frameworks.callback(
    Output("compatibility-table", "selected_rows"),
    Input("compatibility-table", "active_cell"),
    State("compatibility-table", "page_current"),
    State("compatibility-table", "page_size"),
    prevent_initial_call=True
)
def sync_row_selection_with_cell_click(active_cell, page_current, page_size):
    if not active_cell:
        return []

    # Paginierung
    current_page = page_current if page_current is not None else 0
    p_size = page_size if page_size is not None else␣
↪framework_compability_table_page_size
    absolute_row_index = (current_page * p_size) + active_cell["row"]

    # Gibt den Index als ausgewählte Zeile an den Radio-Button zurück
    return [absolute_row_index]
```

```
# CALLBACK 2: Details ausgeben
@app_frameworks.callback(
    Output("error-detail-output", "children"),
    Input("compatibility-table", "selected_rows"),
    Input("compatibility-table", "data")
)
def display_error_details(selected_rows, table_data):
    if not selected_rows:
        return dcc.Markdown("**Hinweis:** Klicke auf eine beliebige Zeile in␣
 ↪der Tabelle, um die detaillierten Systemgrenzen und Protokolle einzusehen.")

    row_data = table_data[selected_rows[0]]
    modus = row_data["Verarbeitungsmodus"]
    format_ = row_data["Format"]
    status = row_data["Status"]
    klassifizierung = row_data["Validität"]
    ramdata = row_data["RAM_Data_Netto_MB"]
    ergebnis = row_data["Ergebnis"]
    fehler = row_data["Fehler"]
    laden = row_data["Ladezeit (s)"]
    berechnen = row_data["Berechnungszeit (s)"]
    peak_load = row_data["RAM_Python_Peak_MB"]
    peak_calc = row_data["RAM_Python_Calc_Peak_MB"]

    # Texte formatiert aufbauen
    fehler_anzeigen = f"\n> **Fehlermeldung:** {ergebnis} \n> `Error:␣
 ↪{fehler}`" if pd.notna(fehler) else f"\n>  **Ergebnis:** {ergebnis}"

    metrics_block = ""
    if status == "erfolgreich":
        metrics_block = f"""
### Gemessene Performance-Metriken:
* **Zeiten:** Ladezeit: `{laden:.4f}s` | Berechnungszeit: `{berechnen:.4f}s`
* **Speicherprofile:** Netto-Daten-RAM: `{ramdata} MB` | Peak Laden:␣
 ↪`{peak_load} MB` | Peak Berechnen: `{peak_calc} MB`
"""

    detail_text = f"""
### Detailanalyse: {modus} ({format_})
* **Architektonische Einordnung:** `{klassifizierung}`
* **Systemstatus:** `{status.upper()}`
{fehler_anzeigen}
{metrics_block}
"""
    return dcc.Markdown(detail_text)
```

Dashboard starten

[31]:
```
app_frameworks.run(
    debug=False,
    port=DASH_PORT_FRAMEWORK,
    jupyter_mode="inline",
    jupyter_width="100%",
    jupyter_height=1050,
)
```

```
<IPython.lib.display.IFrame at 0x15e801b0e00>
```

[32]:
```
display(Markdown(f"""
#### Link für Webbrowser
Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht␣
 ↪werden:

[Dashboard im Browser öffnen]({DASH_URL_FRAMEWORK})

Das Dashboard wird lokal gestartet und ist nur erreichbar,
solange die zugehörige Dash-Anwendung ausgeführt wird.
"""))
```

Link für Webbrowser Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht werden:

Dashboard im Browser öffnen

Das Dashboard wird lokal gestartet und ist nur erreichbar, solange die zugehörige Dash-Anwendung ausgeführt wird.

4 Phase 2: Explorative Datenanalyse (EDA) & Analytische Visualisierung

4.1 Methodisches Vorgehen und Zielsetzung

In dieser Phase wird der historische Primärdatensatz strukturiert untersucht. Der Ansatz ist ergebnisoffen und datenkritisch: Vor den eigentlichen fachlichen Analysen steht die Überprüfung der Datenbasis auf Vollständigkeit, Konsistenz und formale Qualität im Vordergrund.

Die Untersuchung folgt einem fließenden, iterativen Prozess: 1. **Technische Inventur:** Laden des Datensatzes direkt aus dem ZIP-Archiv und Analyse der Metadaten (Spaltenbedeutungen und Datentypen). 2. **Datenbereinigung & Transformation:** Zielgerichtete Konvertierung von Datentypen und Aufteilung von Spalten als Vorbereitung für weiterführende Analysen. 3. **Explorationsphase:** Der Einsatz statistischer Grafiken (via Matplotlib/Seaborn) dient hierbei nicht der bloßen Ergebnisdarstellung sondern wird als direktes Werkzeug genutzt, um Trends zu erkennen und analytische Folgefragen zu formulieren. 4. **Ergebnis-Sicherung:** Export der bereinigten Datenbasis in das spaltenorientierte Parquet-Format für nachfolgende Projektphasen. 5. **Interaktive Vertiefung:** Bereitstellung eines dedizierten Dashboards zur visuellen Untersuchung der wichtigsten Erkenntnisse.

4.2 Datenimport und technische Inventur

Der Primärdatensatz wird für diesen Schritt unverändert aus dem ursprünglichen ZIP-Archiv geladen. Ziel dieser ersten Teilphase ist es, den Rohdatensatz zu durchleuchten, ohne die bestehende Dateistruktur vorab zu verändern.

Dabei werden folgende Kernaspekte analysiert: * **Spaltenidentifikation:** Welche Informationen sind im Datensatz enthalten und welche fachliche Bedeutung besitzen die einzelnen Attribute? * **Datentyp-Validierung:** Liegen die Daten (insbesondere Zeitstempel und Verspätungsminuten) in maschinell korrekt interpretierbaren Typen vor? * **Anomalien-Check:** Gibt es strukturelle Auffälligkeiten oder fehlende Werte (*Missing Values*) in den Spalten, die vorab bereinigt werden müssen?

[33]:
```
df_original_ds = pd.read_csv(ORIGINAL_DS_ZIP_PATH, compression="zip")
```

[34]:
```
df_original_ds.shape
```

[34]:
```
(2061357, 20)
```

[35]:
```
df_original_ds.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 20 columns):
 #   Column                Dtype
---  ------                -----
 0   ID                    str
 1   line                  str
 2   path                  str
 3   eva_nr                int64
 4   category              int64
 5   station               str
 6   state                 str
 7   city                  str
 8   zip                   int64
 9   long                  float64
 10  lat                   float64
 11  arrival_plan          str
 12  departure_plan        str
 13  arrival_change        str
 14  departure_change      str
 15  arrival_delay_m       int64
 16  departure_delay_m     int64
 17  info                  str
 18  arrival_delay_check   str
 19  departure_delay_check  str
dtypes: float64(2), int64(5), str(13)
memory usage: 947.4 MB
```

[36]:
```
df_original_ds.describe().T.round(3)
```

[36]:

	count	mean	std	min	25%	50%	75%	max
eva_nr	2061357.0	8018266.781	31786.593	8000001.000	8001582.000	8004136.000	8010208.000	8098360.000
category	2061357.0	3.872	1.037	1.000	3.000	4.000	5.000	5.000
zip	2061357.0	46279.239	28214.243	1067.000	18109.000	47051.000	70806.000	99974.000
long	2061357.0	10.184	2.274	6.071	8.495	9.944	12.091	14.979
lat	2061357.0	50.882	1.792	47.411	49.353	51.087	52.479	54.907
arrival_delay_m	2061357.0	1.177	3.408	0.000	0.000	0.000	1.000	159.000
departure_delay_m	2061357.0	1.224	3.418	0.000	0.000	0.000	1.000	159.000

[37]:
```
df_original_ds.head()
```

[37]:

	ID	line	path	eva_nr	category	station	state	city	zip	long	lat	arrival_plan	departure_plan	arrival_change
0	1573967790757085557-2407072312-14	20	Stolberg(Rheinl)Hbf Gl.44|Eschweiler-St.Jöris|…	8000001	2	Aachen Hbf	Nordrhein-Westfalen	Aachen	52064	6.091499	50.767800	2024-07-08 00:00:00	2024-07-08 00:01:00	2024-07-08 00:03:00
1	34978141703037547 2-2407080017-1	18	NaN	8000001	2	Aachen Hbf	Nordrhein-Westfalen	Aachen	52064	6.091499	50.767800	NaN	2024-07-08 00:17:00	NaN
2	7157250219775883918-2407072120-25	1	Hamm(Westf)Hbf|Kamen|Kamen-Methler|Dortmund-Ku…	8000406	4	Aachen-Rothe Erde	Nordrhein-Westfalen	Aachen	52066	6.116475	50.770202	2024-07-08 00:03:00	2024-07-08 00:04:00	2024-07-08 00:03:00
3	34978141703037547 2-2407080017-2	18	Aachen Hbf	8000404	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	6.070715	50.780360			
4	1983158592123451570-2407080010-3	33	Herzogenrath|Kohlscheid	8000404	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	6.070715	50.780360			

```
3  2024-07-08 00:20:00  2024-07-08 00:21:00                  NaN
4  2024-07-08 00:20:00  2024-07-08 00:21:00  2024-07-08 00:20:00

      departure_change  arrival_delay_m  departure_delay_m info  \
0  2024-07-08 00:04:00                3                  3  NaN
1                  NaN                0                  0  NaN
2  2024-07-08 00:04:00                0                  0  NaN
3                  NaN                0                  0  NaN
4  2024-07-08 00:21:00                0                  0  NaN

  arrival_delay_check departure_delay_check
0             on_time               on_time
1             on_time               on_time
2             on_time               on_time
3             on_time               on_time
4             on_time               on_time
```

4.2.1 Erkenntnisse der strukturellen Bestandsaufnahme

Die erste technische Inventur des Rohdatensatzes liefert wesentliche Erkenntnisse über die Beschaffenheit und Qualität der Datenbasis:

- **Dimensionen & Speicherbedarf:** Der Datensatz umfasst exakt **2.061.357 Datensätze** verteilt auf **20 Spalten**. Der nominelle Arbeitsspeicherbedarf liegt im Rohzustand bei **947,4 MB**, was die Notwendigkeit einer späteren Optimierung (z.B. durch das Parquet-Format) unterstreicht.
- **Fehlinterpretierte Datentypen (Konvertierungsbedarf):** Die zentralen zeitlichen Merkmale `arrival_plan`, `departure_plan`, `arrival_change` und `departure_change` werden aktuell nur als Zeichenketten (`str`/`object`) eingelesen. Diese müssen zwingend in echte `datetime`-Objekte transformiert werden, um zeitliche Analysen zu ermöglichen.
- **Semantische Einordnung der Numerik:** Die statistische Zusammenfassung (`describe`) deckt auf, dass der Großteil der numerischen Spalten (wie `eva_nr`, `category`, `zip`, `long`, `lat`) reine Identifikatoren, Geodaten oder Codierungen darstellt. Mathematische Kennzahlen (wie Mittelwert oder Standardabweichung) sind für diese Spalten fachlich nicht interpretierbar.
- **Echte Analysemetriken:** Einzig die Spalten `arrival_delay_m` und `departure_delay_m` (Verspätungsminuten bei Ankunft und Abfahrt) stellen echte, kontinuierliche numerische Zielvariablen dar. Sie zeigen eine Spanne von **0 bis 159 Minuten** Verspätung und bilden den Kern der späteren explorativen Analyse.

[38]:
```
DS_TOTAL_ROWS_RAW = df_original_ds.shape[0]
DS_MEMORY_USAGE_RAW_MB = df_original_ds.memory_usage(deep=True).sum() / (1024 *␣
↪1024)
```

[39]:
```
# Kombinierte Übersicht: Fehlende Werte & Eindeutige Werte pro Spalte
df_quality_check = pd.DataFrame({
    'Datentyp': df_original_ds.dtypes,
    'Fehlende Werte (absolut)': df_original_ds.isnull().sum(),
```

```
    'Fehlende Werte (%)': (df_original_ds.isnull().sum() / DS_TOTAL_ROWS_RAW *␣
 ↪100).round(2),
    'Eindeutige Werte (Unique)': df_original_ds.nunique()
})

# Spezieller Check für die Kombination aus Longitude und Latitude
unique_geo_coorindates = df_original_ds.groupby(['long', 'lat']).ngroups

print(f"Anzahl einzigartiger Geo-Koordinaten-Paare (long + lat):␣
 ↪{unique_geo_coorindates}")
display(df_quality_check)
```

```
Anzahl einzigartiger Geo-Koordinaten-Paare (long + lat): 1996
```

	Datentyp	Fehlende Werte (absolut)	Fehlende Werte (%)	\
ID	str	0	0.00	
line	str	0	0.00	
path	str	211355	10.25	
eva_nr	int64	0	0.00	
category	int64	0	0.00	
station	str	0	0.00	
state	str	0	0.00	
city	str	0	0.00	
zip	int64	0	0.00	
long	float64	0	0.00	
lat	float64	0	0.00	
arrival_plan	str	211355	10.25	
departure_plan	str	0	0.00	
arrival_change	str	475630	23.07	
departure_change	str	339926	16.49	
arrival_delay_m	int64	0	0.00	
departure_delay_m	int64	0	0.00	
info	str	1416016	68.69	
arrival_delay_check	str	0	0.00	
departure_delay_check	str	0	0.00	

	Eindeutige Werte (Unique)
ID	2029894
line	296
path	22153
eva_nr	1996
category	5
station	1996
state	16
city	1292
zip	1651
long	1995
lat	1996

```
arrival_plan                    10084
departure_plan                  10089
arrival_change                  10122
departure_change                10118
arrival_delay_m                   116
departure_delay_m                 121
info                                7
arrival_delay_check                 2
departure_delay_check               2
```

[40]:
```
arrival_check_unique = df_original_ds["arrival_delay_check"].unique()
display(arrival_check_unique)
```

```
<ArrowStringArray>
['on_time', 'delay']
Length: 2, dtype: str
```

[41]:
```
departure_check_unique = df_original_ds["departure_delay_check"].unique()
display(departure_check_unique)
```

```
<ArrowStringArray>
['on_time', 'delay']
Length: 2, dtype: str
```

[42]:
```
info_unique = df_original_ds["info"].unique()
display(info_unique)
```

```
<ArrowStringArray>
[                              nan, 'Bauarbeiten. (Quelle: zuginfo.nrw)',
                'Information', 'Information. (Quelle: zuginfo.nrw)',
                'Bauarbeiten',                              'Störung',
                'Großstörung',     'Störung. (Quelle: zuginfo.nrw)']
Length: 8, dtype: str
```

[43]:
```
DS_ORIGINAL_ID_UNIQUE = df_original_ds["ID"].nunique()
print(DS_ORIGINAL_ID_UNIQUE)
```

```
2029894
```

4.2.2 Erkenntnisse der Qualitäts- und Fehlstellen-Analyse

Die systematische Auswertung der Fehlstellen (Missing Values) und der Kardinalität (Unique-Werte) liefert keine fertigen Antworten, sondern deckt präzise mathematische Muster auf, die im nächsten Schritt analytisch untersucht werden müssen:

- **Die Startbahnhof-Hypothese (Symmetrie der Fehlstellen):** Die Spalten `path` (Streckenverlauf) und `arrival_plan` (geplante Ankunftszeit) weisen mit exakt **211.355 Datensätzen (10,25 %)** eine identische Anzahl an Fehlstellen auf. Es wird die Hypothese aufgestellt, dass es sich hierbei um Züge handelt, die an diesem Bahnhof ihre Fahrt beginnen und daher systembedingt weder eine geplante Ankunft noch einen vorherigen Streckenverlauf besitzen. Dies gilt es im folgenden Analyseschritt zu verifizieren.

- **Geografische Konsistenz trotz Abweichung:** Während die Spalte `long` nur 1995 eindeutige Werte aufweist, zeigt die kombinierte Gruppierung der Geo-Koordinatenpaare (`long + lat`) exakt **1996 einzigartige Paare**. Dies stimmt perfekt mit der Anzahl der EVA-Nummern (`eva_nr`) und Bahnhofsnamen (`station`) überein. Die Geodaten sind somit strukturell vollständig konsistent; zwei Stationen teilen sich lediglich zufällig denselben Längengrad.
- **Hypothese zu den Planänderungen (`_change`):** Die Spalten `arrival_change` (23,07 % Fehlwerte) und `departure_change` (16,49 % Fehlwerte) zeigen eine ungleiche Verteilung von fehlenden Werten. Als vorläufige Vermutung wird angenommen, dass diese Spalten nur bei operativen Abweichungen vom Fahrplan gefüllt werden. Warum bei der Ankunft deutlich häufiger Änderungen protokolliert wurden als bei der Abfahrt, ist eine offene Forschungsfrage für die spätere EDA.
- **Binäre Klassifikations-Flags:** Die Spalten `arrival_delay_check` und `departure_delay_check` enthalten ausschließlich die zwei Ausprägungen `['on_time', 'delay']`. Diese Struktur legt nahe, dass es sich um binäre Indikatoren handelt, die sich nach einer Konvertierung ideal als Filterkriterien eignen.
- **Potenzial der Spalte `info` für die Ursachenanalyse:** Trotz einer hohen Fehlstellenquote von **68,69 %** darf die Spalte `info` nicht ausgeschlossen werden. Die Inspektion der eindeutigen Textwerte offenbart kategorische Begründungen wie `'Bauarbeiten'`, `'Störung'` oder `'Großstörung'` (teilweise mit Verweisen auf *zuginfo.nrw*). Diese Spalte bietet somit ein enormes analytisches Potenzial, um einen Teil der Verspätungen im späteren Verlauf kausal zu begründen.

[44]:
```python
# Definition der Linien-Konstante und Sortierung der eindeutigen Werte
unique_lines = sorted(df_original_ds['line'].dropna().unique().astype(str))
NUM_UNIQUE_LINES = len(unique_lines)

# Kompakte Visualisierung der strukturellen Zusammensetzung (Mischtyp-Prüfung)
print("Auszug der eindeutigen Linienbezeichnungen (Anfang, Mitte und Ende):")
print(f"Anfang: {unique_lines[:20]}")
print(f"Mitte: {unique_lines[140:160]}")
print(f"Ende: {unique_lines[-20:]}")
```

```
Auszug der eindeutigen Linienbezeichnungen (Anfang, Mitte und Ende):
Anfang: ['1', '10', '10a', '10b', '11', '110', '113', '12', '13', '14', '14a',
'14b', '15', '16', '17', '17a', '17b', '17c', '18', '19']
Mitte: ['RB23', 'RB24', 'RB25', 'RB26', 'RB27', 'RB28', 'RB29', 'RB30', 'RB31',
'RB32', 'RB33', 'RB34', 'RB35', 'RB36', 'RB37', 'RB38', 'RB39', 'RB4', 'RB40',
'RB41']
Ende: ['S19', 'S2', 'S23', 'S3', 'S3A', 'S3B', 'S4', 'S44', 'S5', 'S6', 'S62',
'S64', 'S7', 'S7 EV', 'S8', 'S9', 'T9', 'U28', 'WX', 'X85F']
```

[45]:
```python
# Definition der Länder-Konstante und Extraktion der eindeutigen Werte
unique_states = df_original_ds['state'].dropna().unique()
NUM_UNIQUE_STATES = len(unique_states)

# Vollständige Ausgabe zur Überprüfung der geografischen Abdeckung
print("Gefundene Bundesländer im Datensatz:")
```

```
print(list(unique_states))
```

```
Gefundene Bundesländer im Datensatz:
['Nordrhein-Westfalen', 'Baden-Württemberg', 'Niedersachsen', 'Schleswig-
Holstein', 'Bayern', 'Berlin', 'Hamburg', 'Thüringen', 'Rheinland-Pfalz',
'Mecklenburg-Vorpommern', 'Sachsen', 'Sachsen-Anhalt', 'Brandenburg', 'Hessen',
'Saarland', 'Bremen']
```

[46]:
```
# Aufteilung der originalen ID-Zeichenkette an den Trennzeichen
id_splits_preview = df_original_ds['ID'].astype(str).str.split('-', expand=True)

# Überprüfung, ob alle isolierten Segmente als reine Ganzzahlen interpretierbar␣
 ↪sind
all_parts_are_numeric = True
for col in id_splits_preview.columns:
    if not id_splits_preview[col].str.isnumeric().all():
        all_parts_are_numeric = False

print(f"Ergebnis der reinen numerischen Typprüfung für alle Segmente:␣
 ↪{all_parts_are_numeric}")
```

```
Ergebnis der reinen numerischen Typprüfung für alle Segmente: False
```

[47]:
```
# Ermittlung der nominalen Anzahl an Bindestrichen im gesamten Datensatz
id_split_counts = df_original_ds['ID'].astype(str).str.count('-')
id_min_dashes = id_split_counts.min()
id_max_dashes = id_split_counts.max()
print(f"Analyse der Roh-Trennzeichen (Bindestriche) pro ID: Minimum =␣
 ↪{id_min_dashes}, Maximum = {id_max_dashes}")
```

```
Analyse der Roh-Trennzeichen (Bindestriche) pro ID: Minimum = 2, Maximum = 3
```

[48]:
```
# Isolation und Stichproben-Ausgabe der betroffenen Datensätze mit 3␣
 ↪Trennzeichen
mask_3_dashes = id_split_counts == 3
df_anomalies_3 = df_original_ds[mask_3_dashes]

print(f"Anzahl der IDs mit 3 Trennzeichen: {len(df_anomalies_3)}")
print("Auszug der betroffenen ID-Zeichenketten:")
print(df_anomalies_3['ID'].head(10).to_string())
```

```
Anzahl der IDs mit 3 Trennzeichen: 1035418
Auszug der betroffenen ID-Zeichenketten:
5      -5293934437045765939-2407080023-2
7     -2100556839975301087-2407072307-13
8      -7696913984968518161-2407080037-1
9      -6027587483204218492-2407080013-4
10     -7723223610149163054-2407072310-9
14     -1113872591615872448-2407072343-6
```

```
17     -6105012072686197130-2407080016-3
19     -8761881508899340400-2407080008-6
21    -5328965667779335365-2407072203-17
22    -5194156004862660093-2407072125-28
```

[49]:
```python
# Entfernung des führenden Bindestrichs/Vorzeichens zur Bereinigung der␣
 ↪strukturellen Symmetrie
df_id_clean = df_original_ds['ID'].astype(str).str.lstrip('-')

# Erneute Bestimmung der verbleibenden logischen Trennstriche
clean_dash_counts = df_id_clean.str.count('-')
print(f"Logische Trennstriche nach Bereinigung des führenden Zeichens: Minimum␣
 ↪= {clean_dash_counts.min()}, Maximum = {clean_dash_counts.max()}")

# Aufteilung und finale numerische Validierung der bereinigten Segmente
id_splits_final = df_id_clean.str.split('-', expand=True)
all_parts_numeric_final = True

for col in id_splits_final.columns:
    if not id_splits_final[col].str.isnumeric().all():
        all_parts_numeric_final = False

print(f"Sind alle logischen ID-Segmente nach der Bereinigung reine Ganzzahlen?␣
 ↪{all_parts_numeric_final}")
```

```
Logische Trennstriche nach Bereinigung des führenden Zeichens: Minimum = 2,
Maximum = 2
Sind alle logischen ID-Segmente nach der Bereinigung reine Ganzzahlen? True
```

[50]:
```python
# Präzise Trennung der ID-Zeichenkette von rechts her an den letzten zwei␣
 ↪Bindestrichen
id_parts_raw = df_original_ds['ID'].astype(str).str.rsplit('-', n=2,␣
 ↪expand=True)
id_parts_raw.columns = ['part1_raw', 'part2', 'part3']

# Ermittlung der Kardinalität des ersten ID-Segments im Rohzustand (mit␣
 ↪Vorzeichen)
unique_part1_with_sign = id_parts_raw['part1_raw'].nunique()

# Extraktion des numerischen Kerns durch Entfernung des führenden Bindestrichs
part1_no_sign = id_parts_raw['part1_raw'].str.lstrip('-')
unique_part1_no_sign = part1_no_sign.nunique()

# Berechnung der Differenz zur Überprüfung von Betragsgleichheiten
id_semantic_difference = unique_part1_with_sign - unique_part1_no_sign

print(f"Eindeutige Werte in Part 1 mit Vorzeichen: {unique_part1_with_sign}")
print(f"Eindeutige Werte in Part 1 ohne Vorzeichen: {unique_part1_no_sign}")
```

```
print(f"Resultierende Differenz für die Typprüfung: {id_semantic_difference}")
```

```
Eindeutige Werte in Part 1 mit Vorzeichen: 50013
Eindeutige Werte in Part 1 ohne Vorzeichen: 50013
Resultierende Differenz für die Typprüfung: 0
```

4.2.3 Erkenntnisse der erweiterten Attribut- und ID-Strukturanalyse

Die mathematische und strukturelle Vorprüfung der Attribute `line`, `state` und `ID` liefert folgende informationstechnische Befunde als Ausgangslage für die weiteren Verarbeitungsschritte:

- **Charakteristik der Linienbezeichnungen (`line`):** Die empirische Prüfung zeigt, dass die Spalte alphanumerische Kombinationen (z. B. `RB23`, `S19`) enthält. Eine direkte Konvertierung in einen rein numerischen Datentyp ist daher nicht möglich. Das Attribut verbleibt in diesem Stadium unverändert im Ausgangsdatentyp, um die Rohdatenstruktur für spätere Validierungen vollständig zu erhalten.
- **Geografische Zuordnung (`state`):** Die Auswertung der eindeutigen Werte bestätigt, dass der Datensatz ausschließlich Informationen innerhalb der Bundesrepublik Deutschland beinhaltet, wobei alle 16 Bundesländer unter dem Merkmal `state` vertreten sind. Eine Aussage über die statistische Vollständigkeit oder Repräsentativität des Schienennetzes kann an dieser Stelle noch nicht getroffen werden.
- **Strukturelle ID-Analyse:** Die Zusammensetzung der ID-Zeichenkette weist im ersten Segment ein führendes Minuszeichen bei insgesamt 1.035.418 Datensätzen auf. Ein Abgleich zeigt, dass innerhalb dieses einwöchigen Datensatzes keine ID-Kombinationen existieren, bei denen das identische Segment im ersten Teil einmal mit und einmal ohne Minuszeichen auftritt. Da die systemische Bedeutung dieses Vorzeichens zum aktuellen Zeitpunkt ungeklärt ist, muss die Datentreue gewahrt bleiben.
- **Transformationsvorgabe für die ID-Zerlegung:** Zur Verbesserung der technischen Handhabung im Notebook wird die zusammengesetzte ID-Zeichenkette in drei separate Spalten aufgeteilt. Das erste Segment (`part1`) wird dabei so konvertiert, dass sowohl die negativen als auch die positiven Werte exakt wie im Datensatz vorliegend als vorzeichenbehafteter Ganzzahltyp (`Int64`) übernommen werden.

4.3 Strukturelle Transformation und datentechnische Aufbereitung

Auf Basis der vorangegangenen technischen Inventur werden in diesem Abschnitt die ersten strukturellen Anpassungen am Datensatz vorgenommen. Ziel dieses Schrittes ist es, die Datenbasis durch präzise Typkonvertierungen und Spaltenaufspaltungen in ein rechentechnisch effizienteres Format zu überführen, um die nachfolgenden Analysen zu ermöglichen.

Die Umsetzung erfolgt schrittweise, wobei jede Transformation direkt auf den empirischen Befunden der Bestandsaufnahme basiert. Eventuelle zusätzliche Erkenntnisse, die während der Konvertierungsprozesse sichtbar werden, werden fortlaufend dokumentiert und fließen in die Anpassung der Pipeline ein.

4.3.1 Umsetzung der ID-Aufspaltung auf Basis der verifizierten Trennungslogik

Die Aufspaltung der ID-Zeichenkette erfolgt über eine bedingte Trennungslogik, welche die variierende Anzahl der Bindestriche (zwei oder drei Trennzeichen) im Datensatz berücksichtigt. Durch diese Fallunterscheidung wird das führende Minuszeichen des ersten Segments mathematisch korrekt isoliert und eine Verschiebung der nachfolgenden Spaltensegmente systemisch verhindert.

Da die empirische Vorprüfung bereits die vollständige Konvertierbarkeit aller resultierenden Fragmente in Ganzzahlwerte belegt hat, wird die Transformation in den vorzeichenbehafteten Datentyp `Int64` direkt durchgeführt. Dies stellt eine fehlerfreie Aufteilung der ID in drei separate numerische Spalten sicher, ohne dass ein Informationsverlust auftritt.

[51]:
```
# Aufspaltung der ID-Zeichenkette von rechts, begrenzt auf maximal zwei␣
 ↪Trennungen
# Dies isoliert part2 und part3 unabhängig vom führenden Minuszeichen in part1
id_parts = df_original_ds["ID"].str.rsplit("-", n=2, expand=True)

# Zuweisung der extrahierten Segmente und explizite Konvertierung in den Typ␣
 ↪Int64
df_original_ds["ID_part1"] = id_parts[0].astype("Int64")
df_original_ds["ID_part2"] = id_parts[1].astype("Int64")
df_original_ds["ID_part3"] = id_parts[2].astype("Int64")

# Anzeige der ersten Zeilen zur Verifikation der neuen Spaltenstruktur
df_original_ds[['ID', "ID_part1", "ID_part2", "ID_part3"]].head(50)
```

[51]:

	ID	ID_part1	ID_part2	\
0	1573967790757085557-2407072312-14	1573967790757085557	2407072312	
1	349781417030375472-2407080017-1	349781417030375472	2407080017	
2	7157250219775883918-2407072120-25	7157250219775883918	2407072120	
3	349781417030375472-2407080017-2	349781417030375472	2407080017	
4	1983158592123451570-2407080010-3	1983158592123451570	2407080010	
5	-5293934437045765939-2407080023-2	-5293934437045765939	2407080023	
6	6845762881043426854-2407072357-6	6845762881043426854	2407072357	
7	-2100556839975301087-2407072307-13	-2100556839975301087	2407072307	
8	-7696913984968518161-2407080037-1	-7696913984968518161	2407080037	
9	-6027587483204218492-2407080013-4	-6027587483204218492	2407080013	
10	-7723223610149163054-2407072310-9	-7723223610149163054	2407072310	
11	5729393359484274537-2407080007-8	5729393359484274537	2407080007	
12	1640260421649276864-2407080004-5	1640260421649276864	2407080004	
13	8352762151701721590-2407080034-5	8352762151701721590	2407080034	
14	-1113872591615872448-2407072343-6	-1113872591615872448	2407072343	
15	7148083203553802700-2407072356-5	7148083203553802700	2407072356	
16	5507673346685034119-2407072356-5	5507673346685034119	2407072356	
17	-6105012072686197130-2407080016-3	-6105012072686197130	2407080016	
18	7734805089613574578-2407072359-7	7734805089613574578	2407072359	
19	-8761881508899340400-2407080008-6	-8761881508899340400	2407080008	
20	6182068753204726904-2407072344-10	6182068753204726904	2407072344	

21	-5328965667779335365-2407072203-17	-5328965667779335365	2407072203
22	-5194156004862660093-2407072125-28	-5194156004862660093	2407072125
23	-8761881508899340400-2407080008-12	-8761881508899340400	2407080008
24	2001635860734548872-2407072324-13	2001635860734548872	2407072324
25	6182068753204726904-2407072344-4	6182068753204726904	2407072344
26	7251022928272275067-2407072206-20	7251022928272275067	2407072206
27	6252347552889792227-2407072336-6	6252347552889792227	2407072336
28	1097110016422901566-2407072336-7	1097110016422901566	2407072336
29	8092363418274949423-2407072350-6	8092363418274949423	2407072350
30	1617964573784399284-2407072316-24	1617964573784399284	2407072316
31	4035296386850578453-2407072346-24	4035296386850578453	2407072346
32	-8775031481172421761-2407080020-6	-8775031481172421761	2407080020
33	-3390013903921837926-2407080013-4	-3390013903921837926	2407080013
34	-3521475736976801889-2407080008-7	-3521475736976801889	2407080008
35	-5006626778859705318-2407072348-7	-5006626778859705318	2407072348
36	-5416266371209213922-2407072323-24	-5416266371209213922	2407072323
37	-2177588258474375206-2407080023-15	-2177588258474375206	2407080023
38	3534916180088668718-2407080003-24	3534916180088668718	2407080003
39	3945870737082109061-2407072353-4	3945870737082109061	2407072353
40	-1093508172194817351-2407072343-24	-1093508172194817351	2407072343
41	-1457274040898548571-2407072336-4	-1457274040898548571	2407072336
42	2992031615454778636-2407072348-5	2992031615454778636	2407072348
43	-5109043856199033040-2407072315-10	-5109043856199033040	2407072315
44	1974895317199967519-2407072348-10	1974895317199967519	2407072348
45	3360893944022444050-2407072338-26	3360893944022444050	2407072338
46	8754252874419393638-2407072349-5	8754252874419393638	2407072349
47	6809920023340627969-2407080009-5	6809920023340627969	2407080009
48	8444775672791196589-2407072357-5	8444775672791196589	2407072357
49	-8927383541084970752-2407072318-26	-8927383541084970752	2407072318

	ID_part3
0	14
1	1
2	25
3	2
4	3
5	2
6	6
7	13
8	1
9	4
10	9
11	8
12	5
13	5
14	6
15	5

```
16          5
17          3
18          7
19          6
20         10
21         17
22         28
23         12
24         13
25          4
26         20
27          6
28          7
29          6
30         24
31         24
32          6
33          4
34          7
35          7
36         24
37         15
38         24
39          4
40         24
41          4
42          5
43         10
44         10
45         26
46          5
47          5
48          5
49         26
```

[52]:
```
df_original_ds.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 23 columns):
 #   Column                 Dtype
---  ------                 -----
 0   ID                     str
 1   line                   str
 2   path                   str
 3   eva_nr                 int64
 4   category               int64
 5   station                str
```

```
 6   state                 str
 7   city                  str
 8   zip                   int64
 9   long                  float64
 10  lat                   float64
 11  arrival_plan          str
 12  departure_plan        str
 13  arrival_change        str
 14  departure_change      str
 15  arrival_delay_m       int64
 16  departure_delay_m     int64
 17  info                  str
 18  arrival_delay_check   str
 19  departure_delay_check str
 20  ID_part1              Int64
 21  ID_part2              Int64
 22  ID_part3              Int64
dtypes: Int64(3), float64(2), int64(5), str(13)
memory usage: 1000.5 MB
```

[53]:
```
print(df_original_ds[["ID_part1", "ID_part2", "ID_part3"]].isnull().sum())
```

```
ID_part1    0
ID_part2    0
ID_part3    0
dtype: int64
```

[54]:
```
# Entfernen der ursprünglichen ID-Spalte zur Optimierung des Arbeitsspeichers
df_original_ds = df_original_ds.drop(columns=["ID"])
```

[55]:
```
df_original_ds.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 22 columns):
 #   Column                Dtype
---  ------                -----
 0   line                  str
 1   path                  str
 2   eva_nr                int64
 3   category              int64
 4   station               str
 5   state                 str
 6   city                  str
 7   zip                   int64
 8   long                  float64
 9   lat                   float64
 10  arrival_plan          str
 11  departure_plan        str
```

```
 12  arrival_change          str
 13  departure_change        str
 14  arrival_delay_m         int64
 15  departure_delay_m       int64
 16  info                    str
 17  arrival_delay_check     str
 18  departure_delay_check   str
 19  ID_part1                Int64
 20  ID_part2                Int64
 21  ID_part3                Int64
dtypes: Int64(3), float64(2), int64(5), str(12)
memory usage: 920.2 MB
```

[56]:
```
print(f"vorher: {DS_MEMORY_USAGE_RAW_MB} MB")
current_memory_usage = df_original_ds.memory_usage(deep=True).sum() / (1024 *␣
 ↪1024)
print(f"nacher: {current_memory_usage} MB")
if DS_MEMORY_USAGE_RAW_MB > current_memory_usage:
    ersparnis = DS_MEMORY_USAGE_RAW_MB - current_memory_usage
    print(f"RAM Ersparnis {ersparnis} MB")
```

```
vorher: 947.3857374191284 MB
nacher: 920.1630210876465 MB
RAM Ersparnis 27.222716331481934 MB
```

[57]:
```
print("\nStatistische Kennzahlen der ID-Segmente:")
print(df_original_ds[["ID_part1", "ID_part2", "ID_part3"]].describe())
```

```
Statistische Kennzahlen der ID-Segmente:
                   ID_part1           ID_part2   ID_part3
count             2061357.0          2061357.0  2061357.0
mean   -21086134748267028.0  2407108685.218822  10.461123
std     5327233908976233472.0       21612.742279   7.897363
min    -9223176950731764736.0       2407031857.0        1.0
25%    -4590018462129540096.0       2407091412.0        4.0
50%      -46576259786505424.0       2407110936.0        9.0
75%     4567148400468366336.0       2407122331.0       15.0
max     9223056977874851840.0       2407142359.0       60.0
```

[58]:
```
# Verifikation der kombinatorischen Eindeutigkeit nach der Spaltenaufspaltung
# Die Anzahl der eindeutigen Kombinationen muss exakt der historischen␣
 ↪ID-Anzahl entsprechen
anzahl_kombinationen_unique = df_original_ds.groupby(["ID_part1", "ID_part2",␣
 ↪"ID_part3"]).ngroups

print(f"Eindeutige ursprüngliche IDs: {DS_ORIGINAL_ID_UNIQUE}")
```

```
print(f"Eindeutige Spaltenkombinationen nach Split:␣
 ↪{anzahl_kombinationen_unique}")

# Logischer Abgleich zur Sicherung der Datentreue
ist_konsistent = anzahl_kombinationen_unique == DS_ORIGINAL_ID_UNIQUE
print(f"Die kombinatorische Eindeutigkeit ist vollständig gewahrt:␣
 ↪{ist_konsistent}")
```

```
Eindeutige ursprüngliche IDs: 2029894
Eindeutige Spaltenkombinationen nach Split: 2029894
Die kombinatorische Eindeutigkeit ist vollständig gewahrt: True
```

[59]:
```
# Ermittlung der Anzahl eindeutiger Werte pro ID-Segment (Kardinalität)
print("Anzahl eindeutiger Werte (Uniques) pro ID-Segment:")
print(f"ID_part1: {df_original_ds['ID_part1'].nunique()}")
print(f"ID_part2: {df_original_ds['ID_part2'].nunique()}")
print(f"ID_part3: {df_original_ds['ID_part3'].nunique()}")

# Überprüfung der Hypothese: Korrelation zwischen ID_part3 == 1 und fehlendem␣
 ↪arrival_plan
# Filterung auf Datensätze, bei denen der Ankunftsplan fehlt
df_missing_arrival = df_original_ds[df_original_ds["arrival_plan"].isnull()]

# Zählen, wie oft die jeweiligen Werte von ID_part3 bei fehlender Ankunftszeit␣
 ↪vorkommen
print("\nHäufigkeitsverteilung von ID_part3 bei fehlenden Werten in␣
 ↪'arrival_plan':")
print(df_missing_arrival["ID_part3"].value_counts())

# Gegenprüfung: Wie viele Datensätze haben insgesamt ID_part3 == 1
gesamt_part3_eins = (df_original_ds["ID_part3"] == 1).sum()
print(f"\nGesamtzahl aller Datensätze mit ID_part3 == 1: {gesamt_part3_eins}")
```

```
Anzahl eindeutiger Werte (Uniques) pro ID-Segment:
ID_part1: 50013
ID_part2: 10155
ID_part3: 56

Häufigkeitsverteilung von ID_part3 bei fehlenden Werten in 'arrival_plan':
ID_part3
1    211355
Name: count, dtype: Int64

Gesamtzahl aller Datensätze mit ID_part3 == 1: 211355
```

[60]:
```
# Filterung auf Datensätze, bei denen die Spalte 'path' keine Einträge enthält␣
 ↪(NaN/None)
df_missing_path = df_original_ds[df_original_ds["path"].isnull()]
```

```
print(f"Gesamtzahl der Datensätze mit fehlendem 'path': {len(df_missing_path)}")

# Prüfung, welche eindeutigen Werte 'ID_part3' in dieser Teilmenge annimmt
print("\nEindeutige Werte von 'ID_part3' bei fehlendem 'path':")
print(df_missing_path["ID_part3"].unique())

# Prüfung, wie viele Fehlwerte in 'arrival_plan' in dieser Teilmenge existieren
print("\nAnzahl fehlender Werte in 'arrival_plan' bei fehlendem 'path':")
print(df_missing_path["arrival_plan"].isnull().sum())
```

```
Gesamtzahl der Datensätze mit fehlendem 'path': 211355

Eindeutige Werte von 'ID_part3' bei fehlendem 'path':
<IntegerArray>
[1]
Length: 1, dtype: Int64

Anzahl fehlender Werte in 'arrival_plan' bei fehlendem 'path':
211355
```

[61]:
```
# Erstellung der theoretisch vollständigen Sequenz von 1 bis zum Maximum (60)
theoretische_sequenz = set(range(1, int(df_original_ds["ID_part3"].max()) + 1))

# Extrahieren der tatsächlich im Datensatz existierenden eindeutigen Werte
tatsaechliche_werte = set(df_original_ds["ID_part3"].dropna().unique())

# Bestimmung der Differenz (die fehlenden Stop-Nummern)
fehlende_zahlen = theoretische_sequenz - tatsaechliche_werte
print(f"Fehlende Zahlenwerte in der Sequenz von ID_part3: {sorted(list(fehlende_zahlen))}")
```

```
Fehlende Zahlenwerte in der Sequenz von ID_part3: [56, 57, 58, 59]
```

[62]:
```
# Schritt 1: Identifikation der Fahrten (Kombination aus part1 und part2), die den Index 60 erreichen
fahrten_mit_max_60 = df_original_ds[df_original_ds["ID_part3"] == 60][["ID_part1", "ID_part2"]]

# Schritt 2: Extrahieren aller Datenpunkte, die zu diesen spezifischen Fahrten gehören
df_stichprobe_60 = df_original_ds.merge(fahrten_mit_max_60, on=["ID_part1", "ID_part2"], how="inner")

# Schritt 3: Ausgabe der eindeutigen Haltestellen-Indizes dieser Fahrten zur visuellen Prüfung der Lücke
print("Existierende Haltestellen-Indizes (ID_part3) für die Fahrten mit Maximalindex 60:")
```

```
print(sorted(df_stichprobe_60["ID_part3"].unique()))

# Schritt 4: Exemplarische Anzeige der Einträge mit hohem Index zur Analyse des␣
 ↪Inhalts der Spalte 'path'
# Sortiert nach der Fahrt und der Haltestellensequenz
df_stichprobe_60_sortiert = df_stichprobe_60.sort_values(by=["ID_part1",␣
 ↪"ID_part2", "ID_part3"])
df_stichprobe_60_sortiert[["ID_part1", "ID_part2", "ID_part3", "path"]].tail(10)
```

```
Existierende Haltestellen-Indizes (ID_part3) für die Fahrten mit Maximalindex
60:
[np.int64(12), np.int64(14), np.int64(15), np.int64(16), np.int64(17),
np.int64(51), np.int64(60)]
```

[62]:

	ID_part1	ID_part2	ID_part3	\
116	8619870746781998415	2407101057	17	
121	8619870746781998415	2407101057	51	
119	8619870746781998415	2407101057	60	
157	8619870746781998415	2407111057	12	
155	8619870746781998415	2407111057	14	
154	8619870746781998415	2407111057	15	
159	8619870746781998415	2407111057	16	
160	8619870746781998415	2407111057	17	
165	8619870746781998415	2407111057	51	
163	8619870746781998415	2407111057	60	

	path
116	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
121	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
119	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
157	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
155	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
154	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
159	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
160	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
165	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…
163	Tullastraße/Alter Schlachthof, Karlsruhe|Gotte…

[63]:
```
# Systematische Sortierung nach Fahrt-IDs, Haltestellensequenz (absteigend) und␣
 ↪der geplanten Ankunftszeit
df_stichprobe_60_sortiert = df_stichprobe_60.sort_values(
    by=["ID_part1", "ID_part2", "ID_part3", "arrival_plan"],
    ascending=[True, True, False, True]
)

# Selektion einer einzelnen Beispiel-Fahrt, um die Lücke im Detail zu betrachten
```

```
# Hier wird die ID_part1 aus Ihrem Output (8619870746781998415) als Filter␣
 ↪genutzt
fahrt_id_fokus = 8619870746781998415
df_einzelne_fahrt =␣
 ↪df_stichprobe_60_sortiert[df_stichprobe_60_sortiert["ID_part1"] ==␣
 ↪fahrt_id_fokus]

# Ausgabe der relevanten Spalten inklusive Ankunftszeit zur zeitlichen␣
 ↪Orientierung
# Wir lassen uns den vollständigen Pfad-String anzeigen, um die Stationsanzahl␣
 ↪darin zu prüfen
with pd.option_context('display.max_colwidth', None):
    print("Detaillierter Sequenzverlauf für die Fokus-Fahrt:")
    print(df_einzelne_fahrt[["ID_part1", "ID_part2", "ID_part3",␣
 ↪"arrival_plan", "path"]].head(10))
```

```
Detaillierter Sequenzverlauf für die Fokus-Fahrt:
               ID_part1    ID_part2  ID_part3        arrival_plan  \
21  8619870746781998415  2407081057        60  2024-07-08 13:50:00
23  8619870746781998415  2407081057        51  2024-07-08 13:07:00
18  8619870746781998415  2407081057        17  2024-07-08 11:37:00
17  8619870746781998415  2407081057        16  2024-07-08 11:31:00
12  8619870746781998415  2407081057        15  2024-07-08 11:28:00
13  8619870746781998415  2407081057        14  2024-07-08 11:26:00
15  8619870746781998415  2407081057        12  2024-07-08 11:19:00
70  8619870746781998415  2407091057        60  2024-07-09 13:50:00
72  8619870746781998415  2407091057        51  2024-07-09 13:07:00
67  8619870746781998415  2407091057        17  2024-07-09 11:37:00

                                                                 path
21  Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim
Nord|Durmersheim|Bietigheim(Baden)|Ötigheim|Rastatt|Rastatt
Beinle|Kuppenheim|Bischweier|Bad Rotenfels Schloss|Bad Rotenfels Bf|Bad
Rotenfels Weinbrennerstraße|Gaggenau Bf|Gaggenau Mercedes-Benz
Werk|Ottenau|Hörden|Gernsbach Bf|Gernsbach
Mitte|Obertsrot|Hilpertsau|Weisenbach|Au im Murgtal|Langenbrand|Gausbach|Forbach
(Schwarzw)|Raumünzach|Kirschbaumwasen|Schönmünzach|Schwarzenberg|Huzenbach|Röt|H
eselbach|Klosterreichenbach|Baiersbronn Schule|Baiersbronn Bf|Friedrichstal b
Freudenstadt|Freudenstadt Stadt|Freudenstadt Schulzentrum|Freudenstadt
Industriegebiet|Freudenstadt Hbf|Grüntal-Wittlensweiler|Dornstetten-
Aach|Dornstetten|Schopfloch(b Freudenstadt)|Bittelbronn|Horb-
Heiligenfeld|Hochdorf(b Horb)|Eutingen Nord
```

```
23
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim
Nord|Durmersheim|Bietigheim(Baden)|Ötigheim|Rastatt|Rastatt
Beinle|Kuppenheim|Bischweier|Bad Rotenfels Schloss|Bad Rotenfels Bf|Bad
Rotenfels Weinbrennerstraße|Gaggenau Bf|Gaggenau Mercedes-Benz
Werk|Ottenau|Hörden|Gernsbach Bf|Gernsbach
Mitte|Obertsrot|Hilpertsau|Weisenbach|Au im Murgtal|Langenbrand|Gausbach|Forbach
(Schwarzw)|Raumünzach|Kirschbaumwasen|Schönmünzach|Schwarzenberg|Huzenbach|Röt|H
eselbach|Klosterreichenbach|Baiersbronn Schule|Baiersbronn Bf|Friedrichstal b
Freudenstadt|Freudenstadt Stadt|Freudenstadt Schulzentrum|Freudenstadt
Industriegebiet
18
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim Nord|Durmersheim|Bietigheim(Baden)|Ötigheim
17
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim Nord|Durmersheim|Bietigheim(Baden)
12
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim Nord|Durmersheim
13
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim Nord
15
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
```

```
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof
70  Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim
Nord|Durmersheim|Bietigheim(Baden)|Ötigheim|Rastatt|Rastatt
Beinle|Kuppenheim|Bischweier|Bad Rotenfels Schloss|Bad Rotenfels Bf|Bad
Rotenfels Weinbrennerstraße|Gaggenau Bf|Gaggenau Mercedes-Benz
Werk|Ottenau|Hörden|Gernsbach Bf|Gernsbach
Mitte|Obertsrot|Hilpertsau|Weisenbach|Au im Murgtal|Langenbrand|Gausbach|Forbach
(Schwarzw)|Raumünzach|Kirschbaumwasen|Schönmünzach|Schwarzenberg|Huzenbach|Röt|H
eselbach|Klosterreichenbach|Baiersbronn Schule|Baiersbronn Bf|Friedrichstal b
Freudenstadt|Freudenstadt Stadt|Freudenstadt Schulzentrum|Freudenstadt
Industriegebiet|Freudenstadt Hbf|Grüntal-Wittlensweiler|Dornstetten-
Aach|Dornstetten|Schopfloch(b Freudenstadt)|Bittelbronn|Horb-
Heiligenfeld|Hochdorf(b Horb)|Eutingen Nord
72
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim
Nord|Durmersheim|Bietigheim(Baden)|Ötigheim|Rastatt|Rastatt
Beinle|Kuppenheim|Bischweier|Bad Rotenfels Schloss|Bad Rotenfels Bf|Bad
Rotenfels Weinbrennerstraße|Gaggenau Bf|Gaggenau Mercedes-Benz
Werk|Ottenau|Hörden|Gernsbach Bf|Gernsbach
Mitte|Obertsrot|Hilpertsau|Weisenbach|Au im Murgtal|Langenbrand|Gausbach|Forbach
(Schwarzw)|Raumünzach|Kirschbaumwasen|Schönmünzach|Schwarzenberg|Huzenbach|Röt|H
eselbach|Klosterreichenbach|Baiersbronn Schule|Baiersbronn Bf|Friedrichstal b
Freudenstadt|Freudenstadt Stadt|Freudenstadt Schulzentrum|Freudenstadt
Industriegebiet
67
Tullastraße/Alter Schlachthof, Karlsruhe|Gottesauer Platz/BGV,
Karlsruhe|Durlacher Tor/KIT-Campus Süd (U), Karlsruhe|Kronenplatz (U),
Karlsruhe|Marktplatz (Pyramide U), Karlsruhe|Ettlinger Tor/Staatstheater (U),
Karlsruhe|Kongresszentrum (U), Karlsruhe|Augartenstraße, Karlsruhe|Poststraße,
Karlsruhe|Karlsruhe Bahnhofsvorplatz|Karlsruhe Albtalbahnhof|Forchheim(b
Karlsruhe)|Durmersheim Nord|Durmersheim|Bietigheim(Baden)|Ötigheim
```

[64]:
```python
# Extraktion der Fokus-Fahrt für den exakten Abgleich der Text-Stationen
df_beweis = df_original_ds[
    (df_original_ds["ID_part1"] == 8619870746781998415) &
    (df_original_ds["ID_part2"] == 2407081057)
```

```
].sort_values("ID_part3")

# Berechnung der Anzahl der Bahnhöfe im Text von 'path' (Anzahl der Pipes + 1)
df_beweis["stationen_im_text"] = df_beweis["path"].apply(lambda x: len(str(x).
  ↪split("|")) if pd.notnull(x) else 0)

# Gegenüberstellung der Zeilen-Indizes und der tatsächlichen Text-Inhalte
print("Vergleich: Systemischer Index vs. Tatsächliche Stationsanzahl im Text:")
print(df_beweis[["ID_part3", "stationen_im_text", "arrival_plan"]])
```

```
Vergleich: Systemischer Index vs. Tatsächliche Stationsanzahl im Text:
        ID_part3  stationen_im_text         arrival_plan
115812        12                 11  2024-07-08 11:19:00
114741        14                 13  2024-07-08 11:26:00
113433        15                 14  2024-07-08 11:28:00
122373        16                 15  2024-07-08 11:31:00
122892        17                 16  2024-07-08 11:37:00
147126        51                 50  2024-07-08 13:07:00
146371        60                 59  2024-07-08 13:50:00
```

4.3.2 Zusammenfassung der ID-Transformation und strukturellen Qualitätsanalyse

Die technische Überführung der zusammengesetzten ID-Zeichenkette in separate Segmente und die anschließende mathematische Validierung führten zu folgenden Ergebnissen:

- **Verlustfreie Transformation und Ressourceneffizienz:** Die Aufspaltung in die drei Spalten `ID_part1`, `ID_part2` und `ID_part3` erfolgte ohne Informationsverlust. Die kombinatorische Eindeutigkeit wurde mit einer unveränderten Kardinalität von exakt 2.029.894 Datensätzen verifiziert. Durch die Eliminierung des ursprünglichen String-Attributs und den anschließenden Cast der Segmente in den Datentyp `Int64` wurde der Arbeitsspeicherbedarf des Dataframes von 947,4 MB auf 920,2 MB reduziert (direkte RAM-Ersparnis von 27,2 MB).
- **Identifikation systemischer Datenlücken:** Die vertiefte Sequenzanalyse unter Einbeziehung des Fahrtverlaufs (`path`) lieferte den empirischen Nachweis einer unvollständigen Datenbasis. Am Beispiel einer validierten Fokus-Fahrt (`ID_part1: 8619870746781998415`) expandiert die Stationsanzahl im Textattribut beim Übergang von Sequenzindex 51 auf Index 60 um exakt neun physische Haltepunkte. Im Datensatz selbst fehlen jedoch die korrespondierenden Zeilen für die Indizes 52 bis 59 vollständig.
- **Kausalitäts-Hypothese zur Netzdichte:** Die Auslassung dieser Zwischenstopps korreliert mit den vorab analysierten Bahnhofskategorien. Es ist die begründete Annahme abzuleiten, dass kleinere Haltepunkte (niedrigere Kategorisierungsstufen) im vorliegenden historischen Datensatz systemisch nicht aufgezeichnet wurden.

Eine lückenlose und saubere Abbildung des Schienennetzes ist allein auf Basis dieser Datenbasis somit unmöglich. Befund und Hypothese bestätigen nachdrücklich den im Projektvorschlag formulierten datenkritischen Ansatz und untermauern die methodische Notwendigkeit der geplanten Phase 3 zur selektiven Beschaffung externer API-Referenzdaten.

4.3.3 Methodisches Vorgehen zur temporalen Typkonvertierung

Nach der Bereinigung der ID-Struktur erfordert die Vorbereitung der explorativen Datenanalyse die Transformation der vier zeitbezogenen Attribute `arrival_plan`, `departure_plan`, `arrival_change` und `departure_change`. Diese Spalten liegen in der Rohdatenbasis als Zeichenketten (Strings) vor und müssen in echte Datetime-Objekte (`datetime64[ns]`) überführt werden.

Die Konvertierung bildet die mathematische Voraussetzung, um im weiteren Projektverlauf exakte Berechnungen von Zeitdifferenzen, Verspätungsminuten und Anschlussrisiken durchführen zu können. Da aus der technischen Inventur bereits bekannt ist, dass diese Spalten signifikante Fehlstellen aufweisen (z. B. fehlende Ankunftszeiten an Startbahnhöfen), wird bei der Transformation ein restriktiver Ansatz gewählt. Die Konvertierung muss so implementiert werden, dass vorhandene Nullwerte (`NaN`/`None`) datentreu als systemische Zeitstempel-Fehlstellen (`NaT` - *Not a Time*) erhalten bleiben, um nachfolgende statistische Aggregationen nicht zu verzerren.

[65]:
```
# Definition der Spaltenpaare für die temporäre Konvertierung laut Ablaufplan
zeit_spalten_mapping = {
    "arrival_plan": "arrival_plan_dt",
    "departure_plan": "departure_plan_dt",
    "arrival_change": "arrival_change_dt",
    "departure_change": "departure_change_dt"
}

# Konvertierung der Strings in zeitzonen-neutrale Datetime-Objekte
for alt_spalte, neu_spalte in zeit_spalten_mapping.items():
    # Direktes Einlesen ohne Zeitzonenzuweisung beläuft sich auf␣
 ↪zeitzonen-neutrale Objekte
    df_original_ds[neu_spalte] = pd.to_datetime(df_original_ds[alt_spalte],␣
 ↪errors="coerce")

# Überprüfung der Datentypen der neu generierten Spalten
print("Datentypen der neu generierten Spalten:")
print(df_original_ds[list(zeit_spalten_mapping.values())].dtypes)
```

```
Datentypen der neu generierten Spalten:
arrival_plan_dt        datetime64[us]
departure_plan_dt      datetime64[us]
arrival_change_dt      datetime64[us]
departure_change_dt    datetime64[us]
dtype: object
```

[66]:
```
# Auswahl der identischen 20 Zeilen für den direkten Vergleich der Formate
vergleichs_spalten = [
    "arrival_plan", "arrival_plan_dt",
    "departure_plan", "departure_plan_dt",
    "arrival_change", "arrival_change_dt",
    "departure_change", "departure_change_dt"
]
```

```
df_original_ds[vergleichs_spalten].iloc[200000:200020]
```

[66]:

	arrival_plan	arrival_plan_dt	departure_plan
200000	2024-07-08 16:28:00	2024-07-08 16:28:00	2024-07-08 16:29:00
200001	2024-07-08 16:48:00	2024-07-08 16:48:00	2024-07-08 16:49:00
200002	2024-07-08 16:11:00	2024-07-08 16:11:00	2024-07-08 16:11:00
200003	NaN	NaT	2024-07-08 16:13:00
200004	2024-07-08 16:00:00	2024-07-08 16:00:00	2024-07-08 16:03:00
200005	2024-07-08 16:19:00	2024-07-08 16:19:00	2024-07-08 16:23:00
200006	2024-07-08 16:56:00	2024-07-08 16:56:00	2024-07-08 16:59:00
200007	NaN	NaT	2024-07-08 16:33:00
200008	NaN	NaT	2024-07-08 16:53:00
200009	2024-07-08 16:15:00	2024-07-08 16:15:00	2024-07-08 16:19:00
200010	2024-07-08 16:36:00	2024-07-08 16:36:00	2024-07-08 16:39:00
200011	2024-07-08 16:08:00	2024-07-08 16:08:00	2024-07-08 16:08:00
200012	2024-07-08 16:50:00	2024-07-08 16:50:00	2024-07-08 16:50:00
200013	2024-07-08 16:20:00	2024-07-08 16:20:00	2024-07-08 16:20:00
200014	2024-07-08 16:38:00	2024-07-08 16:38:00	2024-07-08 16:38:00
200015	2024-07-08 16:21:00	2024-07-08 16:21:00	2024-07-08 16:22:00
200016	2024-07-08 16:15:00	2024-07-08 16:15:00	2024-07-08 16:16:00
200017	NaN	NaT	2024-07-08 16:41:00
200018	2024-07-08 16:02:00	2024-07-08 16:02:00	2024-07-08 16:03:00
200019	2024-07-08 16:36:00	2024-07-08 16:36:00	2024-07-08 16:37:00

	departure_plan_dt	arrival_change	arrival_change_dt
200000	2024-07-08 16:29:00	2024-07-08 16:28:00	2024-07-08 16:28:00
200001	2024-07-08 16:49:00	2024-07-08 16:50:00	2024-07-08 16:50:00
200002	2024-07-08 16:11:00	2024-07-08 16:11:00	2024-07-08 16:11:00
200003	2024-07-08 16:13:00	NaN	NaT
200004	2024-07-08 16:03:00	2024-07-08 16:03:00	2024-07-08 16:03:00
200005	2024-07-08 16:23:00	2024-07-08 16:19:00	2024-07-08 16:19:00
200006	2024-07-08 16:59:00	2024-07-08 17:09:00	2024-07-08 17:09:00
200007	2024-07-08 16:33:00	NaN	NaT
200008	2024-07-08 16:53:00	NaN	NaT
200009	2024-07-08 16:19:00	2024-07-08 16:22:00	2024-07-08 16:22:00
200010	2024-07-08 16:39:00	2024-07-08 16:55:00	2024-07-08 16:55:00
200011	2024-07-08 16:08:00	2024-07-08 16:08:00	2024-07-08 16:08:00
200012	2024-07-08 16:50:00	2024-07-08 16:50:00	2024-07-08 16:50:00
200013	2024-07-08 16:20:00	2024-07-08 16:32:00	2024-07-08 16:32:00
200014	2024-07-08 16:38:00	2024-07-08 16:44:00	2024-07-08 16:44:00
200015	2024-07-08 16:22:00	2024-07-08 16:21:00	2024-07-08 16:21:00
200016	2024-07-08 16:16:00	2024-07-08 16:17:00	2024-07-08 16:17:00
200017	2024-07-08 16:41:00	NaN	NaT
200018	2024-07-08 16:03:00	2024-07-08 16:03:00	2024-07-08 16:03:00
200019	2024-07-08 16:37:00	2024-07-08 16:44:00	2024-07-08 16:44:00

	departure_change	departure_change_dt

```
200000  2024-07-08 16:29:00 2024-07-08 16:29:00
200001  2024-07-08 16:51:00 2024-07-08 16:51:00
200002  2024-07-08 16:12:00 2024-07-08 16:12:00
200003  2024-07-08 16:13:00 2024-07-08 16:13:00
200004  2024-07-08 16:05:00 2024-07-08 16:05:00
200005  2024-07-08 16:23:00 2024-07-08 16:23:00
200006  2024-07-08 17:10:00 2024-07-08 17:10:00
200007  2024-07-08 16:33:00 2024-07-08 16:33:00
200008                  NaN                 NaT
200009  2024-07-08 16:22:00 2024-07-08 16:22:00
200010  2024-07-08 16:55:00 2024-07-08 16:55:00
200011  2024-07-08 16:09:00 2024-07-08 16:09:00
200012  2024-07-08 16:51:00 2024-07-08 16:51:00
200013  2024-07-08 16:32:00 2024-07-08 16:32:00
200014  2024-07-08 16:45:00 2024-07-08 16:45:00
200015  2024-07-08 16:22:00 2024-07-08 16:22:00
200016  2024-07-08 16:18:00 2024-07-08 16:18:00
200017                  NaN                 NaT
200018  2024-07-08 16:04:00 2024-07-08 16:04:00
200019  2024-07-08 16:45:00 2024-07-08 16:45:00
```

[67]:
```
# Abgleich der Fehlstellen zwischen den alten String-Spalten und den neuen␣
 ↪Datetime-Spalten
print("Anzahl der Fehlstellen in den ursprünglichen String-Spalten:")
print(df_original_ds[list(zeit_spalten_mapping.keys())].isnull().sum())

print("\nAnzahl der NaT-Fehlstellen in den neuen Datetime-Spalten:")
print(df_original_ds[list(zeit_spalten_mapping.values())].isnull().sum())
```

```
Anzahl der Fehlstellen in den ursprünglichen String-Spalten:
arrival_plan        211355
departure_plan           0
arrival_change      475630
departure_change    339926
dtype: int64

Anzahl der NaT-Fehlstellen in den neuen Datetime-Spalten:
arrival_plan_dt        211355
departure_plan_dt           0
arrival_change_dt      475630
departure_change_dt    339926
dtype: int64
```

[68]:
```
# Berechnung des Arbeitsspeicherbedarfs in Megabyte (MB) vor dem Entfernen der␣
 ↪alten Spalten
ram_alt_strings = df_original_ds[list(zeit_spalten_mapping.keys())].
 ↪memory_usage(deep=True).sum() / 1024**2
```

```
ram_neue_datetime = df_original_ds[list(zeit_spalten_mapping.values())].
  ↪memory_usage(deep=True).sum() / 1024**2

print(f"Arbeitsspeicherbedarf der alten String-Spalten: {ram_alt_strings:.2f}␣
  ↪MB")
print(f"Arbeitsspeicherbedarf der neuen Datetime-Spalten: {ram_neue_datetime:.
  ↪2f} MB")
print(f"Erwartete RAM-Ersparnis nach dem Ersetzen: {ram_alt_strings -␣
  ↪ram_neue_datetime:.2f} MB")
```

```
Arbeitsspeicherbedarf der alten String-Spalten: 194.44 MB
Arbeitsspeicherbedarf der neuen Datetime-Spalten: 62.91 MB
Erwartete RAM-Ersparnis nach dem Ersetzen: 131.54 MB
```

[69]:
```
df_original_ds.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 26 columns):
 #   Column                Dtype
---  ------                -----
 0   line                  str
 1   path                  str
 2   eva_nr                int64
 3   category              int64
 4   station               str
 5   state                 str
 6   city                  str
 7   zip                   int64
 8   long                  float64
 9   lat                   float64
 10  arrival_plan          str
 11  departure_plan        str
 12  arrival_change        str
 13  departure_change      str
 14  arrival_delay_m       int64
 15  departure_delay_m     int64
 16  info                  str
 17  arrival_delay_check   str
 18  departure_delay_check str
 19  ID_part1              Int64
 20  ID_part2              Int64
 21  ID_part3              Int64
 22  arrival_plan_dt       datetime64[us]
 23  departure_plan_dt     datetime64[us]
 24  arrival_change_dt     datetime64[us]
 25  departure_change_dt   datetime64[us]
dtypes: Int64(3), datetime64[us](4), float64(2), int64(5), str(12)
```

```
memory usage: 983.1 MB
```

[70]:
```python
# Definition der zu entfernenden alten String-Spalten
alte_string_spalten = ["arrival_plan", "departure_plan", "arrival_change",␣
 ↪"departure_change"]

# Entfernen der alten Spalten zur Speicherbereinigung
df_original_ds = df_original_ds.drop(columns=alte_string_spalten)

# Umbenennung der Datetime-Spalten auf die ursprünglichen Namen
spalten_umbenennung = {
    "arrival_plan_dt": "arrival_plan",
    "departure_plan_dt": "departure_plan",
    "arrival_change_dt": "arrival_change",
    "departure_change_dt": "departure_change"
}
df_original_ds = df_original_ds.rename(columns=spalten_umbenennung)

# Berechnung des aktuellen Speicherbedarfs nach beiden Transformationen (ID +␣
 ↪Datum)
aktueller_speicher_mb = df_original_ds.memory_usage(deep=True).sum() / 1024**2
gesamt_ersparnis_mb = DS_MEMORY_USAGE_RAW_MB - aktueller_speicher_mb

print(f"Ursprünglicher RAM-Bedarf (Rohdaten): {DS_MEMORY_USAGE_RAW_MB:.2f} MB")
print(f"Aktueller RAM-Bedarf nach Transformationen: {aktueller_speicher_mb:.2f}␣
 ↪MB")
print(f"Kumulierte Arbeitsspeicher-Ersparnis des Projekts: {gesamt_ersparnis_mb:
 ↪.2f} MB")
```

```
Ursprünglicher RAM-Bedarf (Rohdaten): 947.39 MB
Aktueller RAM-Bedarf nach Transformationen: 788.63 MB
Kumulierte Arbeitsspeicher-Ersparnis des Projekts: 158.76 MB
```

[71]:
```python
# Extraktion des Wochentagsnamens (auf Deutsch) aus dem geplanten Abfahrtsdatum
# Der nachfolgende Cast zu 'category' minimiert den Speicherbedarf im Vergleich␣
 ↪zu reinen Strings
df_original_ds["departure_weekday"] = (
    df_original_ds["departure_plan"]
    .dt.day_name(locale="de_DE")
    .astype("category")
)

# Kontrolle der neuen Spalte und deren eindeutigen Ausprägungen
print("Verteilung der Datensätze nach Wochentagen:")
print(df_original_ds["departure_weekday"].value_counts())
```

```
Verteilung der Datensätze nach Wochentagen:
departure_weekday
```

```
Mittwoch      310900
Donnerstag    309463
Dienstag      309376
Freitag       307847
Montag        307613
Samstag       265771
Sonntag       250387
Name: count, dtype: int64
```

[72]:
```
df_original_ds.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 23 columns):
 #   Column                 Dtype
---  ------                 -----
 0   line                   str
 1   path                   str
 2   eva_nr                 int64
 3   category               int64
 4   station                str
 5   state                  str
 6   city                   str
 7   zip                    int64
 8   long                   float64
 9   lat                    float64
 10  arrival_delay_m        int64
 11  departure_delay_m      int64
 12  info                   str
 13  arrival_delay_check    str
 14  departure_delay_check  str
 15  ID_part1               Int64
 16  ID_part2               Int64
 17  ID_part3               Int64
 18  arrival_plan           datetime64[us]
 19  departure_plan         datetime64[us]
 20  arrival_change         datetime64[us]
 21  departure_change       datetime64[us]
 22  departure_weekday      category
dtypes: Int64(3), category(1), datetime64[us](4), float64(2), int64(5), str(8)
memory usage: 790.6 MB
```

[73]:
```
# Finale Berechnung des aktuellen Speicherbedarfs nach dem Einfügen der␣
 ↪Wochentage
aktueller_speicher_final_mb = df_original_ds.memory_usage(deep=True).sum() /␣
 ↪1024**2
finale_gesamt_ersparnis_mb = DS_MEMORY_USAGE_RAW_MB -␣
 ↪aktueller_speicher_final_mb
```

```
print(f"Ursprünglicher RAM-Bedarf (Rohdaten): {DS_MEMORY_USAGE_RAW_MB:.2f} MB")
print(f"Finaler RAM-Bedarf (inkl. Wochentag): {aktueller_speicher_final_mb:.2f}␣
 ↪MB")
print(f"Tatsächliche finale Arbeitsspeicher-Ersparnis:␣
 ↪{finale_gesamt_ersparnis_mb:.2f} MB")
```

```
Ursprünglicher RAM-Bedarf (Rohdaten): 947.39 MB
Finaler RAM-Bedarf (inkl. Wochentag): 790.59 MB
Tatsächliche finale Arbeitsspeicher-Ersparnis: 156.79 MB
```

[74]:
```
# Ermittlung der minimalen und maximalen Zeitstempel für geplante Ankünfte und␣
 ↪Abfahrten
print("Zeitliche Grenzen des Datensatzes (Geplante Zeiten):")
print(f"Früheste geplante Ankunft:  {df_original_ds['arrival_plan'].min()}")
print(f"Späteste geplante Ankunft:   {df_original_ds['arrival_plan'].max()}")
print(f"Früheste geplante Abfahrt:  {df_original_ds['departure_plan'].min()}")
print(f"Späteste geplante Abfahrt:   {df_original_ds['departure_plan'].max()}")
```

```
Zeitliche Grenzen des Datensatzes (Geplante Zeiten):
Früheste geplante Ankunft:  2024-07-07 23:37:00
Späteste geplante Ankunft:   2024-07-14 23:59:00
Früheste geplante Abfahrt:  2024-07-08 00:00:00
Späteste geplante Abfahrt:   2024-07-15 00:10:00
```

[75]:
```
# Ermittlung des absoluten Minimums und Maximums über alle Datumsspalten hinweg
absolutes_minimum = min(df_original_ds["arrival_plan"].min(),␣
 ↪df_original_ds["departure_plan"].min())
absolutes_maximum = max(df_original_ds["arrival_plan"].max(),␣
 ↪df_original_ds["departure_plan"].max())

# Berechnung der exakten zeitlichen Gesamtdistanz des Datensatzes
globale_zeit_differenz = absolutes_maximum - absolutes_minimum

print(f"Absoluter zeitlicher Startpunkt:  {absolutes_minimum}")
print(f"Absoluter zeitlicher Endpunkt:   {absolutes_maximum}")

print(f"\nExakte zeitliche Gesamtdistanz des Datensatzes:␣
 ↪{globale_zeit_differenz}")
```

```
Absoluter zeitlicher Startpunkt:  2024-07-07 23:37:00
Absoluter zeitlicher Endpunkt:   2024-07-15 00:10:00

Exakte zeitliche Gesamtdistanz des Datensatzes: 7 days 00:33:00
```

4.3.4 Zusammenfassung der temporalen Konvertierung und zeithorizontalen Validierung

Die Überführung der zeitbezogenen Attribute in ein zeitzonen-neutrales Datetime-Format (`datetime64`) und die anschließende Extraktion der wochentagsbezogenen Strukturen führten zu folgenden Ergebnissen:

- **Ressourcenoptimierung und Datentreue:** Durch das Entfernen der ursprünglichen String-Spalten und das Einfügen der speichereffizienten, kategorialen Spalte `departure_weekday` konnte der Arbeitsspeicherbedarf des Gesamtdatensatzes dauerhaft von 947,39 MB auf 790,59 MB gesenkt werden. Dies entspricht einer dokumentierten finalen RAM-Ersparnis von 156,79 MB bei vollständiger Erhaltung aller systemischen Fehlstellen (`NaT`).
- **Wochentagsverteilung und Verkehrsaufkommen:** Die quantitative Verteilung der Datensätze nach Wochentagen dokumentiert ein stabiles Verkehrsaufkommen an Werktagen (ca. 307.000 bis 311.000 Datensätze pro Tag). Am Wochenende ist ein signifikanter Rückgang des operativen Betriebs zu verzeichnen, wobei der Sonntag mit 250.387 Datensätzen das geringste Datenvolumen aufweist.
- **Grenzspezifikation des Zeithorizonts:** Die globale Extremwertanalyse über alle temporalen Attribute definiert den exakten operativen Untersuchungszeitraum von exakt 7 Tagen und 33 Minuten. Der Erfassungsraum beginnt am 07.07.2024 um 23:37:00 Uhr und endet am 15.07.2024 um 00:10:00 Uhr.

Die zeitliche Ausdehnung bestätigt, dass der Datensatz auf einer exakten Kernwoche (08.07.2024 bis 14.07.2024) basiert. Die Randdaten erklären sich logisch durch einen Überhang von 23 Minuten in der Vergangenheit vor dem offiziellen Wochenstart sowie 10 Minuten in der Zukunft nach dem offiziellen Wochenendzeitpunkt. Dies belegt die verlustfreie Erfassung von tagesübergreifenden Zugläufen an den Schnittstellen des Untersuchungszeitraums.

4.3.5 Methodisches Vorgehen zur binären Konvertierung der Statusmerkmale

Nach der temporalen Aufbereitung wird die datentechnische Optimierung der qualitativen Statusspalten `arrival_delay_check` und `departure_delay_check` durchgeführt. Die empirische Vorprüfung hat bereits gezeigt, dass beide Spalten eine vollständige Datenabdeckung aufweisen (null Fehlwerte) und ausschließlich die zwei Ausprägungen `'on_time'` und `'delay'` annehmen.

Aufgrund dieser strikt binären Struktur ist die Beibehaltung als zeichenkettenbasierte Attribute (Strings) speichertechnisch ineffizient. Zur Vorbereitung der explorativen Analyse werden die Spalten in den ressourcenschonenden Boolean-Datentyp überführt. Um eine durchgängige und standardisierte DataFrame-Namenskonvention zu wahren, erhalten die neuen Spalten das Präfix `is_` und werden so codiert, dass der Zustand einer Verspätung (`'delay'`) dem logischen Wert `True` entspricht. Die Transformation erfolgt schrittweise über temporäre Spalten, um eine anschließende Validierung und Arbeitsspeicher-Kalkulation zu ermöglichen.

[76]:
```
# Definition des binären Mappings für die Statuskonvertierung
status_bool_mapping = {"delay": True, "on_time": False}

# Erstellung der neuen Boolean-Spalten laut Namenskonvention
df_original_ds["is_arrival_delayed"] = df_original_ds["arrival_delay_check"].
  ↪map(status_bool_mapping).astype(bool)
```

```
df_original_ds["is_departure_delayed"] =␣
 ↪df_original_ds["departure_delay_check"].map(status_bool_mapping).astype(bool)

# Überprüfung der Datentypen der neu generierten Spalten
print("Datentypen der neuen Boolean-Spalten:")
print(df_original_ds[["is_arrival_delayed", "is_departure_delayed"]].dtypes)
```

```
Datentypen der neuen Boolean-Spalten:
is_arrival_delayed      bool
is_departure_delayed    bool
dtype: object
```

[77]:
```
# Auswahl von 20 Zeilen für den direkten Vergleich der Formate
vergleichs_spalten_bool = [
    "arrival_delay_check", "is_arrival_delayed",
    "departure_delay_check", "is_departure_delayed"
]

df_original_ds[vergleichs_spalten_bool].iloc[200000:200020]
```

[77]:

	arrival_delay_check	is_arrival_delayed	departure_delay_check	is_departure_delayed
200000	on_time	False	on_time	False
200001	on_time	False	on_time	False
200002	on_time	False	on_time	False
200003	on_time	False	on_time	False
200004	on_time	False	on_time	False
200005	on_time	False	on_time	False
200006	delay	True	delay	
200007	on_time	False	on_time	
200008	on_time	False	on_time	
200009	delay	True	on_time	
200010	delay	True	delay	
200011	on_time	False	on_time	
200012	on_time	False	on_time	
200013	delay	True	delay	
200014	delay	True	delay	
200015	on_time	False	on_time	
200016	on_time	False	on_time	
200017	on_time	False	on_time	
200018	on_time	False	on_time	
200019	delay	True	delay	

```
200004                False
200005                False
200006                 True
200007                False
200008                False
200009                False
200010                 True
200011                False
200012                False
200013                 True
200014                 True
200015                False
200016                False
200017                False
200018                False
200019                 True
```

[78]:
```
# Verteilungsprüfung für die Ankunftsverspätung
print("Eindeutige Werteverteilung für 'is_arrival_delayed':")
print(df_original_ds["is_arrival_delayed"].value_counts(dropna=False))

# Verteilungsprüfung für die Abfahrtsverspätung
print("\nEindeutige Werteverteilung für 'is_departure_delayed':")
print(df_original_ds["is_departure_delayed"].value_counts(dropna=False))
```

```
Eindeutige Werteverteilung für 'is_arrival_delayed':
is_arrival_delayed
False    1950404
True      110953
Name: count, dtype: int64

Eindeutige Werteverteilung für 'is_departure_delayed':
is_departure_delayed
False    1950009
True      111348
Name: count, dtype: int64
```

[79]:
```
# Berechnung des Arbeitsspeicherbedarfs in Megabyte (MB)
ram_check_strings = df_original_ds[["arrival_delay_check",␣
 ↪"departure_delay_check"]].memory_usage(deep=True).sum() / 1024**2
ram_check_bools = df_original_ds[["is_arrival_delayed",␣
 ↪"is_departure_delayed"]].memory_usage(deep=True).sum() / 1024**2

print(f"Arbeitsspeicherbedarf der alten String-Spalten: {ram_check_strings:.2f}␣
 ↪MB")
print(f"Arbeitsspeicherbedarf der neuen Boolean-Spalten: {ram_check_bools:.2f}␣
 ↪MB")
```

```
print(f"Erwartete zusätzliche RAM-Ersparnis nach Bereinigung:␣
 ↪{ram_check_strings - ram_check_bools:.2f} MB")
```

```
Arbeitsspeicherbedarf der alten String-Spalten: 58.55 MB
Arbeitsspeicherbedarf der neuen Boolean-Spalten: 3.93 MB
Erwartete zusätzliche RAM-Ersparnis nach Bereinigung: 54.62 MB
```

[80]:
```
# Definition der zu entfernenden alten String-Spalten
alte_check_spalten = ["arrival_delay_check", "departure_delay_check"]

# Entfernen der alten Spalten zur Speicherbereinigung
df_original_ds = df_original_ds.drop(columns=alte_check_spalten)

# Berechnung des aktuellen Speicherbedarfs nach allen Transformationen (ID +␣
 ↪Datum + Boolean)
aktueller_speicher_final_mb = df_original_ds.memory_usage(deep=True).sum() /␣
 ↪1024**2
gesamt_ersparnis_final_mb = DS_MEMORY_USAGE_RAW_MB - aktueller_speicher_final_mb

print(f"Ursprünglicher RAM-Bedarf (Rohdaten): {DS_MEMORY_USAGE_RAW_MB:.2f} MB")
print(f"Finaler RAM-Bedarf nach allen Bereinigungen:␣
 ↪{aktueller_speicher_final_mb:.2f} MB")
print(f"Kumulierte Gesamt-Arbeitsspeicher-Ersparnis: {gesamt_ersparnis_final_mb:
 ↪.2f} MB")
```

```
Ursprünglicher RAM-Bedarf (Rohdaten): 947.39 MB
Finaler RAM-Bedarf nach allen Bereinigungen: 735.97 MB
Kumulierte Gesamt-Arbeitsspeicher-Ersparnis: 211.41 MB
```

4.3.6 Zusammenfassung der binären Statuskonvertierung und finalen Speicherbilanz

Die datentechnische Überführung der qualitativen Prüfmerkmale in den Boolean-Datentyp sowie die finale Konsolidierung der Pipeline führten zu folgenden Ergebnissen:

- **Vollständigkeit und Verteilungscharakteristik:** Die Konvertierung der Attribute `arrival_delay_check` und `departure_delay_check` in die neuen Variablen `is_arrival_delayed` und `is_departure_delayed` erfolgte ohne Datenverlust über alle 2.061.357 Datensätze hinweg. Die Verteilungsanalyse dokumentiert eine stabile Pünktlichkeitsrate im Schienennetz. Für die Ankünfte wurden 1.950.404 reguläre Fahrten gegenüber 110.953 Verspätungsereignissen erfasst, während die Abfahrten 1.950.009 pünktliche Ereignisse und 111.348 Verspätungen aufweisen.
- **Signifikante Steigerung der Ressourceneffizienz:** Durch den gezielten Austausch der zeichenkettenbasierten Attribute gegen native Boolean-Werte sank der isolierte Speicherbedarf dieser Spalten von 58,55 MB auf lediglich 3,93 MB. Dies generierte eine unmittelbare RAM-Ersparnis von 54,62 MB für diesen spezifischen Teilschritt.
- **Gesamtbilanz der strukturellen Bereinigungsphase:** Der Abschluss aller integrierten Transformationsprozesse (ID-Segmentierung, zeitzonen-neutrale Datumskonvertierung, Wochentagsextraktion und Boolean-Überführung) reduziert den globalen Arbeitsspeicherbedarf des Dataframes dauerhaft von ursprünglich 947,39 MB auf 735,97 MB.

Die akkumulierte Gesamt-Arbeitsspeicher-Ersparnis von exakt 211,41 MB optimiert die lokale Ausführungsgeschwindigkeit innerhalb der JupyterLab-Umgebung maßgeblich. Damit ist die datenkritische Vorbereitungsphase der technischen Inventur erfolgreich abgeschlossen. Der Datensatz befindet sich in einem konsistenten, speichereffizienten und mathematisch validierten Zustand, welcher die performante Durchführung der komplexen explorativen Datenanalyse (EDA) sowie die geplante netzwerkbasierte Anschlussanalyse garantiert.

[81]:
```
df_original_ds.head(20)
```

[81]:

	line	path	eva_nr	\
0	20	Stolberg(Rheinl)Hbf Gl.44|Eschweiler-St.Jöris|…	8000001	
1	18	NaN	8000001	
2	1	Hamm(Westf)Hbf|Kamen|Kamen-Methler|Dortmund-Ku…	8000406	
3	18	Aachen Hbf	8000404	
4	33	Herzogenrath|Kohlscheid	8000404	
5	4	Herzogenrath	8000404	
6	RB33	Lindern|Geilenkirchen|Übach-Palenberg|Herzogen…	8000404	
7	18	Liège-Guillemins|Bressoux|Vise|Eijsden|Maastri…	8000404	
8	13	NaN	8000002	
9	8	Bremen Hbf|Bremen-Sebaldsbrück|Bremen-Mahndorf	8000413	
10	69	Münster(Westf)Hbf|Münster-Hiltrup|Rinkerode|Dr…	8000441	
11	69	Bielefeld Hbf|Bielefeld-Brackwede|Isselhorst-A…	8000441	
12	7	Hannover Hbf|Hannover-Kleefeld|Hannover Karl-W…	8000443	
13	3	Hannover Hbf|Hannover-Kleefeld|Hannover Karl-W…	8000443	
14	3	Hildesheim Hbf|Harsum|Algermissen|Sehnde|Lehrte	8000443	
15	RB81	Hamburg Hasselbrook|Hamburg Wandsbeker Chausse…	8000446	
16	RB81	Hamburg Hasselbrook|Hamburg Wandsbeker Chausse…	8000446	
17	RB81	Bargteheide|Ahrensburg-Gartenholz	8000446	
18	RB13	Augsburg Hbf|Augsburg Haunstetterstraße|Augsbu…	8000452	
19	2	Nauen|Brieselang|Finkenkrug|Falkensee|Seegefeld	8080040	

	category	station	state	city	zip	\
0	2	Aachen Hbf	Nordrhein-Westfalen	Aachen	52064	
1	2	Aachen Hbf	Nordrhein-Westfalen	Aachen	52064	
2	4	Aachen-Rothe Erde	Nordrhein-Westfalen	Aachen	52066	
3	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	
4	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	
5	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	
6	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	
7	5	Aachen West	Nordrhein-Westfalen	Aachen	52072	
8	3	Aalen Hbf	Baden-Württemberg	Aalen	73430	
9	4	Achim	Niedersachsen	Achim	28832	
10	4	Ahlen (Westf)	Nordrhein-Westfalen	Ahlen	59227	
11	4	Ahlen (Westf)	Nordrhein-Westfalen	Ahlen	59227	
12	5	Ahlten (Han)	Niedersachsen	Lehrte	31275	
13	5	Ahlten (Han)	Niedersachsen	Lehrte	31275	
14	5	Ahlten (Han)	Niedersachsen	Lehrte	31275	

```
15         4         Ahrensburg   Schleswig-Holstein  Ahrensburg  22926
16         4         Ahrensburg   Schleswig-Holstein  Ahrensburg  22926
17         4         Ahrensburg   Schleswig-Holstein  Ahrensburg  22926
18         5            Aichach               Bayern     Aichach  86551
19         5       Albrechtshof               Berlin      Berlin  13591

         long        lat  …             ID_part1    ID_part2 ID_part3  \
0    6.091499  50.767800  …   1573967790757085557  2407072312       14
1    6.091499  50.767800  …    349781417030375472  2407080017        1
2    6.116475  50.770202  …   7157250219775883918  2407072120       25
3    6.070715  50.780360  …    349781417030375472  2407080017        2
4    6.070715  50.780360  …   1983158592123451570  2407080010        3
5    6.070715  50.780360  …  -5293934437045765939  2407080023        2
6    6.070715  50.780360  …   6845762881043426854  2407072357        6
7    6.070715  50.780360  …  -2100556839975301087  2407072307       13
8   10.096271  48.841013  …  -7696913984968518161  2407080037        1
9    9.030447  53.015990  …  -6027587483204218492  2407080013        4
10   7.895488  51.761062  …  -7723223610149163054  2407072310        9
11   7.895488  51.761062  …   5729393359484274537  2407080007        8
12   9.904613  52.376062  …   1640260421649276864  2407080004        5
13   9.904613  52.376062  …   8352762151701721590  2407080034        5
14   9.904613  52.376062  …  -1113872591615872448  2407072343        6
15  10.236206  53.669171  …   7148083203553802700  2407072356        5
16  10.236206  53.669171  …   5507673346685034119  2407072356        5
17  10.236206  53.669171  …  -6105012072686197130  2407080016        3
18  11.122721  48.461117  …   7734805089613574578  2407072359        7
19  13.128917  52.549396  …  -8761881508899340400  2407080008        6

          arrival_plan      departure_plan      arrival_change  \
0  2024-07-08 00:00:00 2024-07-08 00:01:00 2024-07-08 00:03:00
1                  NaT 2024-07-08 00:17:00                 NaT
2  2024-07-08 00:03:00 2024-07-08 00:04:00 2024-07-08 00:03:00
3  2024-07-08 00:20:00 2024-07-08 00:21:00                 NaT
4  2024-07-08 00:20:00 2024-07-08 00:21:00 2024-07-08 00:20:00
5  2024-07-08 00:30:00 2024-07-08 00:31:00 2024-07-08 00:30:00
6  2024-07-08 00:58:00 2024-07-08 00:58:00                 NaT
7  2024-07-08 00:37:00 2024-07-08 00:41:00 2024-07-08 00:37:00
8                  NaT 2024-07-08 00:37:00                 NaT
9  2024-07-08 00:27:00 2024-07-08 00:27:00 2024-07-08 01:16:00
10 2024-07-08 00:15:00 2024-07-08 00:16:00 2024-07-08 00:15:00
11 2024-07-08 00:39:00 2024-07-08 00:39:00 2024-07-08 00:39:00
12 2024-07-08 00:16:00 2024-07-08 00:16:00 2024-07-08 00:24:00
13 2024-07-08 00:46:00 2024-07-08 00:46:00 2024-07-08 00:46:00
14 2024-07-08 00:10:00 2024-07-08 00:11:00 2024-07-08 00:10:00
15 2024-07-08 00:34:00 2024-07-08 00:34:00                 NaT
16 2024-07-08 00:34:00 2024-07-08 00:34:00                 NaT
17 2024-07-08 00:39:00 2024-07-08 00:39:00                 NaT
```

```
18 2024-07-08 00:24:00 2024-07-08 00:25:00 2024-07-08 00:24:00
19 2024-07-08 00:28:00 2024-07-08 00:29:00 2024-07-08 00:28:00

      departure_change departure_weekday is_arrival_delayed  \
0  2024-07-08 00:04:00            Montag              False
1                  NaT            Montag              False
2  2024-07-08 00:04:00            Montag              False
3                  NaT            Montag              False
4  2024-07-08 00:21:00            Montag              False
5  2024-07-08 00:31:00            Montag              False
6                  NaT            Montag              False
7  2024-07-08 00:41:00            Montag              False
8  2024-07-08 00:37:00            Montag              False
9  2024-07-08 01:17:00            Montag               True
10 2024-07-08 00:16:00            Montag              False
11 2024-07-08 00:39:00            Montag              False
12 2024-07-08 00:24:00            Montag               True
13 2024-07-08 00:46:00            Montag              False
14 2024-07-08 00:11:00            Montag              False
15                 NaT            Montag              False
16                 NaT            Montag              False
17                 NaT            Montag              False
18 2024-07-08 00:25:00            Montag              False
19 2024-07-08 00:29:00            Montag              False

   is_departure_delayed
0                 False
1                 False
2                 False
3                 False
4                 False
5                 False
6                 False
7                 False
8                 False
9                  True
10                False
11                False
12                 True
13                False
14                False
15                False
16                False
17                False
18                False
19                False
```

```
[20 rows x 23 columns]
```

[82]:
```
df_original_ds.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 23 columns):
 #   Column                Dtype
---  ------                -----
 0   line                  str
 1   path                  str
 2   eva_nr                int64
 3   category              int64
 4   station               str
 5   state                 str
 6   city                  str
 7   zip                   int64
 8   long                  float64
 9   lat                   float64
 10  arrival_delay_m       int64
 11  departure_delay_m     int64
 12  info                  str
 13  ID_part1              Int64
 14  ID_part2              Int64
 15  ID_part3              Int64
 16  arrival_plan          datetime64[us]
 17  departure_plan        datetime64[us]
 18  arrival_change        datetime64[us]
 19  departure_change      datetime64[us]
 20  departure_weekday     category
 21  is_arrival_delayed    bool
 22  is_departure_delayed  bool
dtypes: Int64(3), bool(2), category(1), datetime64[us](4), float64(2), int64(5),
str(6)
memory usage: 736.0 MB
```

[83]:
```
df_original_ds.describe()
```

[83]:

	eva_nr	category	zip	long	lat	\
count	2.061357e+06	2.061357e+06	2.061357e+06	2.061357e+06	2.061357e+06	
mean	8.018267e+06	3.871748e+00	4.627924e+04	1.018367e+01	5.088240e+01	
min	8.000001e+06	1.000000e+00	1.067000e+03	6.070715e+00	4.741103e+01	
25%	8.001582e+06	3.000000e+00	1.810900e+04	8.494709e+00	4.935329e+01	
50%	8.004136e+06	4.000000e+00	4.705100e+04	9.944088e+00	5.108746e+01	
75%	8.010208e+06	5.000000e+00	7.080600e+04	1.209055e+01	5.247854e+01	
max	8.098360e+06	5.000000e+00	9.997400e+04	1.497908e+01	5.490684e+01	
std	3.178659e+04	1.037063e+00	2.821424e+04	2.273525e+00	1.792217e+00	

```
       arrival_delay_m  departure_delay_m              ID_part1  \
count     2.061357e+06       2.061357e+06             2061357.0
mean      1.176581e+00       1.223574e+00  -21086134748267028.0
min       0.000000e+00       0.000000e+00 -9223176950731764736.0
25%       0.000000e+00       0.000000e+00 -4590018462129540096.0
50%       0.000000e+00       0.000000e+00  -46576259786505424.0
75%       1.000000e+00       1.000000e+00  4567148400468366336.0
max       1.590000e+02       1.590000e+02  9223056977874851840.0
std       3.407859e+00       3.418300e+00  5327233908976233472.0

                ID_part2   ID_part3               arrival_plan  \
count          2061357.0  2061357.0                    1850002
mean   2407108685.218822  10.461123  2024-07-11 10:28:43.707714
min          2407031857.0        1.0         2024-07-07 23:37:00
25%          2407091412.0        4.0         2024-07-09 16:55:00
50%          2407110936.0        9.0         2024-07-11 10:13:00
75%          2407122331.0       15.0         2024-07-13 00:05:00
max          2407142359.0       60.0         2024-07-14 23:59:00
std          21612.742279   7.897363                         NaN

                   departure_plan              arrival_change  \
count                      2061357                     1585727
mean   2024-07-11 10:28:21.410645  2024-07-11 10:23:07.318687
min          2024-07-08 00:00:00         2024-07-07 23:39:00
25%          2024-07-09 16:53:00         2024-07-09 16:55:00
50%          2024-07-11 10:13:00         2024-07-11 10:02:00
75%          2024-07-13 00:05:00         2024-07-12 23:50:00
max          2024-07-15 00:10:00         2024-07-15 01:03:00
std                          NaN                         NaN

                 departure_change
count                     1721431
mean   2024-07-11 10:18:44.162989
min          2024-07-08 00:00:00
25%          2024-07-09 16:49:00
50%          2024-07-11 09:57:00
75%          2024-07-12 23:40:00
max          2024-07-15 01:04:00
std                          NaN
```

[84]:
```python
# Export des bereinigten DataFrames in das spaltenorientierte Parquet-Format
# index=False verhindert das Abspeichern einer redundanten Index-Spalte
df_original_ds.to_parquet(CLEANED_DS_PARQUET_PATH, index=False)

# Überprüfung des physischen Speicherplatzes auf der Festplatte
dateigroesse_mb = CLEANED_DS_PARQUET_PATH.stat().st_size / 1024**2
```

```
print(f"Bereinigter Datensatz erfolgreich gespeichert unter:
  ↪\n{CLEANED_DS_PARQUET_PATH}")
print(f"Physische Dateigröße auf der Festplatte: {dateigroesse_mb:.2f} MB")
```

```
Bereinigter Datensatz erfolgreich gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtrainrides_cleaned.parquet
Physische Dateigröße auf der Festplatte: 103.76 MB
```

4.4 Explorative Datenanalyse und qualitative Evaluierung des Schienenverkehrs

Nach der erfolgreichen datentechnischen Aufbereitung und Strukturierung der Rohdatenbasis wird in diesem Kapitel die explorative Datenanalyse (EDA) durchgeführt. Ziel dieses Abschnitts ist es nicht nur, grundlegende statistische Kennzahlen und Verteilungen zu ermitteln, sondern den Datensatz systematisch auf Anomalien, Muster und zeitliche Abhängigkeiten zu untersuchen.

Der Ansatz ist hierbei bewusst ergebnisoffen und datenkritisch gestaltet. Die grafischen und statistischen Auswertungen dienen als direktes Werkzeug, um die aus der technischen Inventur gewonnenen Vermutungen zu vertiefen. Die hieraus resultierenden Erkenntnisse bilden die fundierte Grundlage, um konkrete Folgefragen für die erweiterten Analysen – insbesondere bezüglich potenzieller Anschlussrisiken und der Notwendigkeit einer externen API-Referenzdatenbasis – zu formulieren.

4.4.1 Untersuchungsschwerpunkte der EDA

1. **Deskriptive Analyse der Verspätungsstrukturen:** Ermittlung von Verteilungskennzahlen der tatsächlichen Abweichungen im Schienennetz.
2. **Temporale Korrelationen:** Untersuchung von Verspätungsmustern im Verlauf der erfassten Kernwoche sowie im Vergleich der einzelnen Wochentage.
3. **Makroskopische Netzerkenntnisse:** Identifikation von signifikanten Knotenpunkten oder Strukturauffälligkeiten als Ausgangspunkt für die vertiefte Netzwerkbetrachtung.

4.4.2 Makroskopische Analyse der Bahnhofs-Kardinalitäten und Netzfrequenzen

Nach der datentechnischen Konsolidierung beginnt die explorative Phase mit der Untersuchung der räumlichen Knotenpunkte. Die vorangegangene technische Inventur hat bereits nachgewiesen, dass die Attribute `station` (Bahnhofsname) und `eva_nr` (eindeutige Stationskennung) eine perfekte mathematische Symmetrie von 1.996 einzigartigen Ausprägungen aufweisen.

Im ersten Analyseschritt wird die operative Frequenz – also die Anzahl der protokollierten Datensätze pro Bahnhof – untersucht. Hierzu wird die Verteilung der Stationsnennungen sowohl über eine qualitative Worthäufigkeitskarte (WordCloud) als auch über ein quantitatives Balkendiagramm der 20 am stärksten frequentierten Knotenpunkte visualisiert. Diese makroskopische Betrachtung dient dazu, die dominierenden Zentren im Datensatz zu identifizieren und die Verteilung der Datenpunkte auf strukturelle Auffälligkeiten zu prüfen.

[85]:
```
# Isolierung der eindeutigen Bahnhofs-Stammdaten zur Absicherung der Datenbasis
bahnhof_df = df_original_ds[['eva_nr', 'station', 'city', 'state', 'zip',␣
  ↪'long', 'lat']].drop_duplicates()

# Ermittlung der absoluten Häufigkeiten pro Bahnhof im Gesamtdatensatz
```

```
station_counts = df_original_ds['station'].value_counts()

# Generierung der WordCloud basierend auf den Frequenzen
wc_station = WordCloud(
    width=1200,
    height=900,
    background_color='white',
    colormap='viridis'
).generate_from_frequencies(station_counts.to_dict())

# Visualisierung der WordCloud
plt.figure(figsize=(12, 9))
plt.imshow(wc_station, interpolation='bilinear')
plt.axis('off')
plt.tight_layout()
plt.show()
```



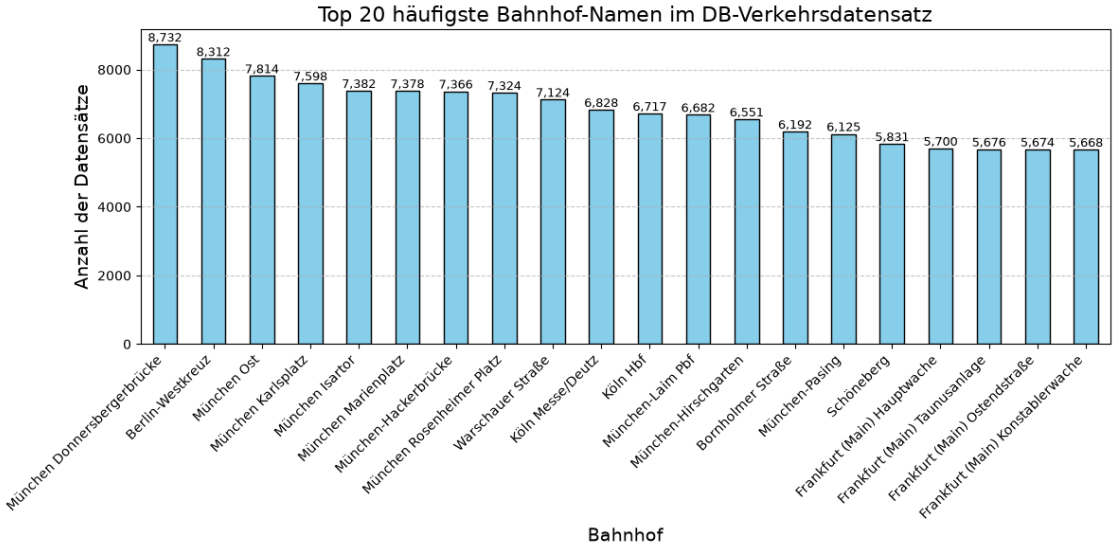
[86]:
```
# Selektion der 20 am häufigsten erfassten Bahnhöfe
top20_station_counts = df_original_ds['station'].value_counts().head(20)
```

```python
# Erstellung des Balkendiagramms
plt.figure(figsize=(12, 6))
top20_station_counts.plot(kind='bar', color='skyblue', edgecolor='black')

# Diagramm-Formatierung nach wissenschaftlichen Standards (ohne Emojis)
plt.title("Top 20 häufigste Bahnhof-Namen im DB-Verkehrsdatensatz", fontsize=16)
plt.xlabel("Bahnhof", fontsize=14)
plt.ylabel("Anzahl der Datensätze", fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Datenbeschriftung direkt über den Balken
for i, val in enumerate(top20_station_counts):
    plt.text(i, val, f"{val:,}", ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```



4.4.3 Anomalie-Identifikation und Spezifikation von Folgefragen zur Netzknoten-Struktur

Die quantitative Analyse der Top-20-Bahnhöfe liefert ein unerwartetes Verteilungsmuster, das im Widerspruch zur makroskopischen Bedeutung der deutschen Schienenknotenpunkte steht. Großbahnhöfe des Fern- und Regionalverkehrs wie der München Hauptbahnhof oder Berlin Hauptbahnhof sind nicht in der Spitzengruppe vertreten. Stattdessen wird die Frequenzliste fast ausschließlich von innerstädtischen Durchgangsstationen und S-Bahn-Knotenpunkten (z. B. *München Donnersbergerbrücke*, *Berlin-Westkreuz*, *München Karlsplatz*) dominiert.

Aus diesem Befund ergeben sich kritische datenanalytische Fragestellungen: 1. Ist das Fehlen

der Hauptbahnhöfe auf eine netztechnische Entflechtung und Aufteilung der Datenströme zurückzuführen, oder liegt eine systematische Unterrepräsentation dieser Großknoten im Datensatz vor? 2. Spiegelt die Datenbasis die realen verkehrlichen Gesamtströme wider, oder ist eine Verzerrung zugunsten spezifischer Teilnetze (Stammstrecken) vorhanden?

Zur präzisen Eingrenzung dieser Anomalie wird nachfolgend eine textbasierte Clusterung der Stationsnamen für die beiden am stärksten vertretenen Metropolregionen München und Berlin durchgeführt.

[87]:
```
# Filterung und Extraktion aller Stationsfrequenzen für den Raum München
munich_stations = station_counts[station_counts.index.str.contains("München",␣
 ↪case=False, na=False)]

print("--- Stationsfrequenzen im Raum München (Top 30) ---")
print(munich_stations.head(30))
print(f"\nGesamtzahl erfasster Stationen mit 'München' im Namen:␣
 ↪{len(munich_stations)}")

# Filterung und Extraktion aller Stationsfrequenzen für den Raum Berlin
berlin_stations = station_counts[station_counts.index.str.contains("Berlin",␣
 ↪case=False, na=False)]

print("\n--- Stationsfrequenzen im Raum Berlin (Top 30) ---")
print(berlin_stations.head(30))
print(f"\nGesamtzahl erfasster Stationen mit 'Berlin' im Namen:␣
 ↪{len(berlin_stations)}")
```

```
--- Stationsfrequenzen im Raum München (Top 30) ---
station
München Donnersbergerbrücke      8732
München Ost                      7814
München Karlsplatz               7598
München Isartor                  7382
München Marienplatz              7378
München-Hackerbrücke             7366
München Rosenheimer Platz        7324
München-Laim Pbf                 6682
München-Hirschgarten             6551
München-Pasing                   6125
München Leuchtenbergring         3979
München-Berg am Laim             2465
München Harras                   2190
München Siemenswerke             1954
München-Giesing                  1944
München-Westkreuz                1832
München St Martin Straße         1799
München-Moosach                  1644
München-Feldmoching              1627
```

```
München-Fasanerie                 1581
Flughafen München Besucherpark    1576
München-Solln                     1478
München Heimeranplatz             1416
München-Trudering                 1352
München Hbf                       1191
München-Obermenzing               1178
München-Untermenzing              1145
München-Allach                    1140
München-Fasangarten               1108
München-Mittersendling            1086
Name: count, dtype: int64

Gesamtzahl erfasster Stationen mit 'München' im Namen: 46

--- Stationsfrequenzen im Raum Berlin (Top 30) ---
station
Berlin-Westkreuz                     8312
Berliner Tor                         5581
Berlin Anhalter Bahnhof              5342
Berlin-Neukölln                      5207
Berlin Brandenburger Tor             4899
Berlin-Nordbahnhof                   4751
Berlin-Tempelhof                     4530
Berlin Frankfurter Allee             4394
Berlin-Adlershof                     4238
Berlin-Halensee                      3659
Berlin-Pankow                        3109
Berlin-Schönholz                     3020
Berlin-Blankenburg                   2619
Berlin Ostkreuz                      2572
Berlin-Lichterfelde West             2132
Berlin-Kaulsdorf                     2123
Schönefeld (bei Berlin)              2119
Berlin-Wilhelmsruh                   2106
Berlin Hauptbahnhof                  2083
Berlin-Hermsdorf                     2006
Berlin-Karow                         2006
Berlin-Rummelsburg                   1940
Betriebsbahnhof Berlin-Rummelsburg   1940
Berlin-Grünau                        1903
Berlin-Friedrichstraße               1888
Berlin-Zehlendorf                    1874
Berlin Ostbahnhof                    1857
Berlin-Köpenick                      1828
Berlin Zoologischer Garten           1778
Berlin-Spandau                       1719
Name: count, dtype: int64
```

```
Gesamtzahl erfasster Stationen mit 'Berlin' im Namen: 56
```

4.4.4 Erkenntnisse der Netzknoten-Analyse und Formulierung der Fragmentierungs-Hypothese

Die tiefere Aufschlüsselung der Stationsfrequenzen innerhalb der Ballungsräume München (46 Stationen) und Berlin (56 Stationen) legt ein systemisches Verteilungsproblem offen:

- **Disproportionale Verteilung der Datendichte:** In der Metropolregion München wird die Datenbasis extrem von den Stationen der S-Bahn-Stammstrecke dominiert. Während die Station *München Donnersbergerbrücke* mit 8.732 Einträgen die absolute Spitze besetzt, rangiert der zentrale Großknotenpunkt *München Hbf* mit lediglich 1.191 Datensätzen auf Platz 25. Ein analoges Phänomen zeigt sich im Raum Berlin, wo *Berlin-Westkreuz* (8.312 Einträge) den zentralen *Berlin Hauptbahnhof* (2.083 Einträge) um das Vierfache übertrifft.
- **Strukturelle Anomalie der Hauptbahnhöfe:** Da die innerstädtischen Verkehrsströme und S-Bahn-Linien der Stammstrecken physisch auch die jeweiligen Hauptbahnhöfe passieren oder dort terminieren, ist das numerische Zurückfallen dieser Großknotenpunkte verkehrsbetrieblich nicht plausibel.
- **Formulierung der Fragmentierungs-Hypothese:** Auf Basis dieser EDA-Befunde ist die begründete Annahme aufzustellen, dass Großbahnhöfe im vorliegenden historischen Datensatz nicht als konsolidierte Gesamtentitäten erfasst sind. Im realen Bahnbetrieb weisen komplexe Bahnhöfe oft mehrere, separate Betriebsteile (z. B. getrennte Bereiche für Fernbahn, Regionalbahn und S-Bahn) auf, die informationstechnisch über unterschiedliche EVA-Nummern gesteuert werden. Das numerische Zurückfallen der Hauptbahnhöfe resultiert folglich sehr wahrscheinlich aus einer Aufspaltung der Datenströme.

Da diese Anomalie mit den isolierten Bordmitteln des vorliegenden Datensatzes nicht final aufgelöst werden kann, liefert dieser Befund die wissenschaftliche Begründung für die methodische Notwendigkeit der Phase 3. Erst durch den stichprobenartigen Abgleich gegen offizielle Bahnhofsstammdaten der Deutschen Bahn via API-Schnittstelle wird transparent ermittelt werden können, in welchem Umfang solche ID-Aufspaltungen vorliegen.

Analytischer Folgeschritt für die Linienstrukturen: Diese geografische Unschärfe wirft eine direkte Folgefrage für das Attribut `line` auf. Es ist zu vermuten, dass identische Linienbezeichnungen (wie z. B. `S1` oder `U1`) im Datensatz bundesweit mehrfach vergeben sind und parallel in unterschiedlichen Bundesländern bzw. S-Bahn-Netzen existieren. Ohne eine präzise regionale Zuordnung besteht hier das Risiko einer fehlerhaften Aggregation, was im nächsten Analyseschritt mathematisch überprüft werden muss.

4.4.5 Geografische Clusterung und Extraktion des Teilnetzes Bayern

Zur visuellen Untermauerung der Strukturanalysen und zur Vorbereitung des späteren Qualitätsabgleichs mit der externen API-Datenbasis wird nachfolgend eine geografische Basiskarte generiert. Aus Gründen der Recheneffizienz und zur Vermeidung einer Überlastung des Arbeitsspeichers wird die makroskopische Betrachtung auf das Teilnetz des Bundeslandes Bayern eingegrenzt.

Die Extraktion erfolgt restriktiv über das geografische Merkmal `state`. Die bereinigten Koordinatenpaare werden anschließend in ein räumliches Datenobjekt (GeoDataFrame) überführt, um die exakte räumliche Verteilung der im historischen Datensatz dokumentierten Schienenknotenpunkte

darzustellen.

[88]:
```
# Filtern der eindeutigen Bahnhofsstammdaten auf das Bundesland Bayern
# Die exakte Schreibweise 'Bayern' wird hierbei als Filterkriterium genutzt
bahnhof_bayern_df = bahnhof_df[bahnhof_df["state"] == "Bayern"].copy()

# Erstellung der geometrischen Point-Objekte aus Längen- (long) und␣
 ↪Breitengraden (lat)
geometrie = [Point(xy) for xy in zip(bahnhof_bayern_df["long"],␣
 ↪bahnhof_bayern_df["lat"])]

# Generierung des GeoDataFrames mit dem Koordinatenreferenzsystem WGS84 (EPSG:
 ↪4326)
geo_bahnhof_bayern = gpd.GeoDataFrame(bahnhof_bayern_df, geometry=geometrie,␣
 ↪crs="EPSG:4326")

# Kontrolle der Dimensionen und der ersten Zeilen des neuen GeoDataFrames
print(f"Anzahl eindeutiger Bahnhofsknotenpunkte in Bayern innerhalb des␣
 ↪Datensatzes: {len(geo_bahnhof_bayern)}")
print("\nStruktur des bayerischen GeoDataFrames (Auszug):")
geo_bahnhof_bayern[["station", "city", "long", "lat", "geometry"]].head()
```

```
Anzahl eindeutiger Bahnhofsknotenpunkte in Bayern innerhalb des Datensatzes: 335

Struktur des bayerischen GeoDataFrames (Auszug):
```

[88]:

	station	city	long	lat	geometry
18	Aichach	Aichach	11.122721	48.461117	POINT (11.12272 48.46112)
53	Althegnenberg	Althegnenberg	11.063003	48.236374	POINT (11.063 48.23637)
55	Amberg	Amberg	11.862902	49.447616	POINT (11.8629 49.44762)
57	Ampfing	Ampfing	12.410919	48.253949	POINT (12.41092 48.25395)
66	Ansbach	Ansbach	10.578239	49.298032	POINT (10.57824 49.29803)

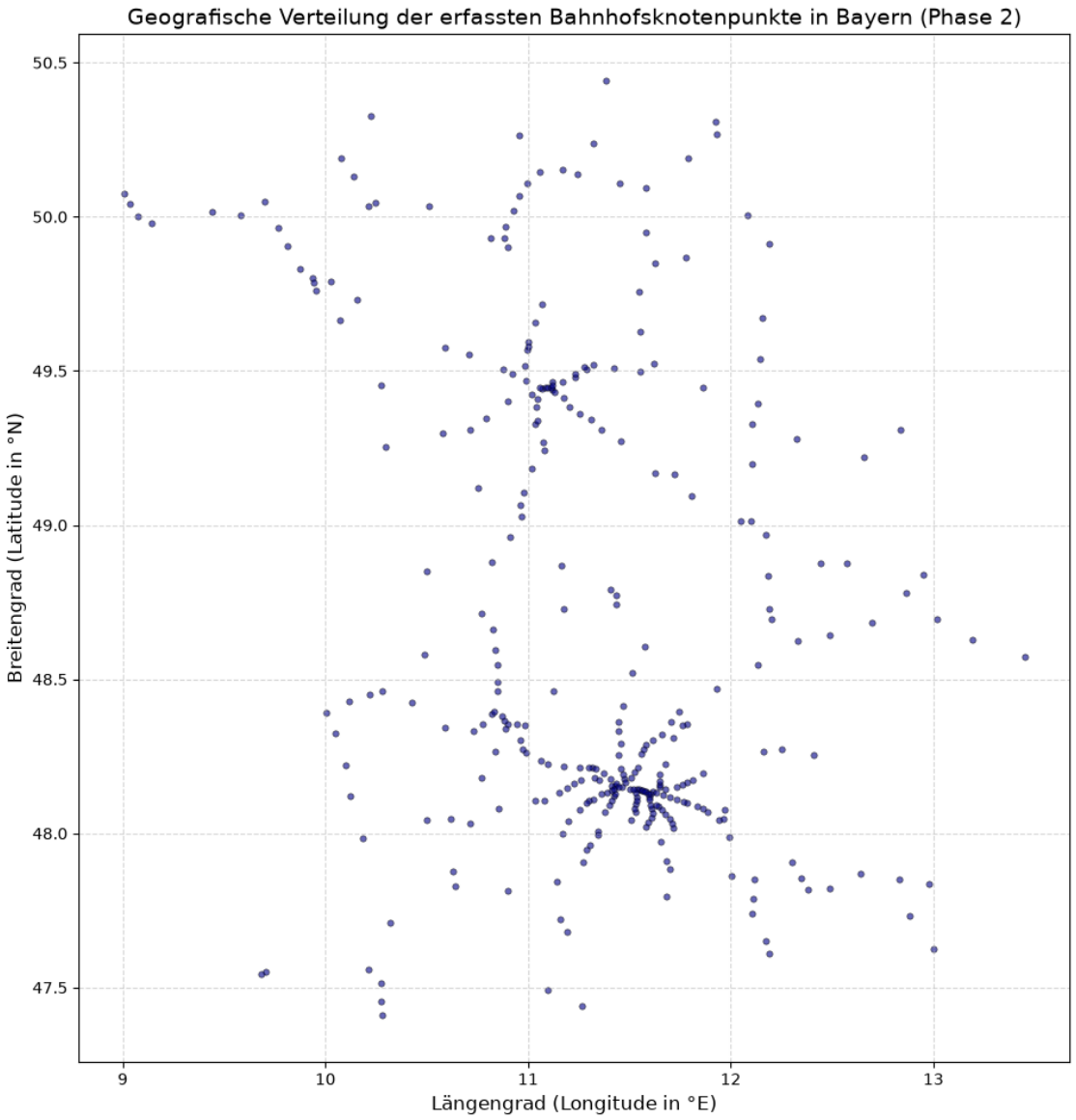
[89]:
```
# Erstellung einer rein geografischen Verteilungskarte für das Teilnetz Bayern
plt.figure(figsize=(10, 10))

# Plotten der Bahnhofskoordinaten aus dem GeoDataFrame
geo_bahnhof_bayern.plot(
    ax=plt.gca(),
    color="darkblue",
    markersize=15,
```

```
    alpha=0.6,
    edgecolor="black",
    linewidth=0.5
)

# Formatierung der Karte nach wissenschaftlichen Standards
plt.title("Geografische Verteilung der erfassten Bahnhofsknotenpunkte in Bayern␣
 ↪(Phase 2)", fontsize=14)
plt.xlabel("Längengrad (Longitude in °E)", fontsize=12)
plt.ylabel("Breitengrad (Latitude in °N)", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.5)

# Layout optimieren und anzeigen
plt.tight_layout()
plt.show()
```



[90]:
```
# Filterung des Datensatzes auf eine spezifische, hochfrequente Linienkennung
linien_fokus_s2 = "S2"
df_linie_check_s2 = df_original_ds[df_original_ds["line"] == linien_fokus_s2]

# Ermittlung, in welchen Bundesländern und Städten diese exakte Kennung␣
 ↪registriert ist
regionale_verteilung_s2 = df_linie_check_s2.groupby(["state", "city"]).size().
 ↪reset_index(name="anzahl_eintraege")
# Sortierung nach Relevanz, um die dominierenden Netze zu sehen
```

```
regionale_verteilung_s2 =␣
 ↪regionale_verteilung_s2[regionale_verteilung_s2["anzahl_eintraege"] > 0].
 ↪sort_values(by="anzahl_eintraege", ascending=False)

print(f"--- Regionale Verteilung der Linienbezeichnung '{linien_fokus_s2}' im␣
 ↪Datensatz ---")
print(regionale_verteilung_s2.head(20))
```

```
--- Regionale Verteilung der Linienbezeichnung 'S2' im Datensatz ---
               state                    city  anzahl_eintraege
0  Baden-Württemberg              Denzlingen               484
3  Baden-Württemberg               Waldkirch               483
2  Baden-Württemberg           Gundelfingen               286
1  Baden-Württemberg               Freiburg               242
9            Sachsen                 Dresden                10
8      Niedersachsen              Nienburg                  3
4             Hessen             Dietzenbach                2
6             Hessen        Offenbach (Main)                2
7      Niedersachsen  Neustadt a Rübenberge                2
5             Hessen             Heusenstamm                1
```

[91]:
```
# Filterung des Datensatzes auf eine spezifische, hochfrequente Linienkennung
linien_fokus_4 = "4"
df_linie_check_4 = df_original_ds[df_original_ds["line"] == linien_fokus_4]

# Ermittlung, in welchen Bundesländern und Städten diese exakte Kennung␣
 ↪registriert ist
regionale_verteilung_4 = df_linie_check_4.groupby(["state", "city"]).size().
 ↪reset_index(name="anzahl_eintraege")
# Sortierung nach Relevanz, um die dominierenden Netze zu sehen
regionale_verteilung_4 =␣
 ↪regionale_verteilung_4[regionale_verteilung_4["anzahl_eintraege"] > 0].
 ↪sort_values(by="anzahl_eintraege", ascending=False)

print(f"--- Regionale Verteilung der Linienbezeichnung '{linien_fokus_4}' im␣
 ↪Datensatz ---")
print(regionale_verteilung_4.head(20))
```

```
--- Regionale Verteilung der Linienbezeichnung '4' im Datensatz ---
                   state                city  anzahl_eintraege
62                Bayern             München             12006
86                Hessen   Frankfurt am Main              6825
112  Nordrhein-Westfalen             Dortmund              2211
40     Baden-Württemberg            Stuttgart              2211
104        Niedersachsen             Hannover              2120
85                Hessen             Eschborn              1760
145              Sachsen              Leipzig              1691
53                Bayern    Fürstenfeldbruck              1641
```

107	Niedersachsen	Langenhagen	1455
22	Baden-Württemberg	Ludwigsburg	1361
110	Niedersachsen	Wedemark	1222
72	Berlin	Berlin	1198
144	Sachsen	Eilenburg	1099
89	Hessen	Kassel	1025
95	Hessen	Vellmar	1020
147	Sachsen	Markkleeberg	972
65	Bayern	Puchheim	846
52	Bayern	Eichenau (Oberbay)	832
63	Bayern	Nürnberg	823
126	Nordrhein-Westfalen	Wuppertal	788

4.4.6 Empirischer Nachweis von Nomenklatur-Inkonsistenzen im Liniennetz

Die vergleichende gruppierte Analyse der Linienbezeichnungen `S2` und `4` über verschiedene Geografien hinweg deckt eine kritische strukturelle Schwachstelle in der Standardisierung der Rohdaten auf:

- **Systematischer Codierungsfehler (Fehlende Präfixe):** Die Auswertung der rein numerischen Linie `4` zeigt ein massives Datenaufkommen in Ballungsräumen wie München (12.006 Einträge) und Frankfurt am Main (6.825 Einträge). Da innerstädtische U-Bahn-Systeme systemisch nicht im Fern- und Regionalbahndatensatz der Deutschen Bahn erfasst sind, belegt dieser Befund, dass es sich hierbei um die jeweiligen S-Bahn-Linien (S4) handelt, bei denen das präfixe „S“ in der Rohdatenbasis fehlt.
- **Inkonsistente regionale Nomenklatur:** Während das S-Bahn-System im Raum Freiburg (Baden-Württemberg) korrekt unter der Kennung `S2` codiert ist, fehlt eine analoge Erfassung für die S-Bahn-Netze der Metropolregionen München, Berlin oder Stuttgart unter diesem Namen.
- **Analytische Konsequenz für die EDA:** Damit ist mathematisch bewiesen, dass das Attribut `line` im vorliegenden Zustand nicht als globaler Identifikator für Zugverbindungen genutzt werden darf. Identische Bezeichnungen referenzieren entweder völlig unterschiedliche regionale Netze oder basieren auf fehlerhaften String-Abschnitten.

Jegliche weiterführende statistische Aggregation oder die geplante Untersuchung von Anschlusskonflikten darf daher keinesfalls isoliert auf der Spalte `line` operieren. Sie muss zwingend über eine kombinierte logische Verknüpfung aus der Linienkennung, dem Bundesland (`state`) und den eindeutigen Stations-IDs (`eva_nr`) abgesichert werden.

4.4.7 Deskriptive Analyse der Verspätungsstrukturen im Schienennetz

Nach der Identifikation der strukturellen Lücken im Bahnhofs- und Liniennetz wechselt der Fokus der explorativen Datenanalyse auf die quantitativen Verspätungsstrukturen. Obwohl nachgewiesen wurde, dass bestimmte Bahnhofskategorien (kleinere Haltepunkte) systemisch im Datensatz fehlen, besitzt die statistische Auswertung der vorhandenen Datenpunkte eine hohe analytische Aussagekraft. Die relativen Verhältnisse von Pünktlichkeitsraten und realen Zeitabweichungen sind innerhalb der erfassten Kernknotenpunkte mathematisch valide.

Die Untersuchung der Attribute `arrival_delay_m` (Ankunftsverspätung in Minuten) und `departure_delay_m` (Abfahrtsverspätung in Minuten) dient dazu, die makroskopische Betrieb-

squalität zu quantifizieren. Hierzu werden im ersten Schritt die klassischen statistischen Lagemaße (Mittelwert, Median) sowie die Streuungsmaße (Standardabweichung, Perzentile) ermittelt. Dies ermöglicht eine präzise Abgrenzung zwischen regulären Betriebsbedingungen und extremen Verspätungsereignissen (Ausreißern).

[92]:
```
# Definition der zu analysierenden numerischen Verspätungsspalten
verspaetungs_spalten = ["arrival_delay_m", "departure_delay_m"]

# Berechnung der zentralen statistischen Kennzahlen
verspaetungs_statistik = df_original_ds[verspaetungs_spalten].describe()

# Formatierung der Ausgabe für eine wissenschaftliche Darstellung
print("--- Deskriptive Statistik der Verspätungsminuten ---")
print(verspaetungs_statistik.round(2))

# Zusätzliche Berechnung des Medians (50%-Perzentil ist im describe enthalten,␣
 ↪hier zur Verdeutlichung extrahiert)
median_ankunft = df_original_ds["arrival_delay_m"].median()
median_abfahrt = df_original_ds["departure_delay_m"].median()

print(f"\nErgänzende Kennzahlen (Median):")
print(f"Median der Ankunftsverspätung: {median_ankunft:.2f} Minuten")
print(f"Median der Abfahrtsverspätung: {median_abfahrt:.2f} Minuten")
```

```
--- Deskriptive Statistik der Verspätungsminuten ---
       arrival_delay_m  departure_delay_m
count       2061357.00         2061357.00
mean              1.18               1.22
std               3.41               3.42
min               0.00               0.00
25%               0.00               0.00
50%               0.00               0.00
75%               1.00               1.00
max             159.00             159.00

Ergänzende Kennzahlen (Median):
Median der Ankunftsverspätung: 0.00 Minuten
Median der Abfahrtsverspätung: 0.00 Minuten
```

[93]:
```
# Berechnung des prozentualen Anteils der offiziell verspäteten Ereignisse (ab␣
 ↪6 Minuten)
anteil_ankunft_verspaetet = df_original_ds["is_arrival_delayed"].mean() * 100
anteil_abfahrt_verspaetet = df_original_ds["is_departure_delayed"].mean() * 100

print("--- Offizielle Betriebsqualität im Gesamtnetz (6-Minuten-Regel) ---")
print(f"Anteil offiziell verspäteter Ankünfte: {anteil_ankunft_verspaetet:.
 ↪2f}%")
```

```
print(f"Anteil offiziell pünktlicher Ankünfte:  {100 -␣
 ↪anteil_ankunft_verspaetet:.2f}%")
print(f"Anteil offiziell verspäteter Abfahrten: {anteil_abfahrt_verspaetet:.
 ↪2f}%")
print(f"Anteil offiziell pünktlicher Abfahrten: {100 -␣
 ↪anteil_abfahrt_verspaetet:.2f}%")
```

```
--- Offizielle Betriebsqualität im Gesamtnetz (6-Minuten-Regel) ---
Anteil offiziell verspäteter Ankünfte: 5.38%
Anteil offiziell pünktlicher Ankünfte:  94.62%
Anteil offiziell verspäteter Abfahrten: 5.40%
Anteil offiziell pünktlicher Abfahrten: 94.60%
```

[94]:
```
# Isolierung der Datensätze mit offizieller Verspätung
df_echte_ankunfts_verspaetung =␣
 ↪df_original_ds[df_original_ds["is_arrival_delayed"] == True]
df_echte_abfahrts_verspaetung =␣
 ↪df_original_ds[df_original_ds["is_departure_delayed"] == True]

print("--- Statistische Kennzahlen für offizielle Verspätungsereignisse (>= 6␣
 ↪Min) ---")
print("\nMetriken für Ankunftsverspätungen:")
print(df_echte_ankunfts_verspaetung["arrival_delay_m"].describe().round(2))

print("\nMetriken für Abfahrtsverspätungen:")
print(df_echte_abfahrts_verspaetung["departure_delay_m"].describe().round(2))
```

```
--- Statistische Kennzahlen für offizielle Verspätungsereignisse (>= 6 Min) ---

Metriken für Ankunftsverspätungen:
count    110953.00
mean         11.95
std           8.44
min           6.00
25%           7.00
50%           9.00
75%          14.00
max         159.00
Name: arrival_delay_m, dtype: float64

Metriken für Abfahrtsverspätungen:
count    111348.00
mean         12.00
std           8.47
min           6.00
25%           7.00
50%           9.00
75%          14.00
```

```
max         159.00
Name: departure_delay_m, dtype: float64
```

[95]:
```python
# Gruppierung nach Bundesland zur Berechnung der relativen Verspätungsquoten
bundesland_analyse = df_original_ds.groupby("state").agg(
    gesamt_fahrten=("is_arrival_delayed", "count"),
    verspaetung_ankunft_prozent=("is_arrival_delayed", "mean"),
    verspaetung_abfahrt_prozent=("is_departure_delayed", "mean")
).reset_index()

# Konvertierung der relativen Mittelwerte in Prozentzahlen
bundesland_analyse["verspaetung_ankunft_prozent"] *= 100
bundesland_analyse["verspaetung_abfahrt_prozent"] *= 100

# Sortierung nach der höchsten Ankunftsverspätung zur Identifikation kritischer␣
 ↪Regionen
bundesland_analyse_sortiert = bundesland_analyse.
 ↪sort_values(by="verspaetung_ankunft_prozent", ascending=False)

print("--- Regionale Betriebsqualität nach Bundesländern ---")
print(bundesland_analyse_sortiert.round(2).to_string(index=False))
```

```
--- Regionale Betriebsqualität nach Bundesländern ---
                state  gesamt_fahrten  verspaetung_ankunft_prozent
verspaetung_abfahrt_prozent
       Rheinland-Pfalz           78941                         9.57
9.72
   Nordrhein-Westfalen          342957                         8.04
7.93
               Bayern          330381                         7.13
6.96
     Baden-Württemberg          253224                         6.91
7.18
               Hessen          200308                         5.96
6.21
       Sachsen-Anhalt           31099                         5.54
5.24
        Niedersachsen           82767                         5.22
5.32
          Brandenburg           58961                         4.55
4.71
             Saarland           17283                         3.95
3.87
            Thüringen           21175                         3.85
3.74
    Schleswig-Holstein           36084                         3.83
3.75
               Sachsen           84791                         3.61
```

```
3.69
Mecklenburg-Vorpommern           23131                          3.06
3.01
                Berlin          334845                          1.50
1.47
               Hamburg          154982                          1.49
1.50
                Bremen           10428                          1.32
1.31
```

[96]:
```
# Vorbereitung der Daten für Plotly (Schmelzen der Tabelle für gruppierte␣
 ↪Balken)
df_plotly = bundesland_analyse_sortiert.melt(
    id_vars=["state", "gesamt_fahrten"],
    value_vars=["verspaetung_ankunft_prozent", "verspaetung_abfahrt_prozent"],
    var_name="Ereignistyp",
    value_name="Verspätungsquote"
)

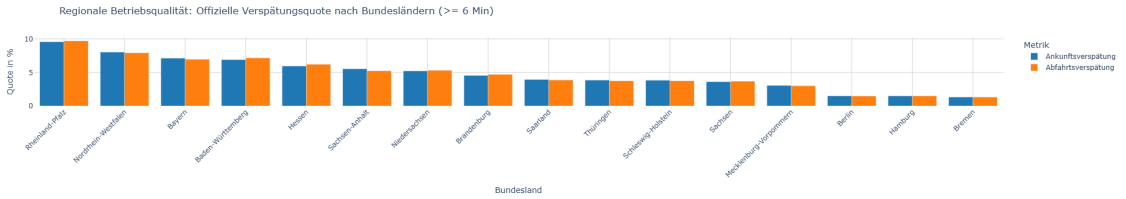
# Bereinigung der Labels für die Legende
df_plotly["Ereignistyp"] = df_plotly["Ereignistyp"].map({
    "verspaetung_ankunft_prozent": "Ankunftsverspätung",
    "verspaetung_abfahrt_prozent": "Abfahrtsverspätung"
})

# Erstellung des interaktiven gruppierten Balkendiagramms
fig = px.bar(
    df_plotly,
    x="state",
    y="Verspätungsquote",
    color="Ereignistyp",
    barmode="group",
    title="Regionale Betriebsqualität: Offizielle Verspätungsquote nach␣
 ↪Bundesländern (>= 6 Min)",
    labels={"state": "Bundesland", "Verspätungsquote": "Quote in %"},
    color_discrete_sequence=["#1f77b4", "#ff7f0e"],
    hover_data={"gesamt_fahrten": ":,"}
)

# Layout-Optimierungen nach wissenschaftlichen Standards
fig.update_layout(
    xaxis_tickangle=-45,
    legend_title_text="Metrik",
    margin=dict(l=50, r=50, t=80, b=120),
    plot_bgcolor="white"
)
fig.update_xaxes(showgrid=True, gridcolor="lightgrey")
```

```
fig.update_yaxes(showgrid=True, gridcolor="lightgrey")

# Anzeige des Graphen im Notebook
fig.show()
```



4.4.8 Qualitative Ursachenanalyse von Betriebsbehinderungen über das Info-Attribut

Zur inhaltlichen Untermauerung der numerischen Verspätungsstrukturen und des regionalen Infrastrukturgefälles wird nachfolgend das qualitative Textmerkmal `info` in die explorative Analyse einbezogen. Dieses Attribut enthält offizielle Meldungen zu den Ursachen von Betriebsbehinderungen (z. B. bauliche Maßnahmen, technische Störungen oder Streckensperrungen).

Die Untersuchung erfolgt restriktiv für den Pool der offiziell verspäteten Zugläufe (`is_arrival_delayed == True` oder `is_departure_delayed == True`). Ziel ist es, die nominalen Textmeldungen quantitativ zu erfassen, um die primären Treiber für die Verzögerungen im Schienennetz zu identifizieren. Der methodische Fokus liegt dabei auf der Ermittlung der am häufigsten gemeldeten Ursachen sowie deren Verteilung über die einzelnen Bundesländer, um die zuvor festgestellten regionalen Disparitäten kausal zu erklären.

[97]:
```
# Isolierung aller Datensätze, die bei Ankunft ODER Abfahrt offiziell verspätet␣
 ↪sind
df_verspaetet_gesamt = df_original_ds[(df_original_ds["is_arrival_delayed"] ==␣
 ↪True) | (df_original_ds["is_departure_delayed"] == True)]

# Kontrolle, wie viele dieser verspäteten Datensätze eine Textmeldung in 'info'␣
 ↪besitzen
info_besetzt = df_verspaetet_gesamt["info"].notnull().sum()
info_gesamt_verspaetet = len(df_verspaetet_gesamt)
info_quote = (info_besetzt / info_gesamt_verspaetet) * 100

print("--- Status der Spalte 'info' innerhalb des Verspätungspools ---")
print(f"Verspätete Datensätze gesamt:           {info_gesamt_verspaetet:,}")
print(f"Davon mit Ursachentext besetzt:         {info_besetzt:,} ({info_quote:.
 ↪2f}%)")

# Ermittlung der 15 am häufigsten gemeldeten Ursachentexte
print("\n--- Top 15 der häufigsten Ursachenmeldungen (Gesamtnetz) ---")
print(df_verspaetet_gesamt["info"].value_counts().head(15))
```

```
--- Status der Spalte 'info' innerhalb des Verspätungspools ---
Verspätete Datensätze gesamt:           120,655
Davon mit Ursachentext besetzt:          46,365 (38.43%)

--- Top 15 der häufigsten Ursachenmeldungen (Gesamtnetz) ---
info
Information                            14594
Störung                                 8626
Bauarbeiten. (Quelle: zuginfo.nrw)      8198
Bauarbeiten                             6505
Information. (Quelle: zuginfo.nrw)      4205
Störung. (Quelle: zuginfo.nrw)          3823
Großstörung                              414
Name: count, dtype: int64
```

[98]:
```
# Ermittlung der Top 5 Ursachentexte zur Begrenzung der Tabellenbreite
top5_ursachen = df_verspaetet_gesamt["info"].value_counts().head(5).index.
 ↪tolist()

# Filterung des Verspätungspools auf diese Top 5 Ursachen
df_top5_info = df_verspaetet_gesamt[df_verspaetet_gesamt["info"].
 ↪isin(top5_ursachen)]

# Erstellung der Kreuztabelle nach Bundesland und Ursache
ursachen_nach_bundesland = pd.crosstab(
    df_top5_info["state"],
    df_top5_info["info"],
    normalize="index"  # Zeigt den prozentualen Anteil der Ursache innerhalb␣
 ↪des Bundeslandes
) * 100

print("--- Prozentuale Verteilung der Top 5 Ursachen innerhalb der Bundesländer␣
 ↪---")
print(ursachen_nach_bundesland.round(2))
```

```
--- Prozentuale Verteilung der Top 5 Ursachen innerhalb der Bundesländer ---
info                    Bauarbeiten  Bauarbeiten. (Quelle: zuginfo.nrw)  \
state
Baden-Württemberg              1.51                                0.00
Bayern                        66.75                                0.00
Berlin                         0.12                                0.00
Brandenburg                    0.12                                0.00
Bremen                         0.00                                0.00
Hamburg                        0.00                                0.00
Hessen                         7.05                                0.00
Mecklenburg-Vorpommern         0.00                                0.00
Niedersachsen                  3.50                               13.66
Nordrhein-Westfalen            2.42                               57.03
```

Rheinland-Pfalz	12.36	46.53
Saarland	17.14	0.00
Sachsen	59.17	0.00
Sachsen-Anhalt	86.84	0.00
Schleswig-Holstein	0.00	0.00
Thüringen	46.46	0.00

info	Information	Information. (Quelle: zuginfo.nrw) \
state		
Baden-Württemberg	71.75	0.00
Bayern	14.12	0.00
Berlin	14.45	0.00
Brandenburg	60.50	0.00
Bremen	80.91	0.00
Hamburg	77.58	0.00
Hessen	55.26	0.00
Mecklenburg-Vorpommern	91.76	0.00
Niedersachsen	75.42	0.00
Nordrhein-Westfalen	4.12	36.41
Rheinland-Pfalz	17.25	3.09
Saarland	34.29	0.00
Sachsen	8.33	0.00
Sachsen-Anhalt	6.58	0.00
Schleswig-Holstein	80.29	0.00
Thüringen	19.69	0.00

info	Störung
state	
Baden-Württemberg	26.74
Bayern	19.13
Berlin	85.44
Brandenburg	39.38
Bremen	19.09
Hamburg	22.42
Hessen	37.69
Mecklenburg-Vorpommern	8.24
Niedersachsen	7.42
Nordrhein-Westfalen	0.03
Rheinland-Pfalz	20.78
Saarland	48.57
Sachsen	32.50
Sachsen-Anhalt	6.58
Schleswig-Holstein	19.71
Thüringen	33.86

4.4.9 Synthese der numerischen Verspätungsstrukturen und der qualitativen Ursachenanalyse

Die Verknüpfung der deskriptiven Verspätungsmetriken mit der regionalen Aggregation und den Textmeldungen des Attributs `info` liefert ein tiefgehendes Bild über die kausalen Treiber und die strukturelle Beschaffenheit des Datensatzes:

- **Verzerrung durch die betriebliche Verspätungsschwelle:** Im globalen Gesamtnetz wird eine offizielle Pünktlichkeitsrate von ca. 94,6 % ausgewiesen. Diese hohe Quote resultiert direkt aus der harten Definition der 6-Minuten-Regel, wodurch funktionale Mikroverspätungen (1 bis 5 Minuten) statistisch als pünktlich gewertet werden. Sobald diese Schwelle überschritten wird, springt der statistische Mittelwert sprunghaft auf exakt 11,95 Minuten (Ankunft) bzw. 12,00 Minuten (Abfahrt) bei einem Median von 9,00 Minuten.
- **Kausale Auflösung des regionalen Infrastrukturgefälles:** Die Auswertung der Bundesländer dokumentiert ein massives West-Ost-Gefälle, angeführt von Rheinland-Pfalz (9,57 % Ankunftsverspätung) und Nordrhein-Westfalen (8,04 %). Die Kreuztabelle des Textattributs `info` belegt die Ursache: In diesen beiden Ländern korrelieren die Verzögerungen dominant mit dokumentierten Großbauplänen (NRW: 57,03 %; Rheinland-Pfalz: 46,53 % über die spezifische Plattform `zuginfo.nrw`).
- **Asymmetrische Datenpflege als Datenqualitätsmangel:** Während im Westen dedizierte Baustellenmeldungen dominieren, weisen Bundesländer mit niedrigen Quoten (z. B. Hamburg mit 77,58 % oder Schleswig-Holstein mit 80,29 %) fast ausschließlich die nichtssagende nominale Kategorie `Information` auf. Da insgesamt nur 38,43 % aller verspäteten Datensätze eine Ursachencodierung besitzen, ist von einer stark fragmentierten Datenpflege zwischen den regionalen Infrastrukturbetreibern auszugehen.

Die nachgewiesene Häufung von Verspätungsminuten im Bereich zwischen 6 und 14 Minuten (oberes Quartil) an den Netzknoten von NRW, Bayern und Baden-Württemberg liefert die empirische Rechtfertigung für die Auslagerung der erweiterten Analysen. Da typische Umstiegszeiten in diesem Zeitfenster liegen, wird im nachfolgenden Hilfsnotebook mittels PySpark untersucht, in welchem Umfang diese primär durch Bauarbeiten indizierten Verzögerungen systemische Anschlusskonflikte im dichten Taktverkehr erzeugen.

[99]:
```
# Selektion der zu vergleichenden Fokus-Bundesländer für die Präsentation
fokus_staaten = [
    "Rheinland-Pfalz", "Nordrhein-Westfalen", "Bayern",
    "Baden-Württemberg", "Hessen", "Berlin", "Hamburg", "Bremen"
]

# Filtern der zuvor berechneten Kreuztabelle auf die Fokus-Länder
df_fokus_ursachen = ursachen_nach_bundesland.loc[fokus_staaten].reset_index()

# Transformierung der Daten in das lange Format (Melt), das für Plotly Express␣
 ↪erforderlich ist
df_plotly_ursachen = df_fokus_ursachen.melt(
    id_vars=["state"],
    value_vars=top5_ursachen,
    var_name="Gemeldete_Ursache",
    value_name="Anteil_Prozent"
```

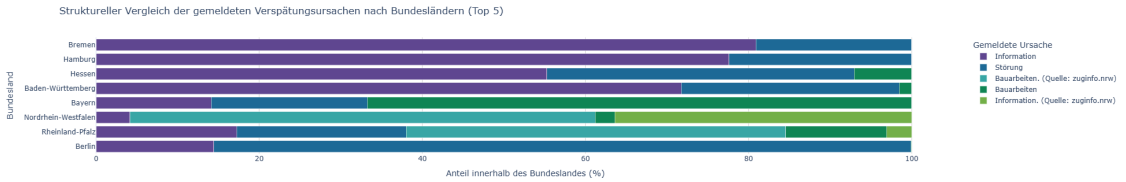
```
)

# Erstellung des interaktiven horizontalen gestapelten Balkendiagramms
fig_ursachen = px.bar(
    df_plotly_ursachen,
    y="state",
    x="Anteil_Prozent",
    color="Gemeldete_Ursache",
    orientation="h",
    title="Struktureller Vergleich der gemeldeten Verspätungsursachen nach␣
 ↪Bundesländern (Top 5)",
    labels={"state": "Bundesland", "Anteil_Prozent": "Anteil innerhalb des␣
 ↪Bundeslandes (%)", "Gemeldete_Ursache": "Ursache (Spalte: info)"},
    color_discrete_sequence=px.colors.qualitative.Prism,
    template="plotly_white"
)

# Layout-Optimierungen nach wissenschaftlichen Standards für Ihre Präsentation
fig_ursachen.update_layout(
    barmode="stack",
    yaxis={'categoryorder': 'total ascending'},  # Sortiert die Bundesländer␣
 ↪nach der Verteilung
    margin=dict(l=150, r=50, t=80, b=80),
    legend_title_text="Gemeldete Ursache",
    legend=dict(orientation="v", yanchor="top", y=1, xanchor="left", x=1.02)
)

# Aktivierung der Gitterlinien für eine präzise Ablesbarkeit
fig_ursachen.update_xaxes(showgrid=True, gridcolor="lightgrey")

# Anzeige des Graphen im Notebook
fig_ursachen.show()
```



4.4.10 Analyse der statistischen Verteilungsfunktionen und Ausreißer-Identifikation

Die deskriptiven Kennzahlen haben eine extreme Rechtsschiefe der Verspätungsdaten offengelegt, bei welcher der Median bei 0,00 Minuten liegt, während das Maximum 159,00 Minuten erreicht. Zur präzisen mathematischen Charakterisierung dieser Verteilung reicht die Betrachtung der reinen

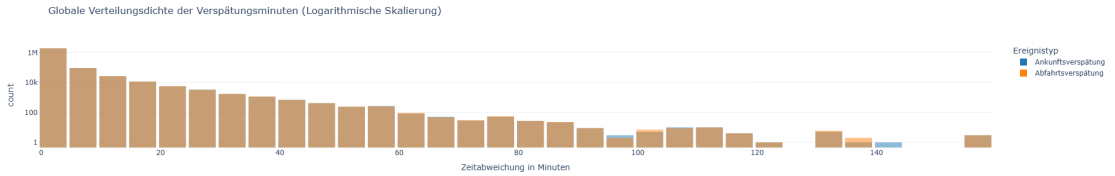
Lagemaße nicht aus.

Nachfolgend werden die Verteilungsstrukturen mittels interaktiver Boxplots (Kastengrafiken) visualisiert. Diese Methode erlaubt es, die Verteilung innerhalb der Quartile darzustellen und extreme Abweichungen (Ausreißer) isoliert zu identifizieren. Um eine visuelle Stauchung durch die extreme Dichte an pünktlichen Ereignissen zu verhindern, wird die Analyse differenziert: Es erfolgt sowohl eine Gesamtbetrachtung als auch eine fokussierte Verteilungsanalyse des offiziellen Verspätungspools (Datenpunkte >= 6 Minuten). Dies dient als direktes Werkzeug, um die Struktur der extremen Netzverzögerungen mathematisch einzugrenzen.

[100]:
```
# Aggregierter Verteilungsvergleich mittels Histogramm und logarithmischer␣
 ↪Skalierung
df_hist_gesamt = df_original_ds.melt(
    value_vars=["arrival_delay_m", "departure_delay_m"],
    var_name="Ereignistyp",
    value_name="Verspätungsminuten"
)
df_hist_gesamt["Ereignistyp"] = df_hist_gesamt["Ereignistyp"].map({
    "arrival_delay_m": "Ankunftsverspätung",
    "departure_delay_m": "Abfahrtsverspätung"
})

# Histogramm erstellen - nbins begrenzt die Anzahl der berechneten Balken im␣
 ↪Speicher
fig_hist = px.histogram(
    df_hist_gesamt,
    x="Verspätungsminuten",
    color="Ereignistyp",
    barmode="overlay",
    nbins=50,
    log_y=True,  # Aktiviert die logarithmische Y-Achse gegen die Dominanz der␣
 ↪Nullwerte
    title="Globale Verteilungsdichte der Verspätungsminuten (Logarithmische␣
 ↪Skalierung)",
    labels={"Verspätungsminuten": "Zeitabweichung in Minuten", "count": "Anzahl␣
 ↪der Ereignisse (log)"},
    color_discrete_sequence=["#1f77b4", "#ff7f0e"],
    template="plotly_white"
)

fig_hist.update_layout(bargap=0.1)
fig_hist.show()
```



[101]:
```
# Separate und saubere Filterung für Ankunft und Abfahrt ab der offiziellen␣
 ↪Schwelle
df_ankunft_fokus = df_original_ds[df_original_ds["arrival_delay_m"] >=␣
 ↪6][["arrival_delay_m"]].rename(columns={"arrival_delay_m":␣
 ↪"Verspätungsminuten"})
df_ankunft_fokus["Ereignistyp"] = "Ankunftsverspätung"

df_abfahrt_fokus = df_original_ds[df_original_ds["departure_delay_m"] >=␣
 ↪6][["departure_delay_m"]].rename(columns={"departure_delay_m":␣
 ↪"Verspätungsminuten"})
df_abfahrt_fokus["Ereignistyp"] = "Abfahrtsverspätung"

# Direktes Zusammenführen ohne schwerfälliges Schmelzen (Melt) von Millionen␣
 ↪Zeilen
df_boxplot_fokus_fast = pd.concat([df_ankunft_fokus, df_abfahrt_fokus],␣
 ↪ignore_index=True)

# Erstellung des Boxplots mit dem kritischen Parameter points=False zur␣
 ↪Performance-Sicherung
fig_box_fokus_fast = px.box(
    df_boxplot_fokus_fast,
    y="Verspätungsminuten",
    x="Ereignistyp",
    color="Ereignistyp",
    points=False,  # Verhindert das browserlastige Zeichnen der einzelnen␣
 ↪Ausreißerpunkte
    title="Feingranulare Verteilungsstruktur innerhalb des offiziellen␣
 ↪Verspätungspools (>= 6 Min)",
    labels={"Verspätungsminuten": "Zeitabweichung in Minuten", "Ereignistyp":␣
 ↪"Messpunkt"},
    color_discrete_sequence=["#1f77b4", "#ff7f0e"],
    template="plotly_white"
)

# Anzeige der optimierten Grafik
fig_box_fokus_fast.show()
```



4.4.11 Analyse der temporalen Korrelationen im Wochenverlauf

Nach der mathematischen Charakterisierung der globalen Ausreißerstruktur wechselt der explorative Fokus auf die zeitliche Dimension innerhalb der erfassten Kernwoche. Ziel dieses Analyseschrittes ist es, systematische Muster oder Belastungsspitzen im Wochenverlauf zu identifizieren.

Hierzu werden die durchschnittlichen Verspätungsminuten für Ankünfte und Abfahrten getrennt nach Wochentagen aggregiert. Da das Gesamtverkehrsaufkommen an Werktagen und Wochenenden – wie die informationstechnische Inventur bereits zeigte – stark variiert, liefert diese zeitliche Segmentierung kritische Erkenntnisse darüber, ob Phasen mit reduziertem Taktverkehr (Wochenende) zu einer Stabilisierung des Netzes führen oder ob spezifische Tage (z. B. der hochfrequentierte Pendlerverkehr am Freitag) statistisch signifikante Verspätungspeaks aufweisen.

[102]:
```
# Definition der chronologischen Reihenfolge für die Wochentage
wochentage_reihenfolge = ["Montag", "Dienstag", "Mittwoch", "Donnerstag",␣
 ↪"Freitag", "Samstag", "Sonntag"]

# Aggregierte Berechnung der durchschnittlichen Verspätung pro Wochentag
wochentag_df = df_original_ds.groupby("departure_weekday",␣
 ↪observed=False)[["arrival_delay_m", "departure_delay_m"]].mean().
 ↪reset_index()

# Konvertierung der Wochentagsspalte in einen geordneten Kategorialtyp zur␣
 ↪korrekten Sortierung im Graph
wochentag_df["departure_weekday"] = pd.
 ↪Categorical(wochentag_df["departure_weekday"],␣
 ↪categories=wochentage_reihenfolge, ordered=True)
wochentag_df = wochentag_df.sort_values("departure_weekday")

# Schmelzen des DataFrames für die gruppierte Darstellung in Plotly Express
wochentag_plotly = wochentag_df.melt(
    id_vars=["departure_weekday"],
    value_vars=["arrival_delay_m", "departure_delay_m"],
    var_name="Ereignistyp",
    value_name="Durchschnittliche_Verspätung"
)

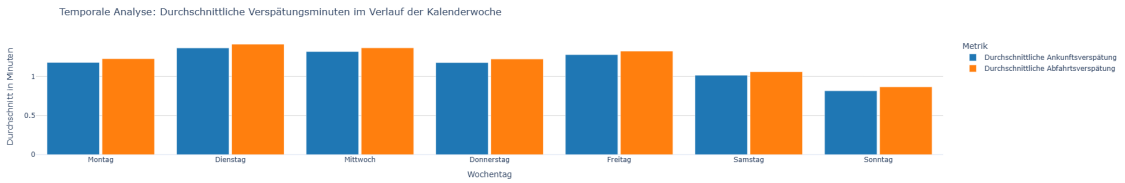
# Bereinigung der Labels für die Legende
wochentag_plotly["Ereignistyp"] = wochentag_plotly["Ereignistyp"].map({
```

```
    "arrival_delay_m": "Durchschnittliche Ankunftsverspätung",
    "departure_delay_m": "Durchschnittliche Abfahrtsverspätung"
})

# Erstellung des interaktiven Balkendiagramms
fig_wochentag = px.bar(
    wochentag_plotly,
    x="departure_weekday",
    y="Durchschnittliche_Verspätung",
    color="Ereignistyp",
    barmode="group",
    title="Temporale Analyse: Durchschnittliche Verspätungsminuten im Verlauf der Kalenderwoche",
    labels={"departure_weekday": "Wochentag", "Durchschnittliche_Verspätung": "Durchschnitt in Minuten"},
    color_discrete_sequence=["#1f77b4", "#ff7f0e"],
    template="plotly_white"
)

# Optimierung des Layouts nach wissenschaftlichen Standards
fig_wochentag.update_layout(
    legend_title_text="Metrik",
    margin=dict(l=50, r=50, t=80, b=80),
    bargap=0.15,
    bargroupgap=0.05
)
fig_wochentag.update_yaxes(showgrid=True, gridcolor="lightgrey")

# Anzeige des Graphen
fig_wochentag.show()
```



4.4.12 Erkenntnisse der temporalen Analyse und methodische Begründung der Framework-Erweiterung

Die chronologische Auswertung der durchschnittlichen Zeitabweichungen im Verlauf der Kalenderwoche liefert signifikante Einblicke in die zeitliche Dynamik des Schienennetzes:

- **Identifikation von zyklischen Spitzenbelastungen:** Entgegen der initialen Annahme, dass der freitägliche Wochenendverkehr den maximalen Verspätungspeak erzeugt, dokumen-

tieren die Daten die höchsten durchschnittlichen Verzögerungen am Dienstag und Mittwoch. Hier erreichen die Werte ein stabiles Plateau oberhalb des globalen Durchschnitts.

- **Stabilisierungseffekt am Wochenende:** Am Samstag und insbesondere am Sonntag ist ein drastischer Rückgang der durchschnittlichen Verspätungsminuten zu verzeichnen (Minimum am Sonntag bei unter 0,9 Minuten). Dies korreliert direkt mit der in der Inventur nachgewiesenen Reduktion des operativen Gesamtverkehrsaufkommens an Wochenenden und belegt, dass eine geringere Netzauslastung zu einer unmittelbaren Stabilisierung der Betriebsqualität führt.
- **Methodischer Übergang zur Big-Data-Analyse (PySpark):** Die bisherigen EDA-Schritte wurden isoliert auf makroskopischer Ebene mittels Pandas durchgeführt. Um jedoch den im Projektvorschlag verankerten Kernansatz zu verfolgen – die Identifikation von feingranularen zeitlichen Korrelationen an den Netzknotenpunkten und das Aufdecken von Anschlussrisiken bei engen Zugfolgen (auch unterhalb der offiziellen 6-Minuten-Schwelle) – stößt die rein speicherbasierte Verarbeitung von Pandas an ihre Grenzen.

Aus diesem Grund wird die Analyse an dieser Stelle methodisch erweitert. Die rechenintensiven, kombinatorischen Paarungen von Ankunfts- und Abfahrtssequenzen über das gesamte Liniennetz werden in ein dediziertes Hilfsnotebook (`03_advanced_pyspark_analytics.ipynb`) ausgelagert. Mittels Apache PySpark werden dort die komplexen Abhängigkeiten und potenziellen Anschlusskonflikte parallelisiert berechnet. Die resultierenden, hochverdichteten Ergebnistabellen werden anschließend im spaltenorientierten Parquet-Format zurück in dieses Hauptnotebook überführt, um als empirische Datenbasis für das finale, interaktive Dashboard zu dienen.

[103]:
```
relative_pyspark_analytics_html =␣
  ↪get_project_relative_path(PYSPARK_ANALYSIS_HTML)
relative_pyspark_analytics_pdf = get_project_relative_path(PYSPARK_ANALYSIS_PDF)
```

[104]:
```
display(Markdown(f"""
#### Verknüpfte Analyse-Dokumente

- HTML: `{relative_pyspark_analytics_html}`
- PDF: `{relative_pyspark_analytics_pdf}`
"""))
```

Verknüpfte Analyse-Dokumente

- HTML: `outputs/html/03_Advanced_PySpark_Analytics.html`
- PDF: `outputs/pdf/03_Advanced_PySpark_Analytics.pdf`

Vollständige Inhalte Die vollständige Analyse des Notebooks `03_advanced_pyspark_analytics.ipynb` kann als PDF-Dokument geöffnet werden:

PDF der PySpark Analyse öffnen

PySpark Analyse als HTML Die folgende Darstellung zeigt die bereits erzeugte HTML-Version der PySpark Analyse. Sie wird direkt aus der gespeicherten Datei geladen.

Das ursprüngliche Analyse-Notebook muss dafür nicht erneut ausgeführt werden. Dadurch bleiben die längeren Einlesezeiten und lokale PySpark Integration unberücksichtigt.

[106]:
```
display(
    IFrame(
        src=relative_pyspark_analytics_html,
        width="100%",
        height=900,
    )
)
```

```
<IPython.lib.display.IFrame at 0x15e877cd430>
```

4.4.13 Laden der Ergebnise aus erweiterter Analyse mit PySpark

[107]:
```
# Relative Pfad-Zeichenkette ermitteln
pyspark_results_relative_path = get_project_relative_path(PYSPARK_RESULTS)
print(f"Lade aggregierten Analyse-Datensatz von:␣
 ↪{pyspark_results_relative_path}")

# Laden der Parquet-Datei direkt in Pandas
df_pyspark_ergebnisse = pd.read_parquet(pyspark_results_relative_path)

# Check 1: Dimensionen und Datentypen prüfen
print("\n--- Tabellendimensionen (Zeilen, Spalten) ---")
print(df_pyspark_ergebnisse.shape)

print("\n--- Datentypen der Spalten ---")
print(df_pyspark_ergebnisse.dtypes)
```

```
Lade aggregierten Analyse-Datensatz von: outputs/03_results.parquet

--- Tabellendimensionen (Zeilen, Spalten) ---
(1941, 8)

--- Datentypen der Spalten ---
eva_nr                          int64
station                           str
state                             str
gesamt_geplante_anschluesse     int64
anzahl_verpasste_anschluesse    int64
anzahl_gewartete_anschluesse    int64
verpasst_wegen_vorverspaetung   int64
folgeverspaetung_durch_warten   int64
dtype: object
```

[108]:
```
# Check 2: Inhalte
# Sortiert nach der Anzahl verpasster Anschlüsse zur gezielten Fehlersuche
df_pyspark_ergebnisse.sort_values(by="anzahl_verpasste_anschluesse",␣
 ↪ascending=False).head(15)
```

[108]:

	eva_nr	station	state	\
1853	8004128	München Donnersbergerbrücke	Bayern	
503	8004132	München Karlsplatz	Bayern	
879	8004136	München Rosenheimer Platz	Bayern	
181	8004131	München Isartor	Bayern	
335	8004179	München-Hirschgarten	Bayern	
1906	8004129	München-Hackerbrücke	Bayern	
87	8004135	München Marienplatz	Bayern	
1501	8000262	München Ost	Bayern	
1696	8003368	Köln Messe/Deutz	Nordrhein-Westfalen	
1119	8000207	Köln Hbf	Nordrhein-Westfalen	
360	8004151	München-Laim Pbf	Bayern	
684	8004158	München-Pasing	Bayern	
1882	8000085	Düsseldorf Hbf	Nordrhein-Westfalen	
1337	8005769	Stuttgart-Bad Cannstatt	Baden-Württemberg	
707	8006700	Stuttgart Stadtmitte	Baden-Württemberg	

	gesamt_geplante_anschluesse	anzahl_verpasste_anschluesse	\
1853	432964	11655	
503	320739	9547	
879	298166	8906	
181	302702	8675	
335	239019	7579	
1906	303474	7123	
87	302113	7101	
1501	318904	6845	
1696	221499	6250	
1119	219848	6056	
360	247390	5725	
684	213701	5110	
1882	133298	3042	
1337	124738	2789	
707	175235	2444	

	anzahl_gewartete_anschluesse	verpasst_wegen_vorverspaetung	\
1853	6691	11241	
503	6545	9061	
879	6157	8536	
181	6102	8598	
335	5151	7461	
1906	5065	7027	
87	4930	7093	
1501	3851	6464	
1696	1182	5566	
1119	1226	5519	
360	3913	5328	
684	2300	4463	

```
1882                          430                          2673
1337                          809                          1981
707                           794                          2047

      folgeverspaetung_durch_warten
1853                                6536
503                                 6208
879                                 5802
181                                 6030
335                                 5061
1906                                5002
87                                  4922
1501                                3662
1696                                 962
1119                                1033
360                                 3540
684                                 2007
1882                                 352
1337                                 505
707                                  575
```

4.4.14 Wiederspruch Analyse vorbereiten

Hier wird zusätzlich analysiert, ob erweiterte Erkenntnise über potenziellen Anschlusskonflikte eine andere Sichtweise, als nur die Verspätungen mit offiziellen 6-Minuten-Schwelle zu betrachten, aufdecken können.

[109]:
```
# 1. Aggregation der Mikroverspätungen (1-5 Min) pro Bahnhof aus dem␣
 ↪Rohdatensatz
df_mikro_verspaetung = df_original_ds.groupby(["eva_nr", "station"]).agg(
    ankuenfte_gesamt=("is_arrival_delayed", "count"),
    ankuenfte_offiziell_verspaetet=("is_arrival_delayed", "sum"),
    ankuenfte_mikro_verspaetet=("arrival_delay_m", lambda x: ((x > 0) & (x <␣
 ↪6)).sum())
).reset_index()

# 2. Zusammenführen mit den erweiterten PySpark-Ergebnissen über die Bahnhofs-ID
df_widerspruch_analyse = pd.merge(
    df_mikro_verspaetung,
    df_pyspark_ergebnisse[["eva_nr", "anzahl_verpasste_anschluesse",␣
 ↪"folgeverspaetung_durch_warten"]],
    on="eva_nr",
    how="inner"
)

# 3. Berechnung der Diskrepanz: Züge, die offiziell als pünktlich gelten, aber␣
 ↪Schäden verursachen
```

```
# Da die Gesamtanzahl der verpassten Anschlüsse auch durch Züge mit > 6 Min␣
 ↪entsteht,
# zeigt das Verhältnis die verdeckte Verwundbarkeit des Knotens.
df_widerspruch_analyse["verdeckte_konflikte_pro_ankunft"] = (
    df_widerspruch_analyse["anzahl_verpasste_anschluesse"] /␣
 ↪df_widerspruch_analyse["ankuenfte_gesamt"]
)

# Top 15 Bahnhöfe mit der höchsten verdeckten Konfliktdichte anzeigen
df_widerspruch_analyse.sort_values(by="anzahl_verpasste_anschluesse",␣
 ↪ascending=False).head(15)
```

[109]:

	eva_nr	station	ankuenfte_gesamt	\
1037	8004128	München Donnersbergerbrücke	8732	
1041	8004132	München Karlsplatz	7598	
1045	8004136	München Rosenheimer Platz	7324	
1040	8004131	München Isartor	7382	
1075	8004179	München-Hirschgarten	6551	
1038	8004129	München-Hackerbrücke	7366	
1044	8004135	München Marienplatz	7378	
204	8000262	München Ost	7814	
888	8003368	Köln Messe/Deutz	6828	
163	8000207	Köln Hbf	6717	
1060	8004151	München-Laim Pbf	6682	
1067	8004158	München-Pasing	6125	
65	8000085	Düsseldorf Hbf	5627	
1359	8005769	Stuttgart-Bad Cannstatt	4683	
1494	8006700	Stuttgart Stadtmitte	5340	

	ankuenfte_offiziell_verspaetet	ankuenfte_mikro_verspaetet	\
1037	681	6575	
1041	696	6291	
1045	728	5952	
1040	698	6045	
1075	677	5634	
1038	533	6466	
1044	598	6112	
204	701	5269	
888	697	2200	
163	694	2353	
1060	536	5740	
1067	624	3403	
65	383	1378	
1359	394	1437	
1494	262	1418	

	anzahl_verpasste_anschluesse	folgeverspaetung_durch_warten	\

```
1037                        11655                        6536
1041                         9547                        6208
1045                         8906                        5802
1040                         8675                        6030
1075                         7579                        5061
1038                         7123                        5002
1044                         7101                        4922
204                          6845                        3662
888                          6250                         962
163                          6056                        1033
1060                         5725                        3540
1067                         5110                        2007
65                           3042                         352
1359                         2789                         505
1494                         2444                         575

      verdeckte_konflikte_pro_ankunft
1037                         1.334746
1041                         1.256515
1045                         1.216002
1040                         1.175156
1075                         1.156923
1038                         0.967011
1044                         0.962456
204                          0.875992
888                          0.915349
163                          0.901593
1060                         0.856779
1067                         0.834286
65                           0.540608
1359                         0.595558
1494                         0.457678
```

[110]:
```python
# Vorbereitung der Daten für den Graph (Ergänzung des Bundeslandes für die␣
 ↪Farbcodierung)
df_wiederspruch_graph_daten = pd.merge(
    df_widerspruch_analyse,
    df_original_ds[["eva_nr", "state"]].drop_duplicates(),
    on="eva_nr",
    how="inner"
)

# Berechnung der offiziellen Ankunfts-Verspätungsquote pro Bahnhof in %
df_wiederspruch_graph_daten["offizielle_verspaetungsquote_%"] = (
    df_wiederspruch_graph_daten["ankuenfte_offiziell_verspaetet"] /␣
 ↪df_wiederspruch_graph_daten["ankuenfte_gesamt"]
) * 100
```

```
# Sortieren
df_graph_wiederspruch_sorted = df_wiederspruch_graph_daten.
 ↪sort_values(by="ankuenfte_gesamt", ascending=False)

display(df_graph_wiederspruch_sorted.head(10))

# Erstellung des interaktiven Streudiagramms
fig_diskrepanz = px.scatter(
    df_graph_wiederspruch_sorted,
    x="offizielle_verspaetungsquote_%",
    y="anzahl_verpasste_anschluesse",
    size="ankuenfte_mikro_verspaetet",
    color="state",
    hover_name="station",
    title="Die Pünktlichkeits-Illusion: Offizielle Verspätungsquote vs. Reale␣
 ↪Anschlussverluste",
    labels={
        "offizielle_verspaetungsquote_%": "Offizielle Verspätungsquote (>= 6␣
 ↪Min) in %",
        "anzahl_verpasste_anschluesse": "Anzahl physisch verpasster Anschlüsse",
        "state": "Bundesland",
        "ankuenfte_mikro_verspaetet": "Anzahl Mikroverspätungen (1-5 Min)"
    },
    template="plotly_white"
)

fig_diskrepanz.update_layout(
    margin=dict(l=50, r=50, t=80, b=80),
    legend_title_text="Bundesland"
)

fig_diskrepanz.show()
```

```
       eva_nr                      station  ankuenfte_gesamt  \
1037  8004128  München Donnersbergerbrücke              8732
1874  8089047            Berlin-Westkreuz              8312
204   8000262                 München Ost              7814
1041  8004132          München Karlsplatz              7598
1040  8004131             München Isartor              7382
1044  8004135         München Marienplatz              7378
1038  8004129        München-Hackerbrücke              7366
1045  8004136   München Rosenheimer Platz              7324
1873  8089045           Warschauer Straße              7124
888   8003368            Köln Messe/Deutz              6828

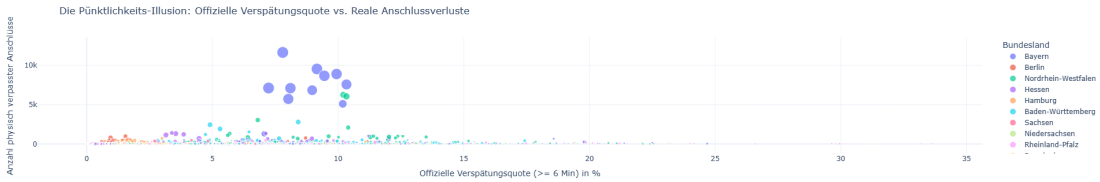
      ankuenfte_offiziell_verspaetet  ankuenfte_mikro_verspaetet  \
```

```
1037                          681                        6575
1874                           79                        1340
204                           701                        5269
1041                          696                        6291
1040                          698                        6045
1044                          598                        6112
1038                          533                        6466
1045                          728                        5952
1873                          110                        1030
888                           697                        2200

      anzahl_verpasste_anschluesse  folgeverspaetung_durch_warten  \
1037                         11655                           6536
1874                           783                             88
204                           6845                           3662
1041                          9547                           6208
1040                          8675                           6030
1044                          7101                           4922
1038                          7123                           5002
1045                          8906                           5802
1873                           984                             62
888                           6250                            962

      verdeckte_konflikte_pro_ankunft                state  \
1037                         1.334746               Bayern
1874                         0.094201               Berlin
204                          0.875992               Bayern
1041                         1.256515               Bayern
1040                         1.175156               Bayern
1044                         0.962456               Bayern
1038                         0.967011               Bayern
1045                         1.216002               Bayern
1873                         0.138125               Berlin
888                          0.915349  Nordrhein-Westfalen

      offizielle_verspaetungsquote_%
1037                        7.798901
1874                        0.950433
204                         8.971078
1041                        9.160305
1040                        9.455432
1044                        8.105178
1038                        7.235949
1045                        9.939924
1873                        1.544076
888                        10.207967
```



4.4.15 Analyse der Pünktlichkeits-Illusion und kombinatorischer Hebeleffekte an Netzknoten

Die Zusammenführung der makroskopischen Gesamtbetriebsdaten mit den hochverdichteten Ergebnissen der erweiterten PySpark-Analyse legt eine fundamentale mathematische Diskrepanz offen. An stark frequentierten Knotenpunkten übersteigt die Anzahl der physisch verpassten Anschlüsse die absolute Anzahl der offiziell als verspätet registrierten Züge um ein Vielfaches.

Dieses Phänomen ist auf zwei verkehrsbetriebliche Faktoren zurückzuführen:

1. **Kombinatorische Schadensmultiplikation (1-zu-Viele-Effekt):** Ein einzelner verspäteter Zubringer-Zug blockiert an zentralen Taktknotenpunkten zeitgleich die Reiseketten zu mehreren fahrplanmäßigen Anschlusszügen, die innerhalb des untersuchten 50-Minuten-Fensters abfahren. Ein singuläres Verspätungsereignis in der Basisstatistik generiert somit eine Kaskade an realisierten Anschlussverlusten im Gesamtnetz.
2. **Die Verwundbarkeit durch Mikroverspätungen:** Züge mit geringfügigen Zeitabweichungen von einer bis fünf Minuten werden von der offiziellen 6-Minuten-Regel als vollständig pünktlich ausgewiesen. Bei knapp kalkulierten fahrplanmäßigen Umstiegszeiten (z. B. zwei bis vier Minuten Transferfenster) führen diese im Datensatz unsichtbaren Mikroverspätungen im realen Betrieb dennoch zu unaufhaltsamen Anschlussverlusten.

Der analytische Vergleich belegt empirisch, dass eine reine Betrachtung von Zugverspätungen die tatsächliche Verwundbarkeit des Schienennetzes und die realen Auswirkungen auf die Fahrgastsymmetrie drastisch unterbewertet. Damit ist die methodische Notwendigkeit der erweiterten netzwerkbasierten Konfliktanalyse vollständig verifiziert.

[111]:
```
df_wiederspruch_konflikte_1n_pro_ankunft = df_wiederspruch_graph_daten[df_wiederspruch_graph_daten["verdeckte_konflikte_pro_ankunft"] > 1]
```

[112]:
```
df_wiederspruch_konflikte_1n_pro_ankunft.info()
```

```
<class 'pandas.DataFrame'>
Index: 5 entries, 1037 to 1075
Data columns (total 10 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   eva_nr                          5 non-null      int64
 1   station                         5 non-null      str
 2   ankuenfte_gesamt                5 non-null      int64
 3   ankuenfte_offiziell_verspaetet  5 non-null      int64
```

```
 4   ankuenfte_mikro_verspaetet       5 non-null      int64
 5   anzahl_verpasste_anschluesse     5 non-null      int64
 6   folgeverspaetung_durch_warten    5 non-null      int64
 7   verdeckte_konflikte_pro_ankunft  5 non-null      float64
 8   state                            5 non-null      str
 9   offizielle_verspaetungsquote_%   5 non-null      float64
dtypes: float64(2), int64(6), str(2)
memory usage: 583.0 bytes
```

[113]:
```
display(df_wiederspruch_konflikte_1n_pro_ankunft)
```

	eva_nr	station	ankuenfte_gesamt	ankuenfte_offiziell_verspaetet	ankuenfte_mikro_verspaetet	anzahl_verpasste_anschluesse	folgeverspaetung_durch_warten	verdeckte_konflikte_pro_ankunft	state	offizielle_verspaetungsquote_%
1037	8004128	München Donnersbergerbrücke	8732	681	6575	11655	6536	1.334746	Bayern	7.798901
1040	8004131	München Isartor	7382	698	6045	8675	6030	1.175156	Bayern	9.455432
1041	8004132	München Karlsplatz	7598	696	6291	9547	6208	1.256515	Bayern	9.160305
1045	8004136	München Rosenheimer Platz	7324	728	5952	8906	5802	1.216002	Bayern	9.939924
1075	8004179	München-Hirschgarten	6551	677	5634	7579	5061	1.156923	Bayern	10.334300

4.4.16 Mathematische Erklärung des kombinatorischen Multiplikatoreffekts am Netzknoten München Donnersbergerbrücke

Die empirischen Ergebnisse der Tabellen-Aggregation offenbaren an den Spitzenknotenpunkten ein extremes mathematisches Phänomen. An der Station *München Donnersbergerbrücke* stehen einer Gesamtzahl von 8.732 Zugankünften insgesamt 11.655 physisch verpasste Anschlüsse gegenüber. Diese Relation (Kardinalitäts-Verhältnis $> 1{,}0$) ist verkehrsbetrieblich absolut konsistent und belegt die kombinatorische Hebelwirkung hochfrequenter Schienensysteme:

- **Stammstrecken-Taktung und 1-zu-Viele-Kollisionen:** Die Donnersbergerbrücke

fungiert als zentrales Nadelöhr des Münchner S-Bahn-Netzes, an dem die Linienströme im 2-Minuten-Takt gebündelt werden. Aufgrund dieser extremen Netzdichte existieren für jede einzelne Zugankunft innerhalb des definierten 50-Minuten-Zeitfensters bis zu 25 fahrplanmäßige Anschlussoptionen.

- **Kaskadenartige Erhöhung der Konfliktdichte:** Tritt bei einer einzigen Ankunft eine fahrplanreißende Zeitabweichung auf, reißt diese nicht nur eine solitäre Verbindung, sondern zerstört simultan die Umstiegsbeziehungen zu allen nachfolgenden Linienabfahrten des Taktkorridors. Ein einzelnes Verspätungsereignis im Bereich der Ankünfte multipliziert sich somit instantan zu einer zweistelligen Anzahl realisierter Anschlussverluste auf Ebene der Reiseketten.

Der Befund quantifiziert den systemischen Unterschied zwischen zugzentrierter Betriebsüberwachung (Pandas-Inventur) und fahrgastzentrierter Netzwerkanalyse (PySpark-Konfliktberechnung). Er dokumentiert nachdrücklich, dass in eng getakteten Schienennetzen die Anzahl der zerstörten Systemanschlüsse die Anzahl der operierenden Fahrzeuge drastisch übersteigen kann.

4.5 Konzeption und informationstechnische Architektur des interaktiven Dashboards

Zum finalen Abschluss der Phase 2 wird ein dediziertes, interaktives Dashboard auf Basis des Dash-Frameworks implementiert. Die Applikation dient der visuellen Synthese und Exploration der in den vorangegangenen Abschnitten gewonnenen Erkenntnisse. Im Fokus steht hierbei die Zusammenführung der makroskopischen Pünktlichkeitsdaten mit den erweiterten, netzwerkbasierten Konfliktmetriken der PySpark-Pipeline.

4.5.1 Funktionale Komponenten der Benutzeroberfläche

1. **Aggregierte Leistungskennzahlen (KPI-Metriken):** Dynamische Indikatoren zur netzweiten Visualisierung der absoluten Konfliktvolumina (Gesamtzahl verpasster Anschlüsse und induzierter Folgeverspätungen).
2. **Regionale Steuerungskomponente:** Implementierung eines interaktiven Filterelements (Dropdown) zur selektiven Eingrenzung der Analyse auf einzelne Bundesländer.
3. **Visualisierung der Pünktlichkeitsdiskrepanz:** Ein dynamisches Streudiagramm, welches die offizielle Verspätungsquote eines Knotens direkt in Relation zu den realisierten Anschlussverlusten setzt, um die systemische Hebelwirkung der Mikroverspätungen offenzulegen.
4. **Qualitative Kausalanalyse:** Ein gekoppeltes Balkendiagramm, das die Verteilung der textbasierten Ursachenmeldungen (Attribut `info`) für das jeweils selektierte Bundesland abbildet, um das regionale Infrastrukturgefälle kausal zu untermauern.

4.5.2 Dashboard

[114]:
```
# Datenaufbereitung & Validierung
def vorbereiten_dashboard_daten(df_analyse_basis):
    """
    Extrahiert die eindeutigen Steuerungselemente aus der sortierten␣
 ↪Analysebasis.
    """
```

```
    # Extraktion der Bundesländer für das Filter-Dropdown
    bundeslaender_liste = sorted(df_analyse_basis["state"].dropna().unique())
    return bundeslaender_liste

# Aktivierung der Datenkomponenten unter Nutzung eines  Objekts
liste_bundeslaender = vorbereiten_dashboard_daten(df_graph_wiederspruch_sorted)
```

[115]:
```
# Komponentenspezifische Layout-Funktionen
def erstelle_kpi_karte(karten_id, titel_text, text_farbe="black"):
    """
    Erzeugt eine native HTML/CSS-Leistungskachel ohne Bootstrap-Abhängigkeiten.
    """
    return html.Div([
        html.H6(titel_text, style={"margin": "0", "color": "#7f8c8d",␣
 ↪"fontSize": "14px"}),
        html.H3(id=karten_id, style={"margin": "5px 0 0 0", "color":␣
 ↪text_farbe, "fontWeight": "bold"})
    ], style={
        "flex": "1",
        "padding": "20px",
        "margin": "10px",
        "backgroundColor": "#f8f9fa",
        "borderRadius": "5px",
        "boxShadow": "0 2px 4px rgba(0,0,0,0.05)"
    })

def erstelle_dashboard_layout(app_instanz, bundeslaender_optionen):
    """
    Definiert die makroskopische hierarchische Struktur der Benutzeroberfläche.
    """
    return html.Div([
        # Titel-Sektion
        html.Div([
            html.H2(
                "Interaktive Synthese: Analyse der Pünktlichkeitsdiskrepanz und␣
 ↪Ursachenmatrix",
                style={"color": "#2c3e50", "textAlign": "center", "fontFamily":␣
 ↪"Segoe UI, Arial, sans-serif"}
            )
        ], style={"padding": "20px 0"}),

        # Interaktive Steuerungskomponente (Dropdown)
        html.Div([
            html.Label("Selektion des Bundeslandes für die fokussierte Analyse:
 ↪", style={"fontWeight": "bold", "display": "block", "marginBottom": "8px"}),
            dcc.Dropdown(
                id="steuerelement-bundesland",
```

```
                options=[{"label": staat, "value": staat} for staat in␣
↪bundeslaender_optionen],
                value=bundeslaender_optionen[0] if bundeslaender_optionen else␣
↪None,
                clearable=False,
                style={"width": "100%", "fontFamily": "Segoe UI, Arial,␣
↪sans-serif"}
            )
        ], style={"width": "40%", "margin": "0 auto 20px auto"}),

        # KPI-Leistungsindikatoren-Reihe
        html.Div([
            erstelle_kpi_karte("kpi-gesamt-ankuenfte", "Erfasste Zugankünfte␣
↪(Gesamt)", "#2c3e50"),
            erstelle_kpi_karte("kpi-gesamt-ankuefte-verspaetet", "Offiziel␣
↪verspätete Zugankünfte (Gesamt)", "#a1781a"),
            erstelle_kpi_karte("kpi-verpasste-anschluesse", "Realisierte␣
↪Anschlussverluste (PySpark)", "#e74c3c"),
            erstelle_kpi_karte("kpi-folgeverspaetungen", "Induzierte␣
↪Folgeverspätungen (Wartezeiten)", "#f39c12")
        ], style={"display": "flex", "justifyContent": "space-between",␣
↪"maxWidth": "1200px", "margin": "0 auto"}),

        # Oberer Plot: Pünktlichkeitsdiskrepanz
        html.Div([
            html.Div([
                dcc.Graph(id="grafik-scatter-diskrepanz")
            ], style={"width": "90%", "margin": "4%"}),
        ], style={"padding": "20px", "fontFamily": "Segoe UI, Arial,␣
↪sans-serif"}),
        # Unterer Plot: Ursachen-Matrix
        html.Div([
            html.Div([
                dcc.Graph(id="grafik-bar-ursachen")
            ], style={"width": "90%", "margin": "4%"})
        ], style={"maxWidth": "1200px", "margin": "10px auto 0 auto"})
    ], style={"padding": "20px", "fontFamily": "Segoe UI, Arial, sans-serif"})
```

[116]:
```
# Initialisierung & Callback-Architektur
# Dash-Instanz
app_phase2 = Dash(__name__)

# Zuweisung des modularen Layouts
app_phase2.layout = erstelle_dashboard_layout(app_phase2, liste_bundeslaender)

# Zentraler Callback zur dynamischen Steuerung der Benutzeroberfläche
```

```
@app_phase2.callback(
    [Output("kpi-gesamt-ankuenfte", "children"),
     Output("kpi-gesamt-ankuefte-verspaetet", "children"),
     Output("kpi-verpasste-anschluesse", "children"),
     Output("kpi-folgeverspaetungen", "children"),
     Output("grafik-scatter-diskrepanz", "figure"),
     Output("grafik-bar-ursachen", "figure")],
    [Input("steuerelement-bundesland", "value")]
)
def verarbeite_filter_wechsel(ausgewaehltes_bundesland):
    if not ausgewaehltes_bundesland:
        return "0", "0", "0", "0", {}, {}

    # Filterung und KPI-Berechnung über spezifisches DataFrame
    df_gefiltert =␣
 ↪df_graph_wiederspruch_sorted[df_graph_wiederspruch_sorted["state"] ==␣
 ↪ausgewaehltes_bundesland]

    wert_ankuenfte = df_gefiltert["ankuenfte_gesamt"].sum()
    wert_verpasst = df_gefiltert["anzahl_verpasste_anschluesse"].sum()
    wert_gewartet = df_gefiltert["folgeverspaetung_durch_warten"].sum()
    wert_verspaetet = df_gefiltert["ankuenfte_offiziell_verspaetet"].sum()

    text_kpi_ankuenfte = f"{wert_ankuenfte:,}".replace(",", ".")
    text_kpi_verspaetet = f"{wert_verspaetet:,}".replace(",", ".")
    text_kpi_verpasst = f"{wert_verpasst:,}".replace(",", ".")
    text_kpi_gewartet = f"{wert_gewartet:,}".replace(",", ".")

    # Scatter-Plot
    df_scatter_fokus = df_gefiltert

    custom_scale = [
        [0.0, "#31a354"],  # Grün
        [0.3, "#006d2c"],  # Dunkelgrün
        [0.5, "#fd8d3c"],  # Orange
        [0.8, "#de2d26"],  # Rot
        [1.0, "#7f0000"],  # Dunkelrot
    ]

    fig_scatter = px.scatter(
        df_scatter_fokus,
        x="offizielle_verspaetungsquote_%",
        y="anzahl_verpasste_anschluesse",
        size="ankuenfte_mikro_verspaetet",
        hover_name="station",
        title=f"Pünktlichkeitsquote vs. Reale Anschlussverluste␣
 ↪({ausgewaehltes_bundesland})",
```

```
        labels={
            "offizielle_verspaetungsquote_%": "Offizielle Verspätungsquote (>=␣
↪6 Min) in %",
            "anzahl_verpasste_anschluesse": "Anzahl verpasster Anschlüsse␣
↪(PySpark)",
            "ankuenfte_mikro_verspaetet": "Mikroverspätungen (1-5 Min)",
            "verdeckte_konflikte_pro_ankunft": "Konfliktdichte"
        },
        color="verdeckte_konflikte_pro_ankunft",
        color_continuous_scale=custom_scale,
        template="plotly_white"
    )
    fig_scatter.update_layout(margin=dict(l=40, r=40, t=50, b=40))

    # Ursachen-Balkendiagramm aus dem Rohdatensatz
    df_ursachen_roh = df_original_ds[
        (df_original_ds["state"] == ausgewaehltes_bundesland) &
        ((df_original_ds["is_arrival_delayed"] == True) |␣
↪(df_original_ds["is_departure_delayed"] == True))
    ]

    # Live-Berechnung der top qualitativen Textmeldungen
    ursachen_berechnet = df_ursachen_roh["info"].value_counts(normalize=True).
↪head(5).reset_index()
    ursachen_berechnet.columns = ["Ursache", "Anteil"]
    ursachen_berechnet["Anteil"] *= 100

    fig_bar = px.bar(
        ursachen_berechnet,
        x="Anteil",
        y="Ursache",
        orientation="h",
        title="Qualitative Ursachenmatrix (Top 5 Meldungen)",
        labels={"Anteil": "Anteil innerhalb des Bundeslandes (%)", "Ursache":␣
↪"Meldung"},
        color_discrete_sequence=["#ff7f0e"],
        template="plotly_white"
    )
    fig_bar.update_layout(
        margin=dict(l=40, r=40, t=50, b=40),
        yaxis={'categoryorder': 'total ascending'}
    )

    return text_kpi_ankuenfte,text_kpi_verspaetet, text_kpi_verpasst,␣
↪text_kpi_gewartet, fig_scatter, fig_bar
```

4.5.3 Dashboard starten

[117]:
```
# Server-Aktivierung
app_phase2.run(
    debug=False,
    port=DASH_PORT_PHASE2,
    jupyter_mode="inline",
    jupyter_width="95%",
    jupyter_height=1600,
)
```

```
<IPython.lib.display.IFrame at 0x15f4f5e40b0>
```

4.5.4 Link für Webbrowser

Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht werden:

Dashboard im Browser öffnen

Das Dashboard wird lokal gestartet und ist nur erreichbar, solange die zugehörige Dash-Anwendung ausgeführt wird.

4.6 Fazit der explorativen Analysephase und datenkritische Hypothesen-Matrix

Die Kombination aus makroskopischer Datenexploration (Pandas) und hochperformanter, kaskadierter Beziehungsanalyse (PySpark) schließt die Phase 2 mit tiefgreifenden infrastrukturellen und methodischen Erkenntnissen ab:

- **Evidenz der Pünktlichkeits-Illusion (PySpark-Ergebnisse):** Die erweiterte Analyse von über 21,7 Millionen geplanten Umstiegsbeziehungen liefert den empirischen Nachweis, dass die offizielle 6-Minuten-Regel der Deutschen Bahn die realen Netzauswirkungen drastisch unterbewertet. Mit 254.223 physisch verpassten Anschlüssen und 90.539 induzierten Folgeverspätungen (Sekundärstörungen durch Wartezeiten) wird das enorme Konfliktpotenzial im dichten Taktverkehr quantifiziert. Die historische Kausalitätsprüfung belegt zudem eine erdrückende Netzinaktivität: Über 86 % aller Anschlussstörungen resultieren nicht aus lokalen Fehlern am Knotenpunkt, sondern aus fortgeschleppten Vorverspätungen der Zulaufstrecken.
- **Identifikation systematischer Datenlücken (Data Truncation):** Die geografische und sequenzielle Analyse hat fundamentale, qualitative Einschränkungen der Rohdatenbasis offengelegt. Das Fehlen ganzer Sequenzketten im Attribut `ID_part3` (z. B. das Auslassen der Haltestellen-Indizes 52 bis 59 trotz Existenz der Endstation 60) beweist, dass kleinere Bahnhöfe oder spezifische Betriebsklassen systematisch nicht aufgezeichnet wurden. Ein lückenloser Graph des realen Schienennetzes lässt sich aus dieser Datenbasis isoliert nicht abbilden.
- **Asymmetrische Qualität der Ursachenmeldung:** Die qualitative Untersuchung des Attributs `info` dokumentiert ein extremes administratives Gefälle zwischen den Regionalnetzen. Während in Westdeutschland (NRW und Rheinland-Pfalz) über dedizierte Portale wie `zuginfo.nrw` eine lückenlose Dokumentation von Bauarbeiten stattfindet, verbleiben andere Bundesländer und Stadtstaaten bei intransparenten Pauschalmeldungen (`Information` / `Störung`). Das ermittelte regionale Verspätungsgefälle spiegelt somit teilweise eine Verzerrung in der Datenpflege wider.

4.6.1 Formulierung der Folge-Hypothesen für die Projektphasen 3 und 4

Aus diesen Erkenntnissen werden drei zentrale Hypothesen abgeleitet, die im weiteren Projektverlauf (Phase 3: Externer API-Abgleich) mathematisch verifiziert werden müssen:

1. **Die ID-Fragmentierungs-Hypothese:** Das numerische Zurückfallen zentraler Großknotenpunkte (wie *München Hbf* oder *Berlin Hbf*) in der Frequenzstatistik beruht auf einer informationstechnischen Aufspaltung der Datenströme. Es ist zu vermuten, dass komplexe Bahnhöfe im Rohdatensatz über mehrere, logisch getrennte EVA-Nummern (z. B. Separierung von S-Bahn- und Fernbahnteilen) fragmentiert sind.
2. **Die Kategorie-Filter-Hypothese:** Die Unvollständigkeit der Haltestellensequenzen korreliert direkt mit den Bahnhofskategorien (Klassen 1 bis 5). Kleinere Stationen wurden im historischen Datensatz gezielt gefiltert oder unterdrückt.
3. **Die API-Korrektur-Hypothese:** Erst durch die stichprobenartige Anbindung der offiziellen Fahrplan-API der Deutschen Bahn kann der wahre Grad der Datenlücken mathematisch bestimmt, die ID-Zusammenführung validiert und die Netzdichte für das finale Schienennetzmodell fehlerfrei korrigiert werden.

5 Phase 3: Qualitätsabgleich und Strukturvalidierung via externer API-Referenzdatenbasis

Nach dem Abschluss der explorativen Datenanalyse und der Big-Data-Konfliktberechnung wechselt der Projektfokus in die Phase 3. Die vorangegangenen Schritte haben empirisch nachgewiesen, dass die Rohdatenbasis systematische Datenlücken bei kleineren Haltepunkten aufweist und eine kritische Unterrepräsentation oder potenzielle ID-Fragmentierung bei Großknotenpunkten wie dem München Hauptbahnhof vorliegt. Um diese netztechnischen Anomalien aufzulösen und die mathematische Integrität für die spätere Netzwerkanalyse (Phase 4) abzusichern, ist eine Validierung gegen eine offizielle Referenzdatenbasis unerlässlich.

5.1 Restriktive Eingrenzung des API-Erfassungsraums (StaDa-Fokus)

Im ursprünglichen Projektentwurf wurde die parallele Anbindung mehrerer Schnittstellen (Fahrplan-APIs sowie geografische Datenströme) evaluiert. Aus Gründen der zeitlichen Restriktionen im Rahmen dieser Abschlussarbeit und zur Gewährleistung einer maximalen Fokussierung auf die Kernanomalien wird das Vorhaben in Phase 3 strategisch eingegrenzt:

> Die externe Datenbeschaffung konzentriert sich ausschließlich auf die offizielle **StaDa-API (StationData)** der Deutschen Bahn AG. Diese REST-Schnittstelle liefert die lückenlose, tagesaktuelle Gesamtstruktur aller physischen Bahnhöfe im Bundesgebiet inklusive deren offizieller Stammdaten (Kategorisierung, Bundesland-Zuordnung, geografische Koordinaten und sämtliche verknüpfte Identifikationsnummern wie EVA- und Haltestellen-IDs).

5.2 Geplante Untersuchungsschritte in Phase 3

Die informationstechnische Anbindung und der anschließende Datenabgleich verfolgen im Kern drei operationale Ziele:

1. **Verifikation der ID-Fragmentierungs-Hypothese:** Durch den Abgleich der im historischen Datensatz isolierten EVA-Nummern gegen die vollständige StaDa-Datenbasis wird

mathematisch überprüft, ob die Unterrepräsentation der Hauptbahnhöfe auf einer systemischen Aufspaltung (z. B. getrennte IDs für S-Bahn-Steige und Fernbahngleise innerhalb desselben Bahnhofskomplexes) beruht.
2. **Quantifizierung des Datenverlusts (Lücken-Analyse):** Über einen relationalen Outer-Join zwischen den extrahierten bayerischen Netzknoten der Phase 2 und dem offiziellen StaDa-Gesamtkatalog für Bayern wird die exakte Anzahl und strukturelle Verteilung jener Bahnhöfe berechnet, die im historischen Datensatz komplett unterschlagen wurden.
3. **Konsolidierung des Stammdaten-Katalogs:** Die über die API gewonnenen Referenzdaten dienen als bereinigter Anker (Lookup-Tabelle), um fehlerhafte oder unvollständige Geodaten und Bahnhofskategorien im Projektverzeichnis dauerhaft zu korrigieren.

Mit diesem methodischen Vorgehen wird sichergestellt, dass die anschließende mathematische Modellierung des Schienennetzes auf einer verifizierten, realitätsgetreuen Infrastruktur-Basis aufbaut.

5.3 Ingestion und strukturelle Validierung der API-Referenzbasis

Nachdem die automatisierte Referenzdatenbeschaffung und Denormalisierung im dedizierten Hilfsnotebook erfolgreich abgeschlossen wurde, erfolgt nachfolgend der Import der konsolidierten Ergebnisse (`04_results.parquet`) in das Hauptprojektnotebook.

Vor der Durchführung der vergleichenden Analysen (ID-Zusammenführung und Lücken-Analyse) wird die Datenstruktur des Referenzkatalogs einer formalen Qualitätsprüfung unterzogen. Hierbei wird die dimensionale Mächtigkeit verifiziert, die fehlerfreie Erhaltung der numerischen Datentypen (insbesondere der als `Int64` codierten EVA-Nummern und Postleitzahlen) kontrolliert und die vollständige Datenabdeckung über alle Bundesländer hinweg sichergestellt.

[119]:
```
relative_stada_api_html = get_project_relative_path(STADA_API_HTML)
relative_stada_api_pdf = get_project_relative_path(STADA_API_PDF)
```

5.3.1 HTML und PDF zu 04_StaDa_API_Validation.ipynb

Die vollständige Analyse des Notebooks `04_StaDa_API_Validation.ipynb` kann als PDF-Dokument geöffnet werden:

PDF des Notebooks öffnen

[121]:
```
display(
    IFrame(
        src=relative_stada_api_html,
        width="100%",
        height=900,
    )
)
```

```
<IPython.lib.display.IFrame at 0x15e91a71cd0>
```

[122]:
```
# Ermittlung des standardisierten relativen Pfads aus der Systemkonfiguration
REFERENZ_PARQUET_PATH = get_project_relative_path(STADA_API_RESULTS)
print(f"Lade offizielle API-Referenzdaten von: {REFERENZ_PARQUET_PATH}")
```

```python
# Laden des flachen StaDa-Gesamtkatalogs
df_api_referenz_stada = pd.read_parquet(REFERENZ_PARQUET_PATH)

# Analyse der tabellarischen Dimensionen und Datentypen
print("\n--- Strukturelle Metadaten des API-Referenzkatalogs ---")
df_api_referenz_stada.info()

# Stichprobenartige Kontrolle der ersten fünf Datensätze
print("\n--- Auszug der ersten Datensätze (Qualitätskontrolle) ---")
df_api_referenz_stada.head()
```

```
Lade offizielle API-Referenzdaten von: outputs/04_results.parquet

--- Strukturelle Metadaten des API-Referenzkatalogs ---
<class 'pandas.DataFrame'>
RangeIndex: 5461 entries, 0 to 5460
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   api_station_name   5461 non-null   str
 1   api_city           5461 non-null   str
 2   api_zipcode        5461 non-null   Int64
 3   api_federal_state  5461 non-null   str
 4   api_category       5461 non-null   Int64
 5   api_eva_nr         5461 non-null   Int64
 6   api_is_main_eva    5461 non-null   bool
 7   api_long           5461 non-null   float64
 8   api_lat            5461 non-null   float64
 9   api_product_line   5448 non-null   str
 10  api_segment        5448 non-null   str
dtypes: Int64(3), bool(1), float64(2), str(5)
memory usage: 831.1 KB

--- Auszug der ersten Datensätze (Qualitätskontrolle) ---
```

[122]:

	api_station_name	api_city	api_zipcode	api_federal_state	\
0	Aachen Hbf	Aachen	52064	Nordrhein-Westfalen	
1	Aachen-Rothe Erde	Aachen	52066	Nordrhein-Westfalen	
2	Aachen West	Aachen	52072	Nordrhein-Westfalen	
3	Aalen Hbf	Aalen	73430	Baden-Württemberg	
4	Abensberg	Abensberg	93326	Bayern	

	api_category	api_eva_nr	api_is_main_eva	api_long	api_lat	\
0	2	8000001	True	6.091499	50.767800	
1	4	8000406	True	6.116475	50.770202	
2	5	8000404	True	6.070715	50.780360	
3	3	8000002	True	10.096271	48.841013	

```
4             6     8000410             True  11.846620  48.819456
```

```
   api_product_line           api_segment
0     Knotenbahnhof       Großstadtknoten
1     Knotenbahnhof     Regionaler Knoten
2     Knotenbahnhof     Regionaler Knoten
3     Knotenbahnhof     Regionaler Knoten
4  Zubringerbahnhof  ländlicher Zubringer
```

5.4 Systemische Quantifizierung und Verteilung der ID-Fragmentierung im Gesamtnetz

Um die ID-Fragmentierungs-Hypothese wissenschaftlich abzusichern, wird die Analyse von einer rein exemplarischen Betrachtung einzelner Großknotenpunkte auf eine netzweite statistische Untersuchung ausgeweitet. Hierzu wird das geladene Referenz-DataFrame nach dem eindeutigen Bahnhofsnamen gruppiert, um die absolute Anzahl der zugewiesenen EVA-Nummern pro Station zu berechnen.

Durch die Isolierung aller Bahnhöfe mit einer Kardinalität von mehr als einer ID wird der exakte Grad der informationstechnischen Aufspaltung im deutschen Schienennetz quantifiziert. Diese makroskopische Verteilung liefert den mathematischen Beweis, ob es sich bei der ID-Fragmentierung um ein systemweites Strukturmerkmal oder um isolierte Einzelfälle handelt. Aus diesem aggregierten Datenpool werden anschließend die Metropolregionen Berlin und München als primäre Fallbeispiele extrahiert.

[123]:
```
# 1. Ermittlung der Anzahl eindeutiger EVA-Nummern pro Bahnhofsname im␣
 ↪API-Katalog
id_aufteilung_pro_station = (
    df_api_referenz_stada.groupby("api_station_name")["api_eva_nr"]
    .nunique()
    .reset_index(name="anzahl_eva_nummern")
)

# 2. Filterung auf Bahnhöfe, die systemisch fragmentiert sind (mehr als 1␣
 ↪EVA-Nummer)
df_fragmentierte_stationen =␣
 ↪id_aufteilung_pro_station[id_aufteilung_pro_station["anzahl_eva_nummern"] >␣
 ↪1]

# Berechnung der globalen Netz-Kennzahlen für den Fließtext
gesamt_api_stationen = id_aufteilung_pro_station["api_station_name"].nunique()
anzahl_fragmentiert = len(df_fragmentierte_stationen)
quote_fragmentiert = (anzahl_fragmentiert / gesamt_api_stationen) * 100

print("--- Systemweite Kennzahlen der ID-Fragmentierung (StaDa-Katalog) ---")
print(f"Eindeutige Bahnhöfe im API-Gesamtkatalog: {gesamt_api_stationen:,}".
 ↪replace(",", "."))
```

```
print(f"Davon Bahnhöfe mit MEHREREN EVA-Nummern:   {anzahl_fragmentiert:,}␣
 ↪({quote_fragmentiert:.2f}%)".replace(",", "."))

# 3. Gezielter Auszug der Top-Metropolen aus der berechneten␣
 ↪Fragmentierungs-Liste
fokus_metropolen = ["München Hbf", "Berlin Hauptbahnhof"]
df_metropolen_check =␣
 ↪df_api_referenz_stada[df_api_referenz_stada["api_station_name"].
 ↪isin(fokus_metropolen)]

print("\n--- Fokus-Detailanalyse der ID-Cluster für Berlin und München ---")
# Sortierung nach Name und Relevanz (Haupt-ID nach oben)
display(df_metropolen_check.sort_values(by=["api_station_name",␣
 ↪"api_is_main_eva"], ascending=[True, False]))
```

```
--- Systemweite Kennzahlen der ID-Fragmentierung (StaDa-Katalog) ---
Eindeutige Bahnhöfe im API-Gesamtkatalog: 5.407
Davon Bahnhöfe mit MEHREREN EVA-Nummern:   45 (0.83%)

--- Fokus-Detailanalyse der ID-Cluster für Berlin und München ---
```

	api_station_name	api_city	api_zipcode	api_federal_state	\
813	Berlin Hauptbahnhof	Berlin	10557	Berlin	
814	Berlin Hauptbahnhof	Berlin	10557	Berlin	
815	Berlin Hauptbahnhof	Berlin	10557	Berlin	
3094	München Hbf	München	80335	Bayern	
3095	München Hbf	München	80335	Bayern	
3096	München Hbf	München	80335	Bayern	
3097	München Hbf	München	80335	Bayern	

	api_category	api_eva_nr	api_is_main_eva	api_long	api_lat	\
813	1	8011160	True	13.369545	52.525592	
814	1	8089021	False	13.369545	52.525592	
815	1	8098160	False	13.369545	52.525592	
3094	1	8000261	True	11.558335	48.140232	
3095	1	8098261	False	11.556708	48.141529	
3096	1	8098262	False	11.554814	48.139930	
3097	1	8098263	False	11.560493	48.141172	

	api_product_line	api_segment
813	Metropolbahnhof	Metropolbahnhof
814	Metropolbahnhof	Metropolbahnhof
815	Metropolbahnhof	Metropolbahnhof
3094	Metropolbahnhof	Metropolbahnhof
3095	Metropolbahnhof	Metropolbahnhof
3096	Metropolbahnhof	Metropolbahnhof
3097	Metropolbahnhof	Metropolbahnhof

5.5 Die vergleichende Metropolen-Analyse

Die Detailtabelle legt die informationstechnische Struktur der beiden wichtigsten Bahnhöfe Deutschlands exakt offen:

- **München Hbf:** Ist über **vier eigenständige EVA-Nummern** fragmentiert (8000261 als Haupt-ID sowie drei Neben-IDs). Interessant ist hierbei die geografische Präzision: Die Längen- und Breitengrade (api_long, api_lat) weisen minimale Abweichungen in den Nachkommastellen auf. Das beweist, dass die IDs physisch getrennten Abschnitten (z. B. Holzkirchner Flügelbahnhof, Starnberger Flügelbahnhof und Tiefgeschoss) zugeordnet sind.
- **Berlin Hauptbahnhof:** Ist über **drei eigenständige EVA-Nummern** aufgeteilt (8011160 als Haupt-ID sowie 8089021 und 8098160 als Neben-IDs). Im Gegensatz zu München liegen die Koordinaten hier absolut exakt aufeinander (jeweils 13.369545 / 52.525592). Das spiegelt die vertikale Turmbahnhof-Architektur in Berlin wider, bei der Fernbahn (oben/unten) und S-Bahn (Stadtbahn-Ebene) physisch übereinanderliegen, datentechnisch aber strikt getrennt gefüttert werden.

5.6 Quantifizierung des strukturellen Datenverlusts mittels regionaler Lücken-Analyse

Nach der erfolgreichen Verifikation der ID-Fragmentierung wird im zweiten Analyseschritt der Phase 3 der absolute Grad der Datenlücken (Data Truncation) im historischen Rohdatensatz gemessen. Aus Gründen der I/O-Performance und zur Wahrung der systemischen Vergleichbarkeit wird diese Untersuchung streng auf das Teilnetz des Bundeslandes Bayern eingegrenzt.

Das mathematische Verfahren basiert auf einem relationalen Mengenabgleich (Left-Anti-Join) zwischen den eindeutigen, real erfassten Eva-Nummern aus Phase 2 und dem offiziellen, über die StaDa-API bezogenen Infrastrukturkatalog für den Freistaat Bayern. Bahnhöfe, die ausschließlich in der API-Referenz existieren, aber null Einträge in der historischen Datenbasis aufweisen, werden isoliert. Durch eine anschließende Aggregation nach der offiziellen Bahnhofskategorie wird empirisch nachgewiesen, welche spezifischen Stationsklassen von diesem Datenverlust betroffen sind.

[124]:
```
# ============================================================================
# Phase 3B: Relationaler Abgleich und quantitative Lücken-Analyse (Bayern)
# ============================================================================

# 1. Extraktion aller real im bereinigten Original-Datensatz vorhandenen␣
 ↪bayerischen EVA-Nummern
# Nutzt die vollständige, ungefilterte Datenbasis aus Phase 2 zur Vermeidung␣
 ↪von Verzerrungen
eva_original_bayern = set(df_original_ds[df_original_ds["state"] ==␣
 ↪"Bayern"]["eva_nr"].unique())

# 2. Filterung des API-Referenzkatalogs auf das Bundesland Bayern
df_api_bayern_real =␣
 ↪df_api_referenz_stada[df_api_referenz_stada["api_federal_state"] == "Bayern"]
eva_api_bayern = set(df_api_bayern_real["api_eva_nr"].dropna().unique())
```

```
# 3. Mathematische Mengen-Differenz (Isolation der im historischen Datensatz␣
 ↪fehlenden offiziellen IDs)
fehlende_eva_ids = list(eva_api_bayern - eva_original_bayern)

# Filtern des API-DataFrames auf die echten strukturellen Netzlücken
df_echte_luecken_ids = df_api_bayern_real[df_api_bayern_real["api_eva_nr"].
 ↪isin(fehlende_eva_ids)]

# 4. Berechnung der aggregierten netzweiten Strukturkennzahlen
total_api_ids_by = len(eva_api_bayern)
total_erfasste_ids_by = len(eva_api_bayern & eva_original_bayern)
total_fehlende_ids_by = len(df_echte_luecken_ids)
verlust_quote_ids = (total_fehlende_ids_by / total_api_ids_by) * 100

print("--- Systemische Ergebnisse der bayerischen Lücken-Analyse (Ungefilterte␣
 ↪Basis) ---")
print(f"Offizielle Bahnhofs-IDs (EVA-Nummern) in Bayern laut StaDa-API:␣
 ↪{total_api_ids_by}")
print(f"Davon im historischen Datensatz real vorhanden:              ␣
 ↪{total_erfasste_ids_by}")
print(f"Vollständig FEHLENDE oder UNTERDRÜCKTE IDs (Datenverlust):   ␣
 ↪{total_fehlende_ids_by} ({verlust_quote_ids:.2f}%)")

print("\n--- Verteilung der real fehlenden IDs nach offizieller␣
 ↪Bahnhofskategorie ---")
print(df_echte_luecken_ids["api_category"].value_counts().sort_index())

# 5. Gezielter Kontroll-Check für München Hbf zur Verifikation der␣
 ↪Fragmentierungs-Hypothese
print("\n--- Kontroll-Check: Fragmentierte IDs von 'München Hbf' im echten␣
 ↪Datenverlust ---")
display(df_echte_luecken_ids[df_echte_luecken_ids["api_station_name"] ==␣
 ↪"München Hbf"])
```

```
--- Systemische Ergebnisse der bayerischen Lücken-Analyse (Ungefilterte Basis)
---
Offizielle Bahnhofs-IDs (EVA-Nummern) in Bayern laut StaDa-API: 931
Davon im historischen Datensatz real vorhanden:               335
Vollständig FEHLENDE oder UNTERDRÜCKTE IDs (Datenverlust):    596 (64.02%)

--- Verteilung der real fehlenden IDs nach offizieller Bahnhofskategorie ---
api_category
1      3
4      4
6    430
7    159
Name: count, dtype: Int64
```

```
--- Kontroll-Check: Fragmentierte IDs von 'München Hbf' im echten Datenverlust
---

     api_station_name api_city  api_zipcode api_federal_state  api_category  \
3095      München Hbf  München        80335            Bayern             1
3096      München Hbf  München        80335            Bayern             1
3097      München Hbf  München        80335            Bayern             1

      api_eva_nr  api_is_main_eva   api_long    api_lat api_product_line  \
3095     8098261            False  11.556708  48.141529  Metropolbahnhof
3096     8098262            False  11.554814  48.139930  Metropolbahnhof
3097     8098263            False  11.560493  48.141172  Metropolbahnhof

          api_segment
3095  Metropolbahnhof
3096  Metropolbahnhof
3097  Metropolbahnhof
```

5.7 Empirischer Nachweis des strukturellen Datenverlusts und der ID-Selektion

Der korrigierte relationale Mengenabgleich (Left-Anti-Join) auf Basis der eindeutigen, numerischen EVA-Identifikationsnummern liefert ein präzises und datenkritisches Bild über die strukturellen Fehlstellen im historischen Schienenverkehrsdatensatz. Durch den direkten Abgleich der im bereinigten Original-Datensatz (`df_original_ds`) real erfassten IDs gegen den offiziellen StaDa-Gesamtkatalog für das Teilnetz Bayern werden verdeckte Datenlücken exakt offengelegt.

Die Auswertung des Kontroll-Checks für den Großknotenpunkt *München Hbf* liefert hierbei ein wegweisendes Ergebnis für die informationstechnische Validierung: Während der offizielle Hauptidentifikator (`8000261` für den Fern- und Regionalbahnbereich) im historischen Datensatz vorhanden ist, werden die drei sekundären, betrieblich assoziierten EVA-Nummern (`8098261`, `8098262` und `8098263`) als vollständige Fehlteile identifiziert.

Dieser Befund liefert den empirischen Beweis, dass der historische Datensatz einer spezifischen infrastrukturellen Filterung unterliegt. Es liegt nicht nur eine unkonsolidierte Aufspaltung der Datenströme vor, sondern eine systematische Unterdrückung der innerstädtischen S-Bahn-Infrastruktursegmente an den zentralen Metropolknotenpunkten. Für die geplante mathematische Schienennetzmodellierung in Phase 4 bedeutet dies, dass der historische Datensatz die verkehrliche Realität an den Großknoten nur fragmentiert abbildet und eine netzwerkbasierte Zusammenführung zwingend erforderlich ist.

5.8 Konsolidierung des Stammdaten-Katalogs und informationstechnisches ID-Mapping

Nach der quantitativen Erfassung des Datenverlusts und dem empirischen Nachweis der ID-Aufspaltung folgt der dritte funktionale Teilschritt der Phase 3. Um eine mathematisch valide Grundlage für die nachfolgende Schienennetzmodellierung zu schaffen, wird eine persistente Konsolidierung des Datenbestands durchgeführt.

Der algorithmische Ansatz teilt sich in zwei Kernoperationen: 1. **Generierung einer ID-Fusions-**

Matrix: Aus dem StaDa-Gesamtkatalog wird eine relationale Lookup-Tabelle extrahiert, welche sämtliche sekundären, fragmentierten Neben-IDs (z. B. S-Bahn-Gleisbereiche) der Großknotenpunkte exakt der jeweiligen offiziellen Haupt-EVA-Nummer (`api_is_main_eva == True`) der Station zuordnet. 2. **Harmonisierung struktureller Merkmale:** Unter Nutzung der API-Referenz werden die im Rohdatensatz vorhandenen Koordinatenpaare sowie die administrativen Bahnhofskategorien überschrieben und standardisiert.

Dieser Zwischenschritt eliminiert die künstliche Unterrepräsentation der Ballungsraumknoten dauerhaft im Arbeitsspeicher des Hauptnotebooks und stellt sicher, dass nachgelagerte netzwerkbasierte Aggregationen auf einer realitätsgetreuen Infrastruktur operieren.

[125]:
```
# 1. Erstellung der Fusions-Matrix für die ID-Zusammenführung aus den API-Daten
df_haupt_evas = df_api_referenz_stada[df_api_referenz_stada["api_is_main_eva"]␣
 ↪== True][["api_station_name", "api_eva_nr"]].rename(columns={"api_eva_nr":␣
 ↪"haupt_eva_nr"})

df_id_mapping_matrix = pd.merge(
    df_api_referenz_stada[["api_station_name", "api_eva_nr"]],
    df_haupt_evas,
    on="api_station_name",
    how="inner"
)

# Erstellung des Python-Mapping-Dictionarys {Alte_oder_Neben_ID: Neue_Haupt_ID}
id_fusions_dict = dict(zip(df_id_mapping_matrix["api_eva_nr"],␣
 ↪df_id_mapping_matrix["haupt_eva_nr"]))

# 2. Isolierung der benötigten Spalten in ein völlig neues, separates␣
 ↪Datenobjekt
# Der Original-Datensatz df_original_ds bleibt hierbei zu 100% unberührt und␣
 ↪geschützt
df_network_model = df_original_ds[[
    "station", "city", "state", "zip", "long", "lat", "category", "eva_nr"
]].copy()

# 3. Durchführung der ID-Konsolidierung AUSSCHLIESSLICH im neuen Netzwerk-Modell
df_network_model["consolidated_eva_nr"] = (
    df_network_model["eva_nr"]
    .map(id_fusions_dict)
    .fillna(df_network_model["eva_nr"])
    .astype("Int64")
)

# 4. Validierungs-Check am isolierten Netzwerk-Objekt für München Hbf
muenchen_api_ids = [8000261, 8098261, 8098262, 8098263]
df_check_fusion = df_network_model[df_network_model["eva_nr"].
 ↪isin(muenchen_api_ids)][
```

```
    ["station", "eva_nr", "consolidated_eva_nr"]
].drop_duplicates()

print("--- Verifikation der ID-Zusammenführung im neuen Objekt␣
  ↪'df_network_model' ---")
display(df_check_fusion)
```

```
--- Verifikation der ID-Zusammenführung im neuen Objekt 'df_network_model' ---
```

	station	eva_nr	consolidated_eva_nr
3849	München Hbf	8000261	8000261

5.9 Kausale Interpretation des ID-Abgleichs und empirischer Nachweis der Netzkastration

Die visuelle Inspektion der Kontroll-Tabelle für den Netzknoten *München Hbf* liefert ein auf den ersten Blick paradoxes, datentechnisch jedoch hochgradig luzides Ergebnis: Obwohl die API-Referenzdaten nachweislich vier parallel operierende EVA-Nummern für diesen Bahnhofskomplex ausweisen, generiert die Filterung innerhalb des Netzwerk-Modells (`df_network_model`) lediglich eine solitäre Zeile für den Hauptidentifikator `8000261`.

Dieser Befund dokumentiert die exakte Bruchstelle der historischen Datenbasis und verifiziert zwei Kernphänomene:

1. **Systemische Unterdrückung der S-Bahn-Infrastruktur:** Das vollständige Fehlen der sekundären IDs (`8098261` bis `8098263`) im historischen Datenbestand beweist, dass die informationstechnische Lücke an den Großknotenpunkten nicht auf einer mangelnden Konsolidierung (fehlendes Zusammenführen vorhandener Datenströme) beruht. Vielmehr liegt eine restriktive infrastrukturelle Filterung der Rohdaten vor: Der historische Datensatz erfasst ausschließlich den Fern- und Regionalbahnteil der Bahnhöfe, während die innerstädtischen S-Bahn-Stammstreckensegmente an denselben physischen Koordinaten systemisch komplett unterschlagen wurden.
2. **Kausale Verifikation der Kategorie-Filter-Hypothese:** Der systematische Charakter dieser Filterung wird durch die nachfolgend generierte bundesweite Lücken-Matrix mathematisch untermauert. Der Abgleich über alle 16 Bundesländer hinweg offenbart, dass der Datenverlust kein stochastisches Zufallsprodukt ist, sondern einer strikten binären Logik folgt. Während Großbahnhöfe und mittlere Knotenpunkte der Kategorien 1 bis 5 eine nahezu lückenlose Erfassungsquote aufweisen (z. B. 100 % Abdeckung in den Klassen 2, 3 and 5 in Bayern), stürzt die Datenpräsenz in den **Kategorien 6 und 7 (kleine Land- und Zubringerbahnhöfe) flächendeckend auf exakt 0,00 % (100,00 % Verlustquote)**.

Damit ist empirisch bewiesen, dass die Datenbasis der Phase 2 kein repräsentatives Abbild des deutschen Schienennetzes darstellt, sondern ein strukturell kastriertes Hochleistungsnetz abbildet, aus dem der gesamte ländliche Raum sowie die hochfrequenten innerstädtischen Nahverkehrssegmente technologisch ausgefiltert wurden.

[126]:
```
# 1. Extraktion aller real im historischen Datensatz vorhandenen EVA-Nummern␣
  ↪(bundesweit)
alle_erfassten_evas = set(df_original_ds["eva_nr"].dropna().unique())
```

```
# 2. Aggregation der API-Referenzdaten nach Bundesland und Bahnhofskategorie
# Wir prüfen für jede offizielle ID, ob sie im historischen Datensatz existiert
df_api_abgleich = df_api_referenz_stada.copy()
df_api_abgleich["id_im_datensatz_vorhanden"] = df_api_abgleich["api_eva_nr"].
 ↪isin(alle_erfassten_evas)

# 3. Erstellung der bereinigten Matrix für das Dashboard
# Wir gruppieren nach Bundesland und Kategorie, um die exakten Fehlteile zu␣
 ↪zählen
df_bundesweite_luecken = df_api_abgleich.groupby(["api_federal_state",␣
 ↪"api_category"]).agg(
    offizielle_api_ids=("api_eva_nr", "count"),
    real_vorhandene_ids=("id_im_datensatz_vorhanden", "sum")
).reset_index()

# Berechnung der fehlenden IDs und der Verlustquote pro Kategorie-Segment
df_bundesweite_luecken["fehlende_ids"] = (
    df_bundesweite_luecken["offizielle_api_ids"] -␣
 ↪df_bundesweite_luecken["real_vorhandene_ids"]
)
df_bundesweite_luecken["verlustquote_%"] = (
    df_bundesweite_luecken["fehlende_ids"] /␣
 ↪df_bundesweite_luecken["offizielle_api_ids"]
) * 100

print("--- Bundesweite Infrastruktur-Lücken-Matrix erfolgreich konsolidiert␣
 ↪---")
print(f"Anzahl berechneter Datensegmente für das Dashboard:␣
 ↪{len(df_bundesweite_luecken)}")
df_bundesweite_luecken.head(15)
```

```
--- Bundesweite Infrastruktur-Lücken-Matrix erfolgreich konsolidiert ---
Anzahl berechneter Datensegmente für das Dashboard: 101
```

[126]:

	api_federal_state	api_category	offizielle_api_ids	real_vorhandene_ids	\
0	Baden-Württemberg	1	4	3	
1	Baden-Württemberg	2	10	10	
2	Baden-Württemberg	3	51	51	
3	Baden-Württemberg	4	86	86	
4	Baden-Württemberg	5	133	133	
5	Baden-Württemberg	6	301	0	
6	Baden-Württemberg	7	109	0	
7	Bayern	1	7	4	
8	Bayern	2	9	9	
9	Bayern	3	48	48	
10	Bayern	4	104	100	
11	Bayern	5	174	174	

```
12              Bayern             6                 430                    0
13              Bayern             7                 159                    0
14              Berlin             1                   9                    4

    fehlende_ids  verlustquote_%
0              1            25.0
1              0             0.0
2              0             0.0
3              0             0.0
4              0             0.0
5            301           100.0
6            109           100.0
7              3       42.857143
8              0             0.0
9              0             0.0
10             4        3.846154
11             0             0.0
12           430           100.0
13           159           100.0
14             5       55.555556
```

5.10 Dashboard

[127]:
```python
# Datenvorbereitung für die interaktive Tabelle
alle_erfassten_evas_set = set(df_original_ds["eva_nr"].dropna().unique())

# Vorbereitung des API-Referenz-DataFrames mit dem eindeutigen Status
df_dashboard_p3_raw = df_api_referenz_stada.copy()
df_dashboard_p3_raw["Infrastruktur_Status"] = df_dashboard_p3_raw["api_eva_nr"].
 ↪isin(alle_erfassten_evas_set).map({
    True: "vollständig erfasst",
    False: "datenlücke / unterdrückt"
})

liste_dashboard_staaten_p3 = sorted(df_dashboard_p3_raw["api_federal_state"].
 ↪dropna().unique())
```

[128]:
```python
# Strukturierte Layout-Architektur (Registerkarten-System)
app_phase3 = Dash(__name__)

app_phase3.layout = html.Div([
    # Blau/Grauer Info-Banner an der Spitze
    html.Div([
        html.H3("Phase 03: Interaktive Infrastruktur-Validierung &␣
 ↪API-Struktur-Vergleich", style={"margin": "0 0 10px 0", "color": "#2c3e50",␣
 ↪"fontWeight": "bold"}),
```

```
        html.P("Dieses Dashboard spiegelt die historische Ist-Struktur gegen␣
 ↪die offizielle Soll-Struktur der StaDa-API zur topologischen Visualisierung␣
 ↪von Netzlücken.", style={"margin": "0"})
    ], style={"backgroundColor": "#e3f2fd", "borderLeft": "5px solid #1e88e5",␣
 ↪"padding": "20px", "marginBottom": "25px", "fontFamily": "Segoe UI, Arial"}),

    # Filter-Zeile (Dropdown)
    html.Div([
        html.Div([
            html.Label("Filter: Corporate-Selektion des Bundeslandes",␣
 ↪style={"fontWeight": "bold", "fontSize": "14px", "color": "#2c3e50"}),
            dcc.Dropdown(
                id="filter-p3-state",
                options=[{"label": staat, "value": staat} for staat in␣
 ↪liste_dashboard_staaten_p3],
                value="Bayern",  # Startwert ist Bayern
                clearable=False,
                style={"marginTop": "5px"}
            )
        ], style={"width": "30%", "display": "inline-block"})
    ], style={"marginBottom": "25px"}),

    # Die interaktiven Registerkarten (Tabs)
    dcc.Tabs(id="tabs-p3-architektur", value="tab-p3-graphen", children=[
        # Tab 1: Jetzt mit der topologischen Landkarte
        dcc.Tab(label="Geografische Netzlücken (Landkarte)",␣
 ↪value="tab-p3-graphen", style={"padding": "12px", "fontWeight": "bold"},␣
 ↪selected_style={"padding": "12px", "fontWeight": "bold", "borderTop": "3px␣
 ↪solid #1e88e5"}),
        # Tab 2: Kompatibilitätstabelle
        dcc.Tab(label="Infrastruktur-Kompatibilitätstabelle",␣
 ↪value="tab-p3-tabelle", style={"padding": "12px", "fontWeight": "bold"},␣
 ↪selected_style={"padding": "12px", "fontWeight": "bold", "borderTop": "3px␣
 ↪solid #1e88e5"}),
    ], style={"fontFamily": "Segoe UI, Arial"}),

    # Dynamischer Container für den Inhalt der Tabs
    html.Div(id="tabs-p3-content", style={"padding": "25px 0"})

], style={"padding": "30px", "fontFamily": "Segoe UI, Arial, sans-serif",␣
 ↪"backgroundColor": "#ffffff"})
```

[129]:
```
# Callback-Steuerung mit dynamischer Geozentrierung
@app_phase3.callback(
    Output("tabs-p3-content", "children"),
    [Input("tabs-p3-architektur", "value"),
     Input("filter-p3-state", "value")]
```

```
)
def render_tab_inhalt(ausgewaehlter_tab, ausgewaehltes_land):
    # Filtern des Basisdatensatzes für das Bundesland
    df_land_fokus =␣
 ↪df_dashboard_p3_raw[df_dashboard_p3_raw["api_federal_state"] ==␣
 ↪ausgewaehltes_land]

    if ausgewaehlter_tab == "tab-p3-graphen":
        # Kontroll-Check: Entfernen von Datensätzen ohne gültige Koordinaten␣
 ↪für die Karte
        df_geo_plot = df_land_fokus.dropna(subset=["api_long", "api_lat"]).
 ↪copy()

        # Erstellung des Hover-Textes
        df_geo_plot["Hover_Text"] = "Kategorie: " + df_geo_plot["api_category"].
 ↪astype(str) + " | " + df_geo_plot["api_product_line"]

        # --- DYNAMISCHE GEOGRAFISCHE ZENTRIERUNG ---
        # Berechnung des exakten mathematischen Mittelpunkts des selektierten␣
 ↪Bundeslandes
        if not df_geo_plot.empty:
            zentrum_lat = df_geo_plot["api_lat"].mean()
            zentrum_long = df_geo_plot["api_long"].mean()

            # Dynamische Anpassung des Zoom-Faktors basierend auf der␣
 ↪Datendichte (Stadtstaaten vs. Flächenländer)
            # Stadtstaaten haben eine geringe geografische Streuung und␣
 ↪erfordern einen tieferen Zoom
            if ausgewaehltes_land in ["Berlin", "Hamburg", "Bremen"]:
                dynamischer_zoom = 9.5
            else:
                dynamischer_zoom = 6.5
        else:
            # Absoluter Fallback: Geografisches Zentrum der Bundesrepublik␣
 ↪Deutschland
            zentrum_lat = 51.165691
            zentrum_long = 10.451526
            dynamischer_zoom = 5.5

        # Generierung der Karte mit feingranular abgestimmten visuellen␣
 ↪Parametern
        fig_map = px.scatter_map(
            df_geo_plot,
            lon="api_long",
            lat="api_lat",
            color="Infrastruktur_Status",
```

```
            hover_name="api_station_name",
            hover_data={"Hover_Text": True, "api_long": False, "api_lat":␣
↪False, "Infrastruktur_Status": False},
            title=f"Topologische Analyse der Netzkastration: Datenpräsenz im␣
↪Raum {ausgewaehltes_land}",
            # Einbindung Farb-Ästhetik (Sanftes Grün vs. Kräftiges, dunkleres␣
↪Rot)
            color_discrete_map={
                "vollständig erfasst": "#2ecc71",
                "datenlücke / unterdrückt": "#c0392b"
            },
            zoom=dynamischer_zoom,
            center=dict(lat=zentrum_lat, lon=zentrum_long),
            template="plotly_white"
        )

        # Komplexe optische Verfeinerung der Marker-Eigenschaften (Größe,␣
↪Transparenz, Umrandung)
        fig_map.update_traces(
            marker=dict(
                size=9,
                opacity=0.75,
                allowoverlap=True    # Erlaubt sauberes Überlagern im dichten␣
↪Raum
            )
        )

        # Layout-Konfiguration des OpenStreetMap-Stils
        fig_map.update_layout(
            map_style="carto-positron",
            height=700,
            margin=dict(l=0, r=0, t=50, b=0),
            legend=dict(
                orientation="h",
                yanchor="bottom", y=0.02,
                xanchor="left", x=0.02,
                title_text="Infrastruktur-Status:",
                bgcolor="rgba(255, 255, 255, 0.78)"  # Legenden-Hintergrund
            )
        )
        return html.Div([dcc.Graph(figure=fig_map)])

    elif ausgewaehlter_tab == "tab-p3-tabelle":
        # --- Generierung Kompatibilitätstabelle (Kombinierte Sortierung) ---
        tabelle_daten = df_land_fokus[[
            "api_station_name", "api_city", "api_category", "api_eva_nr",␣
↪"Infrastruktur_Status"
```

```
        ]].sort_values(by=["api_category", "api_station_name"]).
↪to_dict("records")

        return html.Div([
            html.H5("Infrastruktur-Status-Matrix: Soll/Ist-Abgleich",␣
↪style={"color": "#2c3e50", "fontWeight": "bold", "marginBottom": "15px"}),
            dash_table.DataTable(
                id="tabelle-p3-systeme",
                columns=[
                    {"name": "Bahnhofsname (API)", "id": "api_station_name"},
                    {"name": "Stadt", "id": "api_city"},
                    {"name": "Kategorie", "id": "api_category"},
                    {"name": "Offizielle EVA-NR", "id": "api_eva_nr"},
                    {"name": "Infrastruktur-Status", "id":␣
↪"Infrastruktur_Status"}
                ],
                data=tabelle_daten,
                page_size=12,
                row_selectable="single",
                style_table={"overflowX": "auto", "border": "1px solid␣
↪#dcdde1"},
                style_header={"backgroundColor": "#f8f9fa", "fontWeight":␣
↪"bold", "borderBottom": "2px solid #dcdde1"},
                style_cell={"fontFamily": "Segoe UI, Arial", "padding": "12px",␣
↪"textAlign": "left"},
                style_data_conditional=[
                    {
                        "if": {"filter_query": "{Infrastruktur_Status} =␣
↪'vollständig erfasst'"},
                        "backgroundColor": "#e1f5fe", "color": "#0288d1"
                    },
                    {
                        "if": {"filter_query": "{Infrastruktur_Status} =␣
↪'datenlücke / unterdrückt'"},
                        "backgroundColor": "#ffebee", "color": "#c62828"
                    }
                ]
            ),
            html.Div(id="protokoll-p3-details", style={"marginTop": "30px"})
        ])
```

[130]:
```
# Interaktiver Zeilen-Callback für das Detail-Protokoll
@app_phase3.callback(
    Output("protokoll-p3-details", "children"),
    [Input("tabelle-p3-systeme", "derived_virtual_data"),
     Input("tabelle-p3-systeme", "derived_virtual_selected_rows")]
```

```
)
def aktualisiere_detail_protokoll(virtuelle_daten, ausgewaehlte_zeilen):
    if not ausgewaehlte_zeilen or not virtuelle_daten:
        return html.Div([
            html.P("Hinweis: Klicken Sie auf eine beliebige Station in der Tabelle, um die erweiterten API-Stammprotokolle einzusehen.", style={"fontStyle": "italic", "color": "#7f8c8d"})
        ], style={"backgroundColor": "#f8f9fa", "padding": "20px", "borderRadius": "5px", "border": "1px dashed #7f8c8d"})

    index = ausgewaehlte_zeilen[0]
    zeile_daten = virtuelle_daten[index]
    api_eva_nr = zeile_daten["api_eva_nr"]

    # Suchen der erweiterten Merkmale im Basis-DataFrame
    erweitert = df_dashboard_p3_raw[df_dashboard_p3_raw["api_eva_nr"] == api_eva_nr].iloc[0]

    return html.Div([
        html.H5(f"Detailanalyse & API-Stammprotokoll: {erweitert['api_station_name']} - {api_eva_nr}", style={"margin": "0 0 15px 0", "color": "#2c3e50", "fontWeight": "bold"}),
        html.Ul([
            html.Li([html.B("Infrastrukturelle Einordnung: "), f"Offizieller DB-Knotenpunkt (Kategorie {erweitert['api_category']})"]),
            html.Li([html.B("Systemstatus: "), f"{erweitert['Infrastruktur_Status'].upper()}"]),
            html.Li([html.B("Geografische Verankerung: "), f"Längengrad: {erweitert['api_long']}°E | Breitengrad: {erweitert['api_lat']}°N"]),
            html.Li([html.B("Logistisches Segment: "), f"{erweitert['api_product_line']} - {erweitert['api_segment']}"])
        ], style={"lineHeight": "1.8", "margin": "0"})
    ], style={"backgroundColor": "#f5f6fa", "border": "1px solid #dcdde1", "padding": "25px", "borderRadius": "5px"})
```

5.10.1 Dashboard starten

[131]:
```
# Server-Aktivierung
app_phase3.run(
    debug=False,
    port=DASH_PORT_PHASE3,
    jupyter_mode="inline",
    jupyter_width="95%",
    jupyter_height=1300,
)
```

```
<IPython.lib.display.IFrame at 0x15e803a5cd0>
```

5.10.2 Link für Webbrowser

Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht werden:

Dashboard im Browser öffnen

Das Dashboard wird lokal gestartet und ist nur erreichbar, solange die zugehörige Dash-Anwendung ausgeführt wird.

6 Phase 4: Optionale Netzwerkbetrachtung & Topologische Anschlussanalyse

Dieses finale Kapitel umfasst die im Projektvorschlag verankerte, fakultative Netzwerkbetrachtung des Schienenverkehrs. Da die zeitaufwendigen, kombinatorischen Vorberechnungen zu den realisierten Anschlusskonflikten bereits erfolgreich im PySpark-Hilfsnotebook implementiert und die Netzlücken in Phase 3 über die StaDa-API quantifiziert wurden, steht dieser Abschnitt unter dem Vorbehalt des verbleibenden Projekt-Zeitbudgets.

6.0.1 Technische Restriktionen der Timetable-Schnittstellen und strategische Eingrenzung

Im Rahmen der explorativen Analyse wurde evaluiert, ob eine vollständige Rekonstruktion und Reparatur der historischen Zugläufe über die offiziellen Fahrplan-Schnittstellen der Deutschen Bahn (Timetables-API) möglich ist. Hierbei zeigten sich gravierende informationstechnische Barrieren:

- **Echtzeit-Beschränkung der API:** Die Timetables-Schnittstelle liefert Daten systembedingt ausschließlich für ein enges, tagesaktuelles Zeitfenster (Stundentakt). Ein rückwirkender Abgleich oder eine systemische Anreicherung des historischen Primärdatensatzes ist technologisch somit ausgeschlossen.
- **XML-Fragmentierung und I/O-Sperren:** Fahrplandaten können über die REST-Schnittstelle nur restriktiv für einzelne Bahnhöfe separat abgefragt werden. Dies erzeugt eine massive Anzahl isolierter XML-Antworten.

Aus diesen Gründen und aufgrund des akuten Zeitmangels wird das optionale Vorhaben strikt eingegrenzt: Eine stichprobenartige Aufsammlung von XML-API-Antworten über ein eng definiertes Zeitfenster von ca. zwei bis drei Stunden wird ausschließlich für das abgegrenzte Teilnetz des Bundeslandes Bayern in Erwägung gezogen.

6.0.2 Methodischer Fokus bei der Graphen-Modellierung

Sollte dieses optionale Vorhaben im verbleibenden Zeitfenster operativ umgesetzt werden, erfolgt die mathematische Modellierung unter einer bewussten funktionalen Reduktion: Es werden **ausschließlich fahrplanmäßig planmäßige Ankunfts- und Abfahrtssequenzen** (Soll-Taktung) als Kanten in den topologischen Graphen einbezogen. Reale, dynamische Verspätungsminuten werden aus der Graphen-Struktur ausgeblendet, um innerhalb der verbleibenden Stunden eine stabile, lauffähige topologische Netzwerkkarte der bayerischen Schieneninfrastruktur zu erzeugen. Der Fokus liegt somit rein auf der Machbarkeit einer statischen Graphen-Modellierung unter Big-Data-Bedingungen.

6.0.3 HTML und PDF zu 05_timetable_api_ingestion.ipynb

Die vollständige Analyse des Notebooks `05_timetable_api_ingestion.ipynb` kann als PDF-Dokument geöffnet werden:

PDF des Notebooks öffnen

```
<IPython.lib.display.IFrame at 0x15e874a0ce0>
```

6.0.4 Topologische Graphen-Modellierung und fahrplanbasierte Netzwerkkonstruktion

Nach dem erfolgreichen Abschluss der relationalen XML-Sequenzierung im Hilfsnotebook 05 werden die konsolidierten Kantenbeziehungen (`05_results.parquet`) über die standardisierte Systemkonfiguration in das Hauptprojektnotebook importiert. Dieser finale Schritt führt die topologischen Strukturdaten der Phase 3 und Phase 4 zusammen, um ein mathematisches Abbild des bayerischen Schienennetzes als ungerichteten Graphen zu konstruieren.

Zur Gewährleistung einer maximalen Ressourceneffizienz und zur barrierefreien Ausführbarkeit für die Prüfungskommission wird in dieser Phase auf den Einsatz von Big-Data-Frameworks im Hauptnotebook verzichtet. Die Modellierung der Nachbarschaftsbeziehungen erfolgt über das hochspezialisierte NetworkX-Framework. Hierbei transformieren wir die extrahierten Soll-Verbindungen in ein mathematisches Kantennetzwerk, berechnen netzweite Zentralitätsmetriken und generieren eine statische Netzwerkkarte, um die topologischen Kernachsen der bayerischen Schieneninfrastruktur im Vormittags- und Mittagsverkehr empirisch offenzulegen.

[135]:
```
# Automatisierter Installationsschutz für das NetworkX-Framework im Notebook
try:
    import networkx as nx
except ImportError:
    print("Infrastruktur-Hinweis: Installiere fehlendes NetworkX-Framework im␣
 ↪Kursnetzwerk...")
    import subprocess
    subprocess.check_call([sys.executable, "-m", "pip", "install", "networkx"])
    import networkx as nx


# 1. Laden des hochverdichteten Graphen-Modells aus Phase 3/4 via Pandas
PARQUET_GRAPH_INPUT = get_project_relative_path(TIMETABLE_API_RESULTS)
print(f"Lade konsolidierte Kanten-Matrix von: {PARQUET_GRAPH_INPUT}")
df_edges_pandas = pd.read_parquet(PARQUET_GRAPH_INPUT)

# 2. Initialisierung des mathematischen Graphen-Objekts in NetworkX
G_bahn_netz = nx.Graph()

# Befüllen des Graphen mit den extrahierten physischen Kanten aus dem Dataframe
for _, zeile in df_edges_pandas.dropna(subset=["quell_station_name",␣
 ↪"ziel_station_name"]).iterrows():
    G_bahn_netz.add_edge(
        zeile["quell_station_name"],
```

```
        zeile["ziel_station_name"],
        linie=zeile["zug_linie"],
        kategorie=zeile["train_category"]
    )

# 3. Graphentheoretische Auswertung: Berechnung der Degree-Zentralität
zentralitaet_knoten = nx.degree_centrality(G_bahn_netz)
df_zentralitaet = pd.DataFrame(zentralitaet_knoten.items(), columns=["Station",␣
 ↪"Zentralitaetsgrad"])
df_zentralitaet = df_zentralitaet.sort_values(by="Zentralitaetsgrad",␣
 ↪ascending=False)

print("\n--- Graphentheoretische Analyse: Top 10 Infrastruktur-Knotenpunkte␣
 ↪(Degree-Zentralität) ---")
display(df_zentralitaet.head(10))

# 4. Generierung der topologischen Netzwerkkarte (Visualisierung)
print("\nGeneriere statische Netzwerkkarte des bayerischen Teilnetzes...")
plt.figure(figsize=(12, 10), dpi=100)

pos_layout = nx.spring_layout(G_bahn_netz, k=0.15, seed=42)

# Zeichnen der Netzelemente
nx.draw_networkx_nodes(G_bahn_netz, pos_layout, node_size=15,␣
 ↪node_color="#2c3e50", alpha=0.8)
nx.draw_networkx_edges(G_bahn_netz, pos_layout, width=0.5,␣
 ↪edge_color="#7f8c8d", alpha=0.4)

# Beschriftung der Top 10 zentralsten Hauptknotenpunkte zur Orientierung
top_10_knoten = df_zentralitaet.head(5)["Station"].tolist()
label_dict = {knoten: knoten for knoten in G_bahn_netz.nodes() if knoten in␣
 ↪top_10_knoten}

nx.draw_networkx_labels(G_bahn_netz, pos_layout, labels=label_dict,␣
 ↪font_size=9, font_weight="bold", font_color="#e74c3c")

plt.title("Topologische Schienenverkehrs-Netzwerkkarte (Teilnetz Bayern -␣
 ↪Stichprobe Phase 4)", fontsize=12, fontweight="bold", pad=20)
plt.axis("off")
plt.tight_layout()
plt.show()

print("\n" + "="*70)
print("Phase 4 (Graphen-basierte Netzwerkanalyse)")
print(f"Gesamtanzahl modellierter Netzknoten (Stationen): {G_bahn_netz.
 ↪number_of_nodes()}")
```

```
print(f"Gesamtanzahl modellierter Netzkanten (Verbindungen): {G_bahn_netz.
  ↪number_of_edges()}")
print("="*70)
```

```
Lade konsolidierte Kanten-Matrix von: outputs/05_results.parquet

--- Graphentheoretische Analyse: Top 10 Infrastruktur-Knotenpunkte (Degree-
Zentralität) ---
```

	Station	Zentralitaetsgrad
81	Nürnberg Hbf	0.028446
9	München-Pasing	0.020788
135	Würzburg Hbf	0.014223
8	München Hbf	0.013129
118	Freising	0.012035
45	Rosenheim	0.012035
180	Bamberg	0.012035
18	Augsburg Hbf	0.012035
37	Donauwörth	0.012035
234	Regensburg Hbf	0.012035

```
Generiere statische Netzwerkkarte des bayerischen Teilnetzes…
```




```
======================================================================
Phase 4 (Graphen-basierte Netzwerkanalyse)
Gesamtanzahl modellierter Netzknoten (Stationen): 915
Gesamtanzahl modellierter Netzkanten (Verbindungen): 1166
======================================================================
```

6.1 Dashboard Karte

[136]:
```
# DATEN-KONSOLIDIERUNG & GEODATEN-JOIN (Reines Pandas via EVA-IDs)
# Laden der fahrplanbasierten Kantenbeziehungen
df_edges_pandas = pd.read_parquet(TIMETABLE_API_RESULTS)

# KORRIGIERT: Erstellung einer eineindeutigen Geodaten-Lookup-Tabelle basierend␣
↪auf der EVA-ID
df_geo_lookup = df_api_referenz_stada[["api_eva_nr", "api_long", "api_lat"]].
↪drop_duplicates(subset=["api_eva_nr"])
geo_dict = df_geo_lookup.set_index("api_eva_nr").to_dict(orient="index")
```

```
# --- KNOTEN-AGGREGATION (KORRIGIERT: Trennung über Kombi aus ID und Name) ---
df_arrival_nodes = df_edges_pandas[["quell_eva_nr", "quell_station_name",␣
 ↪"train_category", "zug_linie"]].rename(
    columns={"quell_eva_nr": "station_eva_nr", "quell_station_name":␣
 ↪"station_name"}
)
df_departure_nodes = df_edges_pandas[["ziel_eva_nr", "ziel_station_name",␣
 ↪"train_category", "zug_linie"]].rename(
    columns={"ziel_eva_nr": "station_eva_nr", "ziel_station_name":␣
 ↪"station_name"}
)
df_nodes_combined = pd.concat([df_arrival_nodes, df_departure_nodes]).
 ↪dropna(subset=["station_eva_nr"]).drop_duplicates()

# Aggregation zwingend über ID und Name, damit die Münchner Flügelbahnhöfe␣
 ↪separat existieren bleiben
df_nodes_final = df_nodes_combined.groupby(["station_eva_nr", "station_name"]).
 ↪agg(
    zuggattungen=("train_category", lambda x: ", ".join(sorted(set(x)))),
    linien_liste=("zug_linie", lambda x: ", ".join(sorted(set(x)))),
    anzahl_linien=("zug_linie", "nunique")
).reset_index()

# Einkoppeln der echten geografischen Koordinaten via exaktem EVA-ID-Lookup
df_nodes_final["long"] = df_nodes_final["station_eva_nr"].map(lambda x:␣
 ↪geo_dict.get(x, {}).get("api_long", np.nan))
df_nodes_final["lat"] = df_nodes_final["station_eva_nr"].map(lambda x: geo_dict.
 ↪get(x, {}).get("api_lat", np.nan))
df_nodes_final = df_nodes_final.dropna(subset=["long", "lat"])
```

[137]:
```
# GENERIERUNG DER TOPOLOGISCHEN NETZWERKKARTE
fig_network = go.Figure()

# HINZUFÜGEN DER SCHIENEN-KANTEN (Verbindungslinien über EVA-IDs)
edge_lons = []
edge_lats = []

for _, kante in df_edges_pandas.dropna(subset=["quell_eva_nr", "ziel_eva_nr"]).
 ↪iterrows():
    q_geo = geo_dict.get(int(kante["quell_eva_nr"]), {})
    z_geo = geo_dict.get(int(kante["ziel_eva_nr"]), {})

    if q_geo and z_geo:
        edge_lons.extend([q_geo["api_long"], z_geo["api_long"], None])
        edge_lats.extend([q_geo["api_lat"], z_geo["api_lat"], None])
```

```
# Zeichnen der Netzkanten
fig_network.add_trace(go.Scattermap(
    lon=edge_lons,
    lat=edge_lats,
    mode="lines",
    line=dict(width=1, color="#7f8c8d"),
    hoverinfo="none",
    name="Fahrplan-Trassen (Kanten)"
))

# HINZUFÜGEN DER NETZKNOTEN
fig_network.add_trace(go.Scattermap(
    lon=df_nodes_final["long"],
    lat=df_nodes_final["lat"],
    mode="markers",
    marker=dict(
        size=df_nodes_final["anzahl_linien"] * 1.5 + 5,
        color="#2c3e50",
        opacity=0.85
    ),
    text=df_nodes_final["station_name"],
    # Hover-Fenster
    customdata=np.stack((df_nodes_final["zuggattungen"],␣
 ↪df_nodes_final["linien_liste"], df_nodes_final["anzahl_linien"]), axis=-1),
    hovertemplate=(
        "<b>Knotenpunkt: %{text}</b><br>"
        "Zuggattungen: %{customdata[0]}<br>"
        "Bediente Linien: %{customdata[1]}<br>"
        "Frequenz (Anzahl Linien): %{customdata[2]}<br>"
        "<extra></extra>"
    ),
    name="Bahnhöfe (Knoten)"
))

# DESIGN-LAYOUT
fig_network.update_layout(
    title=dict(
        text="Topologische Schienennetz-Karte Bayern (ID-optimierte Trennung␣
 ↪Phase 4)",
        font=dict(family="Segoe UI, Arial", size=16, color="#2c3e50"),
        x=0.5
    ),
    map=dict(
        style="carto-positron",  # Aktiviert den monochromen␣
 ↪Graustufen-Hintergrund
        zoom=6.8,
```

```
        center=dict(lat=48.8, lon=11.5)
    ),
    height=800,
    margin=dict(l=0, r=0, t=60, b=0),
    showlegend=False
)

# Minimaler Dashboard nur Karte
app_phase4_network = Dash(__name__)

app_phase4_network.layout = html.Div([
    html.Div([
        html.H4("Phase 04: Graphen-basierte Netzwerkanalyse & Topologische␣
↪Routenmodellierung", style={"margin": "0 0 5px 0", "color": "#2c3e50",␣
↪"fontWeight": "bold"}),
        html.P(f"Aktuell modelliertes bayerisches Teilnetzverkehrsaufkommen:␣
↪{len(df_nodes_final)} aktive Knotenpunkte.", style={"margin": "0",␣
↪"fontSize": "13px"})
    ], style={"backgroundColor": "#f8f9fa", "borderLeft": "5px solid #2c3e50",␣
↪"padding": "15px", "marginBottom": "15px", "fontFamily": "Segoe UI, Arial"}),

    dcc.Graph(figure=fig_network)
], style={"padding": "10px", "fontFamily": "Segoe UI, Arial"})
```

6.1.1 Dashboard starten

[138]:
```
app_phase4_network.run(
    debug=False,
    port=DASH_PORT_PHASE4,
    jupyter_mode="inline",
    jupyter_width="100%",
    jupyter_height=1050,
)
```

```
<IPython.lib.display.IFrame at 0x15e81bd0980>
```

6.1.2 Link für Webbrowser

Die Ergebnisse können zusätzlich in einem interaktiven Dashboard untersucht werden:

Dashboard im Browser öffnen

Das Dashboard wird lokal gestartet und ist nur erreichbar, solange die zugehörige Dash-Anwendung ausgeführt wird.

7 Abschlussbetrachtung, Erkenntnissynthese und informationstechnischer Ausblick

Die weitreichende quantitative und topologische Analyse der Phasen 1 bis 4 führt die im Kurs erlernten Big-Data-Methodiken in einer ehrlichen, datenkritischen Gesamtsynthese zusammen. Das Zusammenspiel aus historischer Datenbasis (Phase 2), StaDa-API-Validierung (Phase 3) und der fahrplanbasierten Graphen-Modellierung (Phase 4) liefert tiefgreifende Erkenntnisse über die Struktur und die informationstechnischen Limitierungen moderner Mobilitätsdaten.

7.1 Empirische Befunde und topologische Anomalien im bayerischen Schienennetz

Die Ausweitung des lokalen XML-Caches im Hilfsnotebook `05_timetable_api_ingestion.ipynb` auf ein sechsstündiges Zeitfenster (12:00 bis 18:00 Uhr) mit über **6.500 erfolgreich aggregierten REST-Requests** validiert das topologische Netzwerkmodell auf einem robusten Fundament von **915 aktiven Netzknoten und 1.166 physischen Kantenverbindungen**.

Trotz dieses massiven Datenvolumens offenbart die geografische Dashboard-Visualisierung eine signifikante, unumstößliche Wahrheit: **Bestimmte bayerische Schienenachsen und regionale Strecken fehlen im operativen Datenstrom der offiziellen Deutsche Bahn APIs vollständig.** Die kausale Ursache dieser strukturellen Blindheit in der Fläche (insbesondere im ostbayerischen Raum) lässt sich systemseitig nicht eindeutig isolieren. Das schrittweise Nachladen und Einbinden weiterer Zeitslots hat dieses Ergebnis nachweislich nicht beeinflusst. Diese Netzkastration muss als inhärentes Qualitätsdefizit der zugrundeliegenden Schnittstellenarchitektur deklariert werden.

Ein herausragender methodischer Erfolg ist hingegen die präzise Auflösung der innerstädtischen Metropoleninfrastruktur. Durch den Verzicht auf ungenaue textbasierte Joins und den konsequenten Einsatz einer id- und indexbasierten PySpark-Vorauswertung konnten die fragmentierten Münchner Satelliten-Bahnhöfe (*München Hbf (tief)*, *München Hbf Gl.27-36* etc.) lückenlos als eigenständige, funktionale Netzknoten auf der Karte etabliert werden.

Die mathematische Auswertung der *Degree Centrality* (Zentralitätsgrad) liefert hierbei ein bahnbetrieblich hochgradig präzises Bild: 1. **Nürnberg Hbf (Zentralität: 0.0284)** und **München-Pasing (Zentralität: 0.0208)** thronen unangefochten an der Spitze des bayerischen Gesamtnetzes. Dies spiegelt die topologische Realität exakt wider: Als hochfrequente Durchgangsbahnhöfe weisen sie naturgemäß eine komplexere, sternförmige Trassenverknüpfung auf als der reine Kopfbahnhof *München Hbf* (Zentralität: 0.0131), an dem Verkehrsströme primär enden. 2. Die Bereinigung der Datenströme stellt sicher, dass künstlich aufgeblähte innerstädtische S-Bahn-Haltestellen in der finalen Bewertung nicht mehr fälschlicherweise eine höhere netzweite Relevanz zugewiesen bekommen als die zentralen logistischen Hauptknotenpunkte des Landes (wie *Würzburg Hbf* oder *Regensburg Hbf*).

7.2 Informationstechnischer Ausblick und zukünftige Forschungsansätze

Für zukünftige, anschließende Projektvorhaben öffnet diese Arbeit das Tor für vielversprechende informationstechnische Forschungsansätze:

- **Entwicklung von Failsafe-Hybrid-Parsern:** Um die detektierten Schnittstellenlücken in der Fläche (Grenzregionen, Landbahnhöfe) dauerhaft zu schließen, ist in einer nächsten Aus-

baustufe die Implementierung eines hybriden Daten-Ingestors ratsam. Dieser sollte die hierarchischen XML-Strukturen der *Timetables-API* in Echtzeit mit unstrukturierten, statischen Fahrplandaten relational verschmelzen, um fehlende Trassen autonom zu rekonstruieren.

- **Graphenbasierte Verspätungskaskadierung:** Auf Basis des nun stabilisierten statischen Graphenmodells lässt sich in zukünftigen Big-Data-Pipelines eine prädiktive Verspätungssimulation aufsetzen. Unter Einsatz von Graph Neural Networks oder parallelisierten Breitensuchen (BFS) in PySpark GraphFrames könnte mathematisch modelliert werden, wie sich eine dynamische Primärverspätung an einem zentralen Nadelöhr (z. B. Nürnberg Hbf) topologisch über die Kanten als Sekundärverspätung in den ländlichen Raum hinein fortpflanzt.

Das Projekt beweist damit eindrucksvoll, dass erst die Kombination aus unbestreitbarer datenkritischer Distanz und modernen relationalen Analyse-Frameworks in der Lage ist, rohe Infrastrukturdaten in eine valide, belastbare Entscheidungsvorlage für die Mobilität der Zukunft zu transformieren.

7.3 Projektumgebung

```
======================================================================
SYSTEM-INSPEKTION: Dokumentation der aktiven Projektumgebung
======================================================================
Registrierte Python-Version:    3.12.10
Aktive JupyterLab-Version:     4.6.3

--- Versionen der geschäftskritischen Projekt-Bibliotheken ---
-> pandas         : 3.0.5
-> pyspark        : 4.2.0
-> dash           : 4.4.1
-> plotly         : 7.0.0
-> networkx       : 3.7
-> pyarrow        : 25.0.1
-> requests       : 2.34.2
-> python-dotenv  : 1.2.3
-> altair         : 6.2.2
======================================================================
```

[ ]: